

Alternate-Eye Rendering and Optical-Flow Frame Generation

A Technical Implementation Guide for Injected VR Renderers

Implementation reference for D3D11 / D3D12 / Vulkan targets under OpenXR

Revision 1.0

Contents

1	Scope, Audience, and Method	5
1.1	What this document is	5
1.2	What this document is not	6
1.3	Prerequisites	6
1.4	Notation	6
2	The Stereo Cost Problem	6
2.1	Why stereo is not free, and not exactly 2x	6
2.2	The two rates that matter	7
2.3	Perceptual budget	8
3	Reference Architecture	8
3.1	The layer stack	8
3.2	The three clocks	9
3.3	Where the camera lives, and how you find it	9
3.4	The synthesis pipeline	10
4	OpenXR Integration Essentials	10
4.1	The frame loop and what it guarantees	10
4.2	Building the projection matrix from XrFovf	11
4.3	Swapchain management	11
4.4	Depth submission and why it matters	12
4.5	Reprojection interaction: don't fight the runtime	12
5	Camera Interception Across Three APIs	12
5.1	The fingerprinting problem	12
5.2	D3D11 interception	14
5.3	D3D12 interception	15
5.4	Vulkan interception	16
5.5	Choosing which draws to modify	17
6	Alternate-Eye Rendering: Theory	17
6.1	The core idea	17
6.2	Temporal disparity error	18
6.3	Eye scheduling policies	18
6.4	Half-rate versus alternate: an important distinction	19
7	Alternate-Eye Rendering: Implementation	20
7.1	The per-frame flow	20
7.2	Applying the eye transform	20
7.3	Capturing the depth buffer	21
7.4	Swapchain handling for the held eye	22
7.5	The projection layer	22
8	The AER Artifact Taxonomy	22
8.1	A1: Temporal disparity shear	23
8.2	A2: Inter-ocular judder (retinal rivalry)	23
8.3	A3: Specular and view-dependent shading breakup	23
8.4	A4: Screen-space effect inconsistency	23
8.5	A5: Temporal antialiasing interaction	24
8.6	A6: Vergence-accommodation aggravation	24

8.7	A7: Flicker resonance	24
8.8	Severity ranking	24
9	AER v2: Motion-Compensated Synthesis	25
9.1	The idea	25
9.2	Why the naive stereo warp is nearly free	25
9.3	Two variants of v2, and which one to build	25
9.4	The disocclusion problem	25
9.5	Implementing the warp: forward versus backward	27
9.6	Handling the near-field cutout	28
9.7	Cost accounting for AER v2	29
10	Frame Generation: Fundamentals and Latency	29
10.1	The causality constraint	29
10.2	Extrapolation as the alternative	29
10.3	What “generation” must actually produce	30
10.4	Latency budget accounting	31
11	Motion Field Estimation	31
11.1	Where motion comes from	31
11.2	Hierarchical block matching	32
11.3	Regularisation and confidence	34
11.4	Cost budget	35
12	Warping, Occlusion, and Compositing	35
12.1	The warp target	35
12.2	Forward warping with depth ordering	36
12.3	Detecting and handling stretch	36
12.4	Bidirectional warping for interpolation	37
12.5	Hole filling	37
12.6	Warping the depth buffer	38
13	The Combined Pipeline	38
13.1	Ordering: AER before generation	38
13.2	The unified warp	39
13.3	Effective rate multiplication	39
13.4	Degradation ladder	40
14	UI, HUD, and Exclusion Handling	40
14.1	Why UI breaks everything	40
14.2	Extraction	40
14.3	Re-presentation options	41
14.4	Exclusion masks for the warp	41
15	Synchronization and Scheduling	42
15.1	The concurrency problem	42
15.2	Async compute	42
15.3	Avoiding the pipeline bubble	43
16	Measurement and Validation	44
16.1	Instrumentation	44
16.2	Correctness validation	44
16.3	Artifact-specific test scenes	45
16.4	Subjective evaluation	45
17	Failure Modes and Practical Hardening	45
17.1	The things that will break	45
17.2	Numerical hazards	46
17.3	Configuration surface	46
17.4	Ethical and legal notes	46
18	The Complete Stereo Mode Taxonomy	47
18.1	The three axes	47
18.2	The resulting modes	47
18.3	The key insight about M1	48

18.4	Choosing between the families	48
19	Synchronized Sequential Stereo	48
19.1	The target architecture	48
19.2	Tick locking	49
19.3	The double render invocation	51
19.4	Paired submission	53
19.5	Latency analysis	53
19.6	When same-tick sequential is the right choice	54
19.7	Combining with temporal generation	54
19.8	Definition and motivation	54
19.9	Architecture	54
19.10	The skew problem	55
19.11	Tick identity and matched poses	56
19.12	Interaction with the runtime	57
19.13	When async AER is the right choice	57
20	Engine Re-entrancy: Making a Renderer Run Twice	57
20.1	The three families	57
20.2	Why Family A (replay) fails	57
20.3	Why Family B (re-invoke) fails	58
20.4	Why Family C (native second view) fails	58
20.5	The deferred renderer wall	59
20.6	The decision procedure	60
20.7	Cost of true simultaneous stereo	60
20.8	What it is	60
20.9	Why it works so well	61
20.10	The math	61
20.11	Pose prediction	62
20.12	Combining AFW with the stereo warp	64
20.13	Warping the fresh eye too	64
20.14	Cost	64
21	Hardware Optical Flow and Vendor Interop	65
21.1	What the hardware provides	65
21.2	Integration architecture	65
21.3	Using the flow output	67
21.4	Vendor-specific practical hazards	69
21.5	Graceful fallback	70
22	Engine Motion Vectors and Upscaler Integration	70
22.1	Why engine motion vectors are worth the trouble	70
22.2	Finding them via the upscaler	70
22.3	The three conventions you must resolve	71
22.4	Jitter	72
22.5	Combining engine vectors with estimated flow	72
22.6	Resolution mismatch	72
22.7	The cost model	72
22.8	Quality model	74
22.9	Decision table	74
22.10	Summary of what each technique buys	75
23	The Mode-Switching Runtime	75
23.1	State and transitions	75
23.2	Transition safety	76
23.3	Capability probing at startup	76
24	Bring-Up Order	77
24.1	Stage 1 — Mono in the headset	77
24.2	Stage 2 — Camera control	77
24.3	Stage 3 — Projection control	77
24.4	Stage 4 — Depth capture	77
24.5	Stage 5 — Naive AER (M2)	77
24.6	Stage 6 — Alternate frame warp (M5)	77
24.7	Stage 7 — Stereo warp (M4)	77

24.8	Stage 8 — Tick lock and M1	77
24.9	Stage 9 — Frame generation (M6)	77
24.10	Stage 10 — Mode switching and hardening	77
25	Extended Failure Catalogue	78
25.1	Tick-lock failures	78
25.2	Async AER failures	78
25.3	Warp and generation failures	78
25.4	Mode-switching failures	78
25.5	Why a census	79
25.6	Stable pass identity	79
25.7	What to record	80
25.8	The two probes that classify everything	80
25.9	Deriving the classification	81
25.10	Persisting and shipping the census	81
25.11	Census-driven decisions	82
26	Temporal Upscaling and Antialiasing Under Stereo Amortisation	82
26.1	The structural problem	82
26.2	Per-eye history separation	82
26.3	Motion vector scaling	83
26.4	Jitter sequence management	83
26.5	Upscaling and the warp	83
26.6	Sharpness alternation	84
27	Canted Displays and Non-Standard Frusta	84
27.1	The problem	84
27.2	Detection	84
27.3	Handling it correctly	84
27.4	Parallel projection mode	85
27.5	Other non-standard frusta	85
28	Foveation and Peripheral Quality	85
28.1	Why it interacts with everything here	85
28.2	Two-zone foveation: the cheapest effective form	85
28.3	Graduated foveation in the warp	86
28.4	Eye-tracked foveation	87
28.5	Peripheral degradation as a degradation-ladder step	87
29	Near-Field Content: Weapons, Hands, and Cockpits	87
29.1	Why the near field dominates artifact severity	87
29.2	Detecting the near-field pass	87
29.3	Handling strategy by mode	88
29.4	Depth-range unification	88
29.5	Cockpits and large near-field geometry	89
29.6	The gather formulation	89
29.7	Bidirectional sampling and why it matters	89
29.8	The weight cascade	90
29.9	Extrapolation in the gather formulation	92
29.10	Gather versus scatter: when to use which	93
30	Robust Motion Fusion by Agreement Gating	93
30.1	The principle	93
30.2	Implementation	93
30.3	Applying it to both auxiliary sources	94
30.4	Why this beats calibration	95
30.5	Diagnostics	95
30.6	Generalising the pattern	95
30.7	Projection matrices	96
30.8	Stereo disparity	96
30.9	AER temporal error	96
30.10	Disocclusion gap width	96
30.11	Reprojection	96
30.12	Off-screen margin	96
30.13	Speedup model	97

30.14	Tick-lock and same-tick stereo	97
30.15	Asynchronous AER skew	97
30.16	Alternate frame warp	97
30.17	Pose prediction	97
30.18	Geometry-shader stereo injection	98
30.19	Hardware flow decoding	98
30.20	Combined mode cost	98
30.21	Composite quality score	98
31	Appendix B: Glossary	98
32	Appendix C: Reading List	99
33	Appendix D: Reference Warp Implementation	100
33.1	Host-side parameter construction	100
33.2	The warp vertex shader	101
33.3	The warp pixel shader	102
33.4	The rotation-only gather path	103
33.5	Dispatch	103
33.6	Grid index buffer construction	104
34	Appendix E: Configuration Reference	104
34.1	World and camera	104
34.2	Mode selection	105
34.3	Tick lock	105
34.4	Alternate-eye and warp	105
34.5	Frame generation	105
34.6	Foveation	106
34.7	UI	106
34.8	Diagnostics	106
35	Appendix F: Symptom to Cause	106
35.1	Geometry and stereo	106
35.2	Motion and temporal	107
35.3	Synthesis artifacts	107
35.4	Performance and stability	107
35.5	UI	108
36	Appendix G: Test Scene Specifications	108
36.1	T1 — Depth-staggered poles (disparity shear)	108
36.2	T2 — Oblique checkerboard (binocular rivalry)	108
36.3	T3 — Polished sphere (specular breakup)	109
36.4	T4 — Reflective floor with edge-crossing object (screen-space effects)	109
36.5	T5 — Fine foliage (temporal antialiasing)	109
36.6	T6 — Strobe and rotor (flicker resonance)	109
36.7	T7 — Near pole against distant wall (disocclusion)	109
36.8	T8 — Fast small object over flat background (flow failure)	109
36.9	T9 — Rapid turn (off-screen intake)	109
36.10	T10 — Dense HUD (UI handling)	109
36.11	Harness structure	110

1 Scope, Audience, and Method

1.1 What this document is

This is an implementation guide for two techniques used to make demanding real-time applications tractable in virtual reality when the application itself was never designed for stereo rendering:

1. **Alternate-Eye Rendering (AER)** — amortising the cost of stereo by rendering one eye per application frame and synthesising the other, in several progressively more sophisticated variants. We refer to the naive form as **AER v1** and the motion-compensated form as **AER v2**.
2. **Optical-flow frame generation** — synthesising intermediate or extrapolated frames from a dense motion field derived from previously rendered frames, in order to decouple the display refresh rate from the application's render rate.

The target context is an **injected renderer**: code that attaches to a running application that has no knowledge of VR, intercepts its graphics API calls, rewrites its camera state, and re-presents its output through an OpenXR runtime. This is a substantially

harder environment than an engine-integrated implementation, because almost nothing about the application’s rendering is known a priori, and everything useful must be inferred at runtime.

Coverage spans **Direct3D 11**, **Direct3D 12**, and **Vulkan** on the graphics side, and **OpenXR** on the runtime side. Where the three APIs differ materially — and they differ most in resource state management and synchronisation — each is treated separately.

1.2 What this document is not

It is not a description of any particular existing product, and it does not contain any material derived from disassembly, decompilation, or inspection of third-party binaries. Everything here is developed from published literature, publicly documented API behaviour, and first principles. Where a technique has a well-known prior art lineage — asynchronous timewarp, spacewarp, motion-compensated frame interpolation in video coding — the lineage is cited so you can read the primary sources.

It is also not a copy-paste codebase. The listings are deliberately reduced to the essential logic. Production code needs error handling, device-loss recovery, and capability negotiation that would triple the length of every listing without teaching anything.

1.3 Prerequisites

You should be comfortable with: homogeneous coordinates and projective transforms; the structure of a modern deferred or forward+ renderer; GPU compute shaders and memory barriers; and low-level function interception (import table patching, virtual table patching, or trampoline hooks). Familiarity with the OpenXR frame loop is assumed but the relevant parts are restated in Chapter 4.

1.4 Notation

Symbol	Meaning
V	View matrix (world $\rightarrow$ view)
P	Projection matrix (view $\rightarrow$ clip)
M	Model / world matrix
$\mathbf{p}_w, \mathbf{p}_v, \mathbf{p}_c$	Point in world, view, clip space
$\mathbf{u}$	Screen-space coordinate, normalised to $[0, 1]^2$
z_n, z_f	Near and far plane distances
d	Post-projective depth (the value in the depth buffer)
b	Interpupillary distance (IPD), metres
f_a	Application render rate (Hz)
f_d	Display refresh rate (Hz)
T_{mp}	Motion-to-photon latency
$\mathbf{F}_{t \rightarrow t'}$	Dense flow field mapping frame t to frame t'

Row-vector convention is used throughout ($\mathbf{p}' = \mathbf{p}MVP$), matching Direct3D’s historical convention and HLSL’s `mul(v, M)` ordering. Vulkan/GLSL users should transpose. Depth is assumed reversed (1 at near, 0 at far) unless stated, because reversed-Z with a floating-point depth buffer is now the default in essentially every renderer worth injecting into, and it materially changes the precision analysis in Chapter 9.

2 The Stereo Cost Problem

2.1 Why stereo is not free, and not exactly 2x

The naive model says stereo rendering costs twice as much as mono. Reality is more interesting, and the deviations are exactly what AER exploits.

Decompose a frame’s GPU cost into three buckets:

- **View-independent work** (C_i): shadow map rendering, skinning, particle simulation, GPU culling of world-space structures, environment probe updates, cubemap convolution, virtual texture feedback processing, global illumination updates. This work depends on scene state, not on where the camera is.
- **View-dependent work** (C_v): the geometry pass, lighting, screen-space effects, post-processing. This scales with the number of views.
- **Fixed overhead** (C_f): command list construction, resource transitions, present, driver overhead.

Total mono cost is $C_i + C_v + C_f$. Naive stereo is $C_i + 2C_v + C'_f$, where $C'_f > C_f$ because you now submit roughly twice the draw calls. The stereo multiplier is therefore:

$$\alpha = \frac{C_i + 2C_v + C'_f}{C_i + C_v + C_f}$$

For a modern deferred renderer with heavy shadow casting and expensive post, C_i can be 25–40% of frame time, so α lands around 1.6–1.75 rather than 2.0. In a CPU-bound title where C_f dominates, the naive injected implementation can be *worse* than

2x, because injection typically forces a second full submission of the command stream without the engine’s own instancing or multi-view optimisations.

This matters for AER because **AER only saves C_v** . If your target application is CPU-bound on draw submission, or dominated by view-independent work, AER’s ceiling is low and you should measure before building.

2.1.1 Measuring the split empirically

You cannot ask the application what its C_i is, but you can bound it. Insert timestamp queries around every render pass you can identify by its render-target signature, then run the application at two different resolutions — say 100% and 50% linear scale. Passes whose duration scales roughly with pixel count are view-dependent; passes whose duration is invariant are view-independent or CPU-bound. A 4x pixel-count change is a strong enough lever to classify most passes in a single session.

```
// D3D11 sketch: per-pass GPU timing via timestamp disjoint queries.
struct PassTimer {
    ID3D11Query* disjoint;
    ID3D11Query* begin;
    ID3D11Query* end;
};

void BeginPass(ID3D11DeviceContext* ctx, PassTimer& t) {
    ctx->Begin(t.disjoint);
    ctx->End(t.begin);          // timestamp queries use End() only
}

void EndPass(ID3D11DeviceContext* ctx, PassTimer& t) {
    ctx->End(t.end);
    ctx->End(t.disjoint);
}

// Resolve on a later frame to avoid stalling:
double ResolveMs(ID3D11DeviceContext* ctx, PassTimer& t) {
    D3D11_QUERY_DATA_TIMESTAMP_DISJOINT dj{};
    if (ctx->GetData(t.disjoint, &dj, sizeof(dj), 0) != S_OK || dj.Disjoint)
        return -1.0;
    UINT64 a = 0, b = 0;
    ctx->GetData(t.begin, &a, sizeof(a), 0);
    ctx->GetData(t.end, &b, sizeof(b), 0);
    return 1000.0 * double(b - a) / double(dj.Frequency);
}
```

Identify passes by hashing their render-target descriptors plus the bound pixel shader’s bytecode hash. This gives you a stable pass identity across frames without any engine knowledge — a technique you will reuse constantly throughout this document.

2.2 The two rates that matter

VR imposes two separate requirements that people routinely conflate:

Throughput. The compositor must be handed a fresh image every display interval. At 90 Hz that is one image per 11.1 ms; at 120 Hz, 8.33 ms. Miss it and the runtime either reprojects the previous frame (visible judder on animation) or drops to a lower effective rate.

Latency. The angular error between where the head actually is when photons are emitted and where the renderer assumed it was. This is the motion-to-photon time T_{mp} . Angular error is $\omega \cdot T_{mp}$ for head angular rate ω . A brisk head turn is 200–400 deg/s; peak saccadic head motion exceeds 500 deg/s. At 300 deg/s, every 10 ms of latency is 3 degrees of world swim.

These are decoupled. You can have 120 Hz throughput with 60 ms latency (deep pipelining) or 30 Hz throughput with 12 ms latency (late-latch with heavy reprojection). Every technique in this document trades one against the other, and the *first design decision you make* should be which one your target application is starved of.

Technique	Throughput	Latency	Spatial fidelity
Naive stereo	Baseline	Baseline	Reference
AER v1	~1.4–1.8x	Unchanged	Degraded (stale eye)
AER v2	~1.4–1.8x	Unchanged	Mildly degraded
Runtime reprojection (ATW)	Fills gaps	Improves rotational	Positional artifacts
Optical-flow interpolation	2x	Worsens by 1 frame	Warp artifacts
Optical-flow extrapolation	2x	Unchanged or better	Worse warp artifacts

That penultimate row is the single most important line in this document and the one most often missed. **True interpolation cannot be causal.** To interpolate a frame between t and $t + 1$, you must have frame $t + 1$, which means you must withhold frame t from the display until frame $t + 1$ exists. That is a mandatory added latency of one full application frame interval. In flat gaming this is a tolerable trade. In VR, adding 16 ms of motion-to-photon latency is often worse than the judder you were trying to fix. Chapter 10 treats this in detail and describes the extrapolation-based alternative that avoids it.

2.3 Perceptual budget

Some empirical anchors from the human-factors literature, useful as design targets:

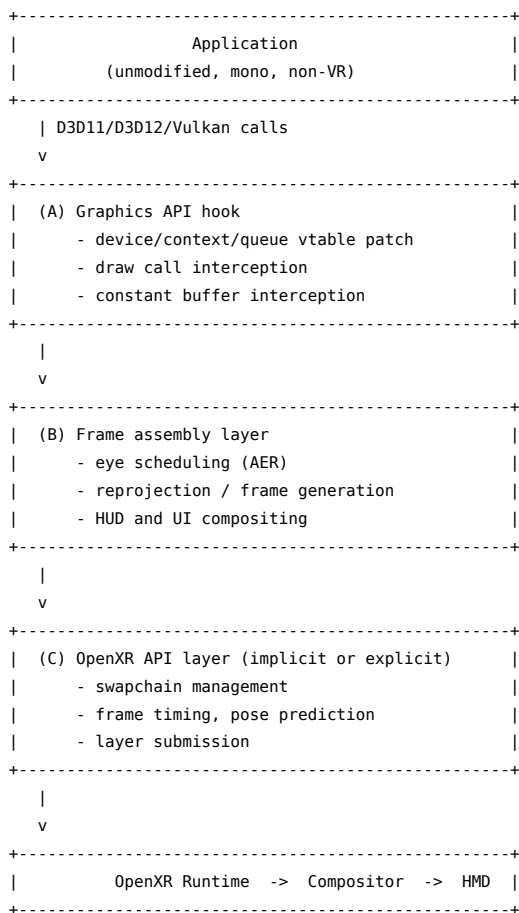
- **Vernier acuity** for stereo disparity in the fovea is on the order of 10–30 arcseconds under ideal conditions. Practically, disparity errors below roughly 1 arcminute are invisible in a consumer HMD, because pixel pitch is coarser than that anyway (a typical 2064x2208-per-eye panel over ~100 deg FOV is around 1.7 arcmin/pixel).
- **Binocular rivalry** onset: sustained inter-ocular differences in luminance pattern of more than a few percent over regions larger than ~1 degree provoke rivalry and viewer discomfort. Transient differences lasting under ~20 ms are largely fused.
- **Judder detection** threshold for a tracked moving object is roughly 1–2 pixels of positional discontinuity per display interval at typical HMD pixel pitch.
- **Latency detection** threshold with a head-locked reference is around 15–20 ms for most people; sensitivity to *variance* in latency is considerably higher than to its absolute value.

The 20 ms fusion window is the key number for AER. It says that if you alternate eyes at 90 Hz — so each eye is refreshed every 22.2 ms — you are sitting right at the edge of the fusion window, which is precisely why AER v1 is usable at all and precisely why it becomes unusable below about 60 Hz application rate.

3 Reference Architecture

3.1 The layer stack

An injected VR renderer sits between an unmodified application and the OpenXR runtime. Conceptually there are four interception surfaces, and choosing among them determines almost everything else about your implementation.



Surface (A) is where camera manipulation happens. Surface (B) is where AER and frame generation live. Surface (C) is where you negotiate with the runtime for the swapchain formats and timing you need.

A crucial architectural decision: **should (B) live in your hook DLL or in a separate OpenXR API layer?** Both work. Putting

it in an API layer (a DLL registered via the `XR_API_LAYER_PATH` mechanism, exposing `xrNegotiateLoaderApiLayerInterface`) gives you cleaner separation and lets your frame-generation stage operate on submitted swapchain images regardless of which application produced them. Putting it in the hook gives you access to the depth buffer and motion vectors that the application never submits to OpenXR, which you absolutely need for good results.

The recommendation is **hybrid**: do all synthesis in the hook, where you have depth and G-buffer access, and use a thin API layer only if you need to override runtime behaviour such as disabling the runtime's own reprojection.

3.2 The three clocks

Injected VR renderers fail in confusing ways because three independent clocks get conflated:

1. **Simulation clock** — the rate at which the application advances game state. Often coupled to the render loop, sometimes fixed-step with interpolation.
2. **Render clock** — the rate at which the application produces complete frames. This is what you observe at `Present/vkQueuePresentKHR`.
3. **Display clock** — the HMD refresh, driven by the runtime via `xrWaitFrame`.

Your job is to make (3) always satisfied while (2) runs slower, and to prevent (1) from being disturbed by your manipulations. That last constraint is subtle: many applications derive their simulation timestep from the wall-clock duration of the previous frame. If your frame-generation stage adds GPU work that lengthens the frame, the application will see a longer delta-time and may change its physics behaviour. Worse, if you block the application's render thread inside your hook to synchronise with `xrWaitFrame`, you can induce a feedback oscillation where the application's adaptive quality system fights your pacing system.

Mitigation: never block the application's render thread on display timing. Decouple with a queue. The application renders into your ring buffer as fast as it can; a separate submission thread services `xrWaitFrame/xrBeginFrame/xrEndFrame` and picks whichever image is freshest, synthesising as needed.

```
// Decoupled architecture, in outline.
struct RenderedFrame {
    GpuTexture color;           // application's final color, mono, one eye
    GpuTexture depth;           // captured depth for reprojection
    Eye eye;                     // which eye this was rendered for
    XrPosef cameraPose;         // pose used to build the view matrix
    Mat4 view, proj;
    uint64_t gpuFenceValue;     // signals completion
    double cpuTimestamp;
};

class FrameRing {
    static constexpr int N = 4;
    RenderedFrame slots[N];
    std::atomic<uint64_t> writeIndex{0};
    std::atomic<uint64_t> readIndex{0};
public:
    RenderedFrame* AcquireForWrite(); // called from app render thread
    void PublishWrite();               // makes it visible to submitter
    const RenderedFrame* LatestComplete(); // called from submission thread
    const RenderedFrame* PreviousComplete(); // needed for flow estimation
};
```

Four slots is the practical minimum: one being written by the application, one being read by the flow estimator as the “previous” frame, one being read as the “current” frame, and one in flight on the compositor.

3.3 Where the camera lives, and how you find it

The single hardest problem in injected VR is not stereo or frame generation. It is **reliably identifying and rewriting the application's view and projection matrices** every frame, in an application whose renderer you did not write.

There are four viable strategies, in decreasing order of robustness and increasing order of generality:

1. **Known-offset patching.** If the application's binary is known and versioned, locate the camera structure in memory by pattern scan and write to it directly. Maximally robust, zero per-frame cost, completely non-portable. Not applicable to the “any game” target, but worth knowing because it is the fallback when everything else fails.
2. **Constant buffer fingerprinting.** Intercept `Map/Unmap` (D3D11), or the upload path (D3D12/Vulkan), scan the written bytes for 4x4 float matrices, and classify them by structure. A view-projection matrix has a recognisable signature. This is the primary technique and Chapter 5 develops it fully.
3. **Draw-call heuristics.** Identify the main scene pass by its render target dimensions, depth-stencil binding, draw call count, and primitive count. Then identify which constant buffer slot changes per-view versus per-object. Works well; requires per-application tuning of thresholds.
4. **Depth-buffer inversion.** If you cannot find the projection matrix, you can sometimes *recover* it from the depth buffer statistics,

by fitting the depth distribution to a known projection form. Fragile, but useful as a validator.

In practice a production implementation runs (2) and (3) together, cross-validating, with a confidence score that gates whether stereo is enabled at all. Rendering stereo with a misidentified projection matrix is far worse than rendering mono — it produces a violently incorrect disparity that is genuinely nauseating.

3.4 The synthesis pipeline

Once camera control is established, the per-display-frame pipeline looks like this:

```
[App frame N] ---> capture color + depth + camera state
|
v
[Eye scheduler] --- decides: this app frame is LEFT (or RIGHT)
|
+--> rendered eye: pass through (possibly with timewarp)
|
+--> opposite eye: synthesise
|       |
|       +-- AER v1: reuse previous frame's opposite-eye image
|       +-- AER v2: reproject rendered eye by stereo disparity,
|                   plus temporal motion compensation
v
[Frame generation stage] --- if display rate > app rate
|
+-- estimate flow between app frames N-1 and N
+-- warp to intermediate/extrapolated time
|
v
[Composite] --- UI layer, reticle, comfort vignette
|
v
[OpenXR submission] --- xrEndFrame with projection layer
```

Note the ordering: **AER before frame generation**. The frame generator operates on stereo pairs, not mono frames, because the flow field differs between eyes and because generating mono then applying AER would compound the errors. Chapter 12 discusses the alternative ordering and why it is usually wrong.

4 OpenXR Integration Essentials

4.1 The frame loop and what it guarantees

The OpenXR frame loop is three calls per display frame:

```
XrFrameState frameState{XR_TYPE_FRAME_STATE};
xrWaitFrame(session, nullptr, &frameState);
// frameState.predictedDisplayTime -- when photons will hit the eye
// frameState.predictedDisplayPeriod -- nominal display interval
// frameState.shouldRender -- false if session is not visible

xrBeginFrame(session, nullptr);

// ... render ...

XrFrameEndInfo endInfo{XR_TYPE_FRAME_END_INFO};
endInfo.displayTime = frameState.predictedDisplayTime;
endInfo.environmentBlendMode = XR_ENVIRONMENT_BLEND_MODE_OPAQUE;
endInfo.layerCount = 1;
endInfo.layers = layers;
xrEndFrame(session, &endInfo);
```

`xrWaitFrame` is the pacing primitive: it blocks to throttle the application to display rate and returns the predicted display time. This is why the submission thread must own it and the application's render thread must not be coupled to it.

The critical property for our purposes: **predictedDisplayTime is the time you should render for**, and you get a fresh, more accurate pose by calling `xrLocateViews` as late as possible before submission:

```
XrViewLocateInfo locateInfo{XR_TYPE_VIEW_LOCATE_INFO};
locateInfo.viewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
locateInfo.displayTime = frameState.predictedDisplayTime;
```

```
locateInfo.space = appSpace;

XrViewState viewState{XR_TYPE_VIEW_STATE};
uint32_t viewCount = 2;
XrView views[2] = {{XR_TYPE_VIEW}, {XR_TYPE_VIEW}};
xrLocateViews(session, &locateInfo, &viewState, 2, &viewCount, views);
```

Each `XrView` carries a pose (position + orientation) and an `XrFovf` with four tangent half-angles: `angleLeft`, `angleRight`, `angleUp`, `angleDown`. These are signed tangent angles, generally asymmetric, and the asymmetry is significant — canted-display headsets have substantially off-axis frusta.

4.2 Building the projection matrix from `XrFovf`

You must construct the projection yourself; OpenXR does not hand you a matrix. For reversed-Z with infinite far plane, which is what you want for reprojection precision:

```
Mat4 ProjectionFromFov(const XrFovf& fov, float zNear) {
    const float tanL = tanf(fov.angleLeft);
    const float tanR = tanf(fov.angleRight);
    const float tanU = tanf(fov.angleUp);
    const float tanD = tanf(fov.angleDown);

    const float tanW = tanR - tanL; // total horizontal tangent extent
    const float tanH = tanU - tanD; // total vertical tangent extent

    Mat4 m{};
    m[0][0] = 2.0f / tanW;
    m[1][1] = 2.0f / tanH;
    m[2][0] = (tanR + tanL) / tanW; // horizontal frustum shift
    m[2][1] = (tanU + tanD) / tanH; // vertical frustum shift
    m[2][2] = 0.0f;                // reversed-Z, infinite far
    m[2][3] = -1.0f;               // right-handed, looking down -Z
    m[3][2] = zNear;
    m[3][3] = 0.0f;
    return m;
}
```

This is row-vector / row-major. The `m[2][0]` and `m[2][1]` terms are the frustum shift that encodes the asymmetry. **Applications that assume a symmetric frustum will produce visibly wrong results if you naively substitute only the FOV scale and drop the shift.** This is one of the most common injection bugs: the image looks almost right, but objects near the screen edge have incorrect parallax, and the sense of scale is subtly wrong.

For reversed-Z with a finite far plane z_f :

$$m_{22} = \frac{z_n}{z_f - z_n}, \quad m_{32} = \frac{z_n z_f}{z_f - z_n}$$

The depth-to-view-space inverse, which you will need constantly in Chapters 8–11:

$$z_v = \frac{m_{32}}{d - m_{22}}$$

with d the raw depth-buffer value. For the infinite reversed-Z form above this reduces to the very convenient $z_v = z_n/d$.

4.3 Swapchain management

Request swapchain images from the runtime rather than presenting your own textures; this lets the runtime use its preferred format and, on some runtimes, avoid a copy.

```
uint32_t formatCount = 0;
xrEnumerateSwapchainFormats(session, 0, &formatCount, nullptr);
std::vector<int64_t> formats(formatCount);
xrEnumerateSwapchainFormats(session, formatCount, &formatCount, formats.data());
// Prefer, in order: R16G16B16A16_FLOAT (if you need HDR headroom for
// reprojection), R8G8B8A8_SRGB, B8G8R8A8_SRGB. Avoid non-sRGB 8-bit
// unless you are certain about your own gamma handling.

XrSwapchainCreateInfo ci{XR_TYPE_SWAPCHAIN_CREATE_INFO};
ci.usageFlags = XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT
```

```

        | XR_SWAPCHAIN_USAGE_SAMPLED_BIT;
ci.format      = chosenFormat;
ci.sampleCount = 1;           // MSAA resolve before submission
ci.width       = recommendedWidth;
ci.height      = recommendedHeight;
ci.faceCount   = 1;
ci.arraySize   = 1;           // or 2 for a texture-array swapchain
ci.mipCount    = 1;
xrCreateSwapchain(session, &ci, &swapchain);

```

An important decision: **one swapchain with arraySize = 2, or two separate swapchains?** For AER, two separate swapchains are strongly preferable, because they let you submit a *stale* image for one eye without re-copying it. With an array swapchain, each acquired image gives you both array slices and you must write both every frame. With separate swapchains you can, in principle, hold one eye’s image across frames — though note that OpenXR requires you to release an acquired image before submitting, and you must re-acquire each frame, so the saving is in the copy, not in the acquisition.

4.4 Depth submission and why it matters

If the runtime supports `XR_KHR_composition_layer_depth`, submitting depth alongside color allows the runtime’s own reprojection to be positionally correct rather than rotation-only. This interacts with your frame generation in a way you must think about explicitly:

```

XrCompositionLayerDepthInfoKHR depthInfo{
    XR_TYPE_COMPOSITION_LAYER_DEPTH_INFO_KHR};
depthInfo.subImage      = depthSubImage;
depthInfo.minDepth      = 0.0f;
depthInfo.maxDepth      = 1.0f;
depthInfo.nearZ         = zNear;
depthInfo.farZ          = INFINITY; // reversed-Z infinite
projectionView.next     = &depthInfo;

```

The interaction: if you have generated a frame by warping, its color no longer corresponds exactly to the depth buffer you captured. Submitting mismatched color and depth causes the runtime’s reprojection to produce artifacts *on top of* your warp artifacts. You have three options, in order of preference:

1. Warp the depth buffer alongside the color, keeping them consistent. Costs a little more but is correct. Chapter 11 covers depth warping.
2. Submit depth only for genuine rendered frames, and omit it for generated frames. Runtimes handle a missing depth layer gracefully.
3. Don’t submit depth at all, and disable runtime reprojection if the runtime lets you. Simplest, and the right choice if your own reprojection is good.

4.5 Reprojection interaction: don’t fight the runtime

Every major runtime has its own reprojection (SteamVR Motion Smoothing, Oculus ASW, WMR LSR). If you generate frames *and* the runtime generates frames, you get compounded artifacts and, frequently, oscillating behaviour where the runtime’s heuristics decide to engage and disengage repeatedly.

Detect and manage this. SteamVR exposes settings that can be queried and set through its own API; Oculus exposes per-frame flags. The general principle: if you are always submitting at full display rate — which you should be, once frame generation is working — the runtime’s reprojection will never engage, because it only engages on missed frames. **The correct strategy is therefore to guarantee you never miss a submission deadline**, which makes the question moot. This is a strong argument for extrapolation-based generation with a hard time budget and a graceful degradation path, as developed in Chapter 10.

5 Camera Interception Across Three APIs

5.1 The fingerprinting problem

You need to find, every frame, the memory the application uses to store its view-projection matrix, and change it before the GPU reads it. The application will not tell you where that is. You must infer it.

The core insight is that **a view-projection matrix has strong structural constraints** that random float data does not satisfy. Given 16 consecutive floats interpreted as a row-major 4x4 matrix A , test:

- **Perspective signature.** $A_{03} \approx 0$, $A_{13} \approx 0$, $|A_{23}| \approx 1$, $A_{33} \approx 0$. A perspective view-projection has this shape because the w output is $\pm z_v$.
- **Basis orthonormality.** The view matrix’s upper-left 3x3 is a rotation, so its rows are unit-length and mutually orthogonal. In a *combined* view-projection this is scaled by the projection terms, but the ratios are still constrained: rows 0 and 1 of the upper 3x3 remain orthogonal after independent scaling.
- **Magnitude plausibility.** Translation components are in world units. If the application uses metres, $|A_{30}|$, $|A_{31}|$, $|A_{32}|$ are

typically 10^0 to 10^4 . Values of 10^{-8} or 10^{12} indicate you have found something else.

- **Temporal coherence.** The real camera matrix changes smoothly. Sample the candidate over 30 frames; a genuine camera matrix's derived position traces a continuous path with plausible velocity. Random data does not.
- **Finiteness.** No NaN, no Inf, no denormals.

Combine these into a score, and require a candidate to score highly for several consecutive frames before accepting it.

```
struct MatrixCandidate {
    size_t    bufferId;    // which constant buffer
    uint32_t  byteOffset;  // offset within it
    float     score;
    int       consecutiveHits;
    Mat4      lastValue;
    Vec3      lastDerivedPos;
};

float ScoreAsViewProjection(const float* m) {
    float s = 0.0f;

    // 1. Non-finite rejection.
    for (int i = 0; i < 16; ++i)
        if (!std::isfinite(m[i])) return 0.0f;

    // 2. Perspective column signature (row-vector convention:
    //    column 3 is the w-producing column).
    const float w0 = m[3], w1 = m[7], w2 = m[11], w3 = m[15];
    if (fabsf(w0) < 1e-4f && fabsf(w1) < 1e-4f &&
        fabsf(fabsf(w2) - 1.0f) < 1e-3f && fabsf(w3) < 1e-4f)
        s += 0.4f;
    else
        return 0.0f; // hard requirement

    // 3. Row-scale plausibility. For a VP matrix the first three rows'
    //    xyz parts have magnitudes on the order of the projection scale.
    const float r0 = sqrtf(m[0]*m[0] + m[4]*m[4] + m[8]*m[8]);
    const float r1 = sqrtf(m[1]*m[1] + m[5]*m[5] + m[9]*m[9]);
    if (r0 > 1e-3f && r0 < 1e3f && r1 > 1e-3f && r1 < 1e3f) s += 0.2f;

    // 4. Aspect-ratio consistency: r0/r1 should approximate 1/aspect.
    const float ratio = r0 / std::max(r1, 1e-6f);
    if (ratio > 0.3f && ratio < 3.0f) s += 0.2f;

    // 5. Translation magnitude plausibility.
    const float t = sqrtf(m[12]*m[12] + m[13]*m[13] + m[14]*m[14]);
    if (t < 1e6f) s += 0.2f;

    return s;
}
```

5.1.1 Decomposing to recover view and projection separately

Applications usually upload a combined VP or even MVP . To inject stereo you need them separated, because you must modify V (eye offset) and P (FOV, frustum shift) independently. You can recover them.

Given $A = VP$ with P of the standard perspective form, the projection scale terms are recoverable because V 's rotation part is orthonormal. Specifically, write $V = \begin{bmatrix} R & 0 \\ \mathbf{t} & 1 \end{bmatrix}$ and $P = \text{diag-ish with entries } p_{00}, p_{11}, p_{22}, p_{32}, p_{23}$. Then the upper 3x3 of A is $R \cdot P_{3 \times 3}$, and since R is orthonormal, the column norms of A 's upper 3x3 give you $|p_{00}|, |p_{11}|, |p_{22}|$ directly:

$$p_{00} = \left\| \begin{bmatrix} A_{00} \\ A_{10} \\ A_{20} \end{bmatrix} \right\|, \quad p_{11} = \left\| \begin{bmatrix} A_{01} \\ A_{11} \\ A_{21} \end{bmatrix} \right\|$$

and $p_{23} = A_{23}$ (the w -producing term), $p_{32} = A_{32} - \mathbf{t} \cdot (\dots)$ which requires knowing $\mathbf{t}$ — obtained from the last row after dividing out. In practice the cleanest approach:

```

bool DecomposeViewProjection(const Mat4& A, Mat4& outV, Mat4& outP) {
    // Column norms of upper-left 3x3 give projection scales.
    Vec3 c0{A[0][0], A[1][0], A[2][0]};
    Vec3 c1{A[0][1], A[1][1], A[2][1]};
    Vec3 c2{A[0][2], A[1][2], A[2][2]};

    const float p00 = Length(c0);
    const float p11 = Length(c1);
    if (p00 < 1e-6f || p11 < 1e-6f) return false;

    // Recover the rotation rows by normalising.
    // For c2 the projection term p22 is unknown a priori, but the
    // rotation's third column must be orthogonal to the first two;
    // recover it by cross product and use it to solve for p22.
    Vec3 r0 = c0 / p00;
    Vec3 r1 = c1 / p11;
    Vec3 r2 = Cross(r0, r1); // right-handed; flip sign if needed
    const float p22 = Dot(c2, r2);

    // Handedness check: the w column tells us the sign convention.
    if (A[2][3] > 0.0f) r2 = -r2;

    outV = Mat4Identity();
    // Rotation is the transpose arrangement of r0,r1,r2 as columns.
    outV[0][0]=r0.x; outV[0][1]=r1.x; outV[0][2]=r2.x;
    outV[1][0]=r0.y; outV[1][1]=r1.y; outV[1][2]=r2.y;
    outV[2][0]=r0.z; outV[2][1]=r1.z; outV[2][2]=r2.z;

    // Translation: last row of A is t * P, so t = lastRow * P^-1.
    outP = Mat4Zero();
    outP[0][0] = p00; outP[1][1] = p11; outP[2][2] = p22;
    outP[2][3] = A[2][3]; outP[3][2] = A[3][2] /* refined below */;

    // Solve for view translation from A's last row.
    Vec3 lastRow{A[3][0], A[3][1], A[3][2]};
    Vec3 tView{lastRow.x / p00, lastRow.y / p11, 0.0f};
    // tView.z requires p32, which we get from the depth range if known,
    // or by assuming the standard form. See text.
    outV[3][0] = -(Dot(r0, tView)); // etc.
    return true;
}

```

The z translation and p_{32} are genuinely coupled and cannot be separated from A alone — one equation, two unknowns. Three resolutions:

1. **Use the depth buffer.** Render one frame, read back the depth histogram, and fit. The near plane appears as the maximum depth value in reversed- Z .
2. **Assume a canonical near plane.** Most engines use 0.01–0.1 m. Guess, then validate by checking that reconstructed world positions of screen-space samples are self-consistent across two frames with known camera motion.
3. **Find the separate matrices instead.** Many engines upload V and P separately in the same constant buffer, often adjacent. Fingerprint for a *pure* projection matrix (mostly zeros, recognisable sparsity pattern) and a *pure* view matrix (orthonormal 3x3, last row is translation, last column is $(0, 0, 0, 1)^T$). This is the easiest path and works more often than you would expect — scan for the sparsity pattern first, before attempting decomposition.

A pure projection matrix's sparsity mask, in row-major order, is highly distinctive:

```

[ X 0 0 0 ]
[ 0 X 0 0 ]
[ A B X S ]    A,B = frustum shift (often 0); S = +/-1
[ 0 0 X 0 ]

```

Twelve of sixteen entries are exactly zero in the common symmetric case. Scanning for “exactly 0.0f in these twelve positions and nonzero in the other four” gives you essentially zero false positives.

5.2 D3D11 interception

Constant buffer updates arrive by two paths: Map/Unmap with D3D11_MAP_WRITE_DISCARD, and UpdateSubresource. Hook both.

```
// Vtable-patched Map.
HRESULT STDMETHODCALLTYPE Hooked_Map(
    ID3D11DeviceContext* ctx, ID3D11Resource* res, UINT sub,
    D3D11_MAP mapType, UINT flags, D3D11_MAPPED_SUBRESOURCE* out)
{
    HRESULT hr = g_realMap(ctx, res, sub, mapType, flags, out);
    if (SUCCEEDED(hr) && IsConstantBuffer(res)) {
        // Record the pointer so Unmap can inspect what was written.
        g_pendingMaps[ctx].push_back({res, out->pData, SizeOf(res)});
    }
    return hr;
}

void STDMETHODCALLTYPE Hooked_Unmap(
    ID3D11DeviceContext* ctx, ID3D11Resource* res, UINT sub)
{
    auto it = FindPendingMap(ctx, res);
    if (it) {
        // The application has finished writing. Inspect and rewrite in place.
        ScanAndPatchMatrices(static_cast<float*>(it->ptr), it->sizeBytes, res);
    }
    g_realUnmap(ctx, res, sub);
}
```

Writing into the mapped pointer before `Unmap` is the key trick. The memory is still CPU-visible and the GPU has not read it. You can rewrite the matrix in place with zero copies and no additional synchronisation.

For `UpdateSubresource`, you must copy the source data, patch the copy, and pass the patched buffer through:

```
void STDMETHODCALLTYPE Hooked_UpdateSubresource(
    ID3D11DeviceContext* ctx, ID3D11Resource* dst, UINT sub,
    const D3D11_BOX* box, const void* src, UINT rowPitch, UINT depthPitch)
{
    if (IsConstantBuffer(dst) && HasKnownMatrixAt(dst)) {
        thread_local std::vector<uint8_t> scratch;
        const size_t n = SizeOf(dst);
        scratch.assign((const uint8_t*)src, (const uint8_t*)src + n);
        ScanAndPatchMatrices((float*)scratch.data(), n, dst);
        g_realUpdateSubresource(ctx, dst, sub, box, scratch.data(),
                                rowPitch, depthPitch);
    }
    return;
}

g_realUpdateSubresource(ctx, dst, sub, box, src, rowPitch, depthPitch);
}
```

Watch for deferred contexts. If the application uses `ID3D11DeviceContext` in deferred mode and builds command lists on worker threads, your per-context state must be thread-local, and patching must happen at record time, not at `ExecuteCommandList` time.

5.3 D3D12 interception

D3D12 has no `Map` per draw. Constant data reaches the GPU by three routes:

1. **Upload heap writes.** The application maps a large upload buffer once (persistently) and writes into sub-ranges. You will never see a `Map` call at the time of the write.
2. **Root constants.** `SetGraphicsRoot32BitConstants` — easy to hook, and if the application puts its matrix here, you are lucky.
3. **Root CBVs / descriptor tables.** `SetGraphicsRootConstantBufferView` gives you a GPU virtual address. You can map that back to a resource if you tracked allocations.

The persistent-map case is the hard one and it is the common one. The technique:

Track every `Map` on upload-heap resources and record the CPU pointer and the resource's GPU virtual address range. Then, when you see `SetGraphicsRootConstantBufferView(rootIndex, gpuVA)`, resolve `gpuVA` back to a CPU pointer via your table, and patch there — at command-list *record* time, before the command list is executed.

```
struct UploadRange {
    ID3D12Resource*      res;
    D3D12_GPU_VIRTUAL_ADDRESS gpuBase;
    uint8_t*             cpuBase;
    UINT64                size;
```

```

};
std::vector<UploadRange> g_uploadRanges; // guarded by a lock

uint8_t* ResolveGpuVA(D3D12_GPU_VIRTUAL_ADDRESS va) {
    for (auto& r : g_uploadRanges)
        if (va >= r.gpuBase && va < r.gpuBase + r.size)
            return r.cpuBase + (va - r.gpuBase);
    return nullptr;
}

void STDMETHODCALLTYPE Hooked_SetGraphicsRootCBV(
    ID3D12GraphicsCommandList* cl, UINT rootIndex,
    D3D12_GPU_VIRTUAL_ADDRESS va)
{
    if (uint8_t* cpu = ResolveGpuVA(va)) {
        ScanAndPatchMatrices((float*)cpu, kProbeBytes, va);
    }
    g_realSetGraphicsRootCBV(cl, rootIndex, va);
}

```

This has a serious hazard. In D3D12 the application may reuse the same upload region for the next frame while the current frame is still in flight, and it manages that reuse with its own fences that you cannot see. If you patch at record time you are safe — the data must be valid then. If you defer patching, you race. Always patch at record time.

A second hazard: patching the *same* memory twice. If the application records a command list once and executes it multiple times, or if you see the same GPU VA from multiple root CBV calls, you will apply your eye offset repeatedly. Guard with a per-frame set of already-patched addresses, and clear it at frame boundaries.

An alternative D3D12 strategy that avoids all of this: **shadow buffers**. Instead of patching the application’s memory, allocate your own upload buffer, copy the application’s constant data into it with your modifications, and rewrite the root CBV to point at your buffer. Costs a copy of a few hundred bytes per draw — negligible — and is completely safe with respect to aliasing and reuse. This is the recommended approach for D3D12 despite being slightly more work.

```

void STDMETHODCALLTYPE Hooked_SetGraphicsRootCBV_Shadow(
    ID3D12GraphicsCommandList* cl, UINT rootIndex,
    D3D12_GPU_VIRTUAL_ADDRESS va)
{
    uint8_t* src = ResolveGpuVA(va);
    if (src && SlotCarriesCamera(rootIndex)) {
        auto [shadowCpu, shadowVA] = g_shadowRing.Allocate(kCBSize, 256);
        memcpy(shadowCpu, src, kCBSize);
        ApplyEyeTransform((float*)shadowCpu, g_currentEye);
        g_realSetGraphicsRootCBV(cl, rootIndex, shadowVA);
        return;
    }
    g_realSetGraphicsRootCBV(cl, rootIndex, va);
}

```

The shadow ring must be sized for the deepest frame pipeline the application uses (query `IDXGISwapChain::GetDesc` for buffer count and multiply generously) and recycled with your own fence.

5.4 Vulkan interception

Vulkan is intercepted most cleanly as an **implicit layer**, registered via a JSON manifest and `VK_INSTANCE_LAYERS`, which avoids vtable patching entirely and gives you a well-defined dispatch chain.

Constant data arrives via:

1. **Push constants** — `vkCmdPushConstants`. Trivially hookable, and the data is right there in the call.
2. **Uniform buffers via descriptor sets** — the application writes into a mapped `HOST_VISIBLE` buffer and binds a descriptor set. Same persistent-map problem as D3D12.
3. **`vkCmdUpdateBuffer`** — inline data in the command buffer, up to 65536 bytes. Hookable, data is in the call.

```

VKAPI_ATTR void VKAPI_CALL Layer_CmdPushConstants(
    VkCommandBuffer cb, VkPipelineLayout layout,
    VkShaderStageFlags stages, uint32_t offset, uint32_t size,
    const void* values)
{
    if (size >= 64) {

```

```

thread_local std::vector<uint8_t> scratch;
scratch.assign((const uint8_t*)values, (const uint8_t*)values + size);
ScanAndPatchMatrices((float*)scratch.data(), size, layout);
g_dispatch.CmdPushConstants(cb, layout, stages, offset, size,
                           scratch.data());

return;
}
g_dispatch.CmdPushConstants(cb, layout, stages, offset, size, values);
}

```

For the mapped-uniform-buffer case, hook `vkMapMemory` to record pointers, and `vkCmdBindDescriptorSets` to know which buffers are live. Then patch at bind time, same as D3D12. Note that Vulkan permits `HOST_COHERENT` memory where writes become visible without a flush — you have no `vkFlushMappedMemoryRanges` call to intercept as a signal that writing is complete. Patch at bind, not at write.

A Vulkan-specific advantage: if the application uses `VK_KHR_multiview`, you may be able to enable stereo almost for free by extending the view mask and providing per-view matrices in an array. Detect `VkRenderPassMultiviewCreateInfo` in `vkCreateRenderPass`; if present, the application’s shaders already index by `gl_ViewIndex` and you have a far easier job. This is rare in non-VR applications but worth the check.

5.5 Choosing which draws to modify

Not every draw should get the stereo transform. Specifically:

- **World geometry** — yes, full stereo transform.
- **Skybox** — yes for rotation, but the translation component must be dropped or the skybox will exhibit incorrect parallax. Detect skyboxes by: depth writes disabled or depth test set to always/equal-to-far, a single large draw, and a cubemap or equirect texture bound.
- **HUD / UI** — no. These are drawn in screen space with an orthographic or identity transform and must be handled by a separate compositing path (Chapter 13).
- **Shadow maps** — no. Shadow map rendering uses a light-space matrix that has nothing to do with the eye. Patching it produces spectacularly broken shadows. Detect by: render target is depth-only, no color target bound, and the “projection” matrix is orthographic (its *w* column is (0, 0, 0, 1)).
- **Post-process fullscreen passes** — no transform, but see Chapter 7 for how screen-space effects break under AER.
- **Reflection / cubemap captures** — no. Detect by render target being a cube-compatible texture or by unusual aspect ratio.

The classifier:

```

enum class PassKind { World, Skybox, ShadowMap, UI, PostProcess, Capture, Unknown };

PassKind ClassifyPass(const BoundState& s) {
    if (s.depthTargetOnly && s.projectionIsOrthographic) return PassKind::ShadowMap;
    if (s.rtIsCubeCompatible || s.rtAspect < 0.5f || s.rtAspect > 2.5f)
        return PassKind::Capture;
    if (!s.depthTestEnabled && s.rtMatchesBackbufferSize && s.blendEnabled)
        return PassKind::UI;
    if (s.vertexCount <= 6 && s.noVertexBuffer) return PassKind::PostProcess;
    if (s.depthWriteDisabled && s.depthFuncIsFarPlane) return PassKind::Skybox;
    if (s.rtMatchesSceneSize && s.depthTestEnabled) return PassKind::World;
    return PassKind::Unknown;
}

```

Treat `Unknown` conservatively: apply the transform, because missing world geometry is more visually catastrophic than an incorrectly transformed minor effect. But log it, because `Unknown` is where your per-application tuning effort goes.

6 Alternate-Eye Rendering: Theory

6.1 The core idea

Naive stereo renders both eyes every application frame:

App frame:	1	2	3	4
Rendered:	L,R	L,R	L,R	L,R
Cost:	2Cv	2Cv	2Cv	2Cv

AER renders one eye per application frame and reuses or synthesises the other:

App frame:	1	2	3	4
Rendered:	L	R	L	R
Displayed:	L,R'	L',R	L,R'	L',R
Cost:	1Cv	1Cv	1Cv	1Cv

where the primed eye is not freshly rendered. In the simplest scheme (**AER v1**), the primed eye is the *previous frame's* image for that eye, held unchanged.

The immediate consequence: at application rate f_a , **each eye is refreshed at $f_a/2$** . Both eyes still receive a new image every display interval (because you submit the held image again), but the *content* of each eye is 0 or $1/f_a$ seconds old, alternating.

6.2 Temporal disparity error

This is the central artifact and the thing you must quantify.

Consider a scene point that is stationary in world space, with the head translating at velocity $\mathbf{v}$. At time t , the left eye renders the point at screen position $\mathbf{u}_L(t)$. At time $t + \Delta$ (where $\Delta = 1/f_a$), the right eye renders it at $\mathbf{u}_R(t + \Delta)$. The viewer, however, fuses these two images as though they were simultaneous.

The apparent disparity is:

$$\delta_{\text{apparent}} = \mathbf{u}_R(t + \Delta) - \mathbf{u}_L(t) = \underbrace{[\mathbf{u}_R(t) - \mathbf{u}_L(t)]}_{\text{true disparity}} + \underbrace{\Delta \cdot \frac{\partial \mathbf{u}_R}{\partial t}}_{\text{error}} + O(\Delta^2)$$

The error term is the screen-space velocity of the point multiplied by the frame interval. This is exactly the per-frame motion of the object on screen. So:

The temporal disparity error of AER equals the per-frame screen-space motion.

Convert to depth error. For a point at distance z with IPD b , the true angular disparity is approximately $\theta = b/z$ radians. An error ϵ in angular disparity corresponds to a perceived depth error:

$$\frac{\partial z}{\partial \theta} = -\frac{b}{\theta^2} = -\frac{z^2}{b} \quad \Rightarrow \quad \Delta z \approx \frac{z^2}{b} \epsilon$$

With $b = 0.064$ m, a point at 5 m, and an angular error of 1 arcminute (2.9×10^{-4} rad):

$$\Delta z \approx \frac{25}{0.064} \times 2.9 \times 10^{-4} \approx 0.11 \text{ m}$$

which is a 2% depth error — imperceptible. But the z^2 scaling is brutal. At 20 m the same angular error yields 1.8 m of depth error, and at 50 m it yields 11 m. **Fortunately this works in your favour**, because at those distances true disparity is already below fusion threshold and the visual system is relying on monocular cues anyway. The problematic regime is the middle: 1–10 m, where disparity is a strong depth cue and moderate errors are detectable.

6.2.1 The strafe case

The worst case for AER is **lateral head translation**, because it produces screen-space motion that is *differential between the eyes* and in the same axis as disparity itself.

Head translating laterally at v m/s. A point at depth z has screen-space angular velocity v/z . Over $\Delta = 1/f_a$, the angular error is $v\Delta/z$. Compare to true disparity b/z : the ratio is

$$\frac{\text{error}}{\text{disparity}} = \frac{v\Delta/z}{b/z} = \frac{v\Delta}{b}$$

Depth-independent. At $f_a = 45$ Hz ($\Delta = 22$ ms) and $v = 1$ m/s (a gentle sidestep), the ratio is $0.022/0.064 = 0.34$. You are introducing a 34% error in disparity across the entire scene. That is a gross, obvious, immediately uncomfortable distortion — the world appears to shear in depth, with near objects and far objects skewing coherently.

At $f_a = 90$ Hz the ratio halves to 17%, still visible. This single equation explains why AER v1 is only tolerable at high application frame rates, and why motion compensation (AER v2) is not optional for a serious implementation.

The mitigating factor is that **pure lateral head translation at sustained velocity is rare in seated VR**. Rotational head motion, which is far more common, produces error that is *identical for both eyes* to first order — it shifts both images by the same amount rather than changing their relative disparity — and is therefore perceived as latency rather than as depth distortion. Latency is unpleasant but not stereoscopically confusing. This asymmetry is the reason AER v1 is usable in practice despite the analysis above.

6.3 Eye scheduling policies

Which eye do you render on which frame? Three policies:

Strict alternation. L, R, L, R, ... Simplest. Produces a fixed 2-frame period. Any resonance between the alternation period and scene motion (a strobing light at $f_a/2$, a rotor, a wheel) produces a pronounced inter-ocular flicker.

Dominant-eye weighting. L, L, R, L, L, R, ... Refreshes the dominant eye at $2f_a/3$ and the non-dominant at $f_a/3$. Exploits ocular dominance: roughly two-thirds of people have a strong eye preference, and errors in the non-dominant eye are perceptually suppressed to a degree. Effective in practice, but requires user configuration and is not a large win.

Motion-adaptive. Track head angular and linear velocity. When motion is low, alternate. When lateral velocity exceeds a threshold, either (a) fall back to full stereo for a few frames, accepting a frame rate dip that is masked by the motion itself, or (b) increase the aggressiveness of motion compensation. Option (a) is counterintuitive but works: during fast motion, judder is masked by the motion blur of the visual system, and the disparity error is at its worst. Spending frame time exactly when it matters most is a good trade.

```
Eye ScheduleEye(const HeadState& head, uint64_t frameIndex, Policy p) {
    switch (p) {
    case Policy::Alternate:
        return (frameIndex & 1) ? Eye::Right : Eye::Left;

    case Policy::DominantWeighted: {
        const Eye dom = g_config.dominantEye;
        const Eye sub = Other(dom);
        return (frameIndex % 3 == 2) ? sub : dom;
    }

    case Policy::MotionAdaptive: {
        const float lateralSpeed = Length(ProjectOntoRight(head.linearVelocity,
                                                             head.orientation));

        if (lateralSpeed > g_config.fullStereoThreshold) {
            g_forceFullStereoFrames = 3; // hysteresis
        }

        if (g_forceFullStereoFrames > 0) {
            --g_forceFullStereoFrames;
            return Eye::Both;
        }

        return (frameIndex & 1) ? Eye::Right : Eye::Left;
    }
    }
}
```

Note the hysteresis counter. Without it, the policy oscillates at the threshold and produces a worse experience than either mode alone. Three frames is a reasonable minimum; tune with a slow attack and fast release if you can afford the frame budget.

6.4 Half-rate versus alternate: an important distinction

There is a related but different technique often confused with AER: rendering both eyes at half rate and holding both. That is:

App frame:	1	2	3	4
Rendered:	L,R	-	L,R	-
Displayed:	L,R	L,R	L,R	L,R

This has the *same* GPU cost as AER — one eye-renderer per display interval on average — but a completely different artifact profile. Both eyes are always temporally consistent with each other, so **there is no disparity error at all**. Instead the entire image judders at $f_a/2$.

Which is better? It depends entirely on content:

Content	AER better	Half-rate better
Static scene, rotating head	Similar	Similar
Fast lateral head motion	No	Yes
Fast object motion, static head	Yes	No
High-contrast edges, high spatial frequency	No	Yes
Smooth gradients, low frequency	Yes	No

AER's advantage is that *something* is fresh every frame, so temporal aliasing of motion is halved relative to half-rate. Its disadvantage is that the freshness is asymmetric between the eyes, which the visual system is specifically evolved to detect.

A hybrid worth building: run AER, but when the disparity error metric (computed from head velocity and scene depth statistics, per the equations above) exceeds a threshold, switch to half-rate for a few frames. Both modes cost the same, so the switch is free in performance terms; you are simply choosing which artifact to accept moment to moment.

7 Alternate-Eye Rendering: Implementation

7.1 The per-frame flow

```
void OnApplicationPresent(FrameContext& fc) {
    const Eye eye = ScheduleEye(g_head, fc.index, g_config.policy);

    // 1. The camera transform for THIS frame was already applied during
    // draw interception, using the eye chosen at frame start.
    // Here we capture the result.
    CaptureColorAndDepth(fc);

    // 2. Fill the opposite eye.
    const Eye other = Other(eye);
    if (g_config.aerVersion == 1) {
        // Hold: the swapchain image for `other` retains last frame's content.
        // Nothing to do except ensure we do not overwrite it.
    } else {
        SynthesizeOppositeEye(fc, other); // Chapter 8
    }

    // 3. Submit both eyes.
    SubmitStereoFrame(fc);

    // 4. Choose next frame's eye and pre-apply so that interception during
    // the next frame's draws uses the right transform.
    g_pendingEye = ScheduleEye(g_head, fc.index + 1, g_config.policy);
}
```

The step-4 ordering is critical. Draw interception happens *during* the frame, so the eye selection must be decided *before* the frame's first draw. In practice you choose it at the previous frame's `Present`, which is the earliest reliable frame boundary an injected renderer has.

Detecting the frame boundary reliably is itself nontrivial. `Present` is the obvious hook, but some applications call `Present` multiple times per logical frame, or render to an intermediate target and blit. Cross-check with: `ClearRenderTargetView` on the primary target, the first draw to the scene depth buffer, or a monotonic query-based heartbeat. Use the union with debouncing.

7.2 Applying the eye transform

Given the application's mono view matrix V , the stereo view matrix for eye e is:

$$V_e = T_e \cdot V_{\text{head}}, \quad T_e = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \mp b/2 & 0 & 0 & 1 \end{bmatrix}$$

in row-vector convention, with the sign chosen by eye. But this is only correct if the application's camera is *at the head origin* — and injected cameras usually are not. You typically want:

$$V_e = (\text{HeadPose}_e)^{-1} \cdot (\text{WorldFromApp}) \cdot V$$

where HeadPose_e comes from `xrLocateViews` and includes both the eye offset and the actual tracked head pose, and `WorldFromApp` is a configuration matrix mapping the application's world scale and orientation conventions to the VR world (handedness flip, unit scale, up-axis swap).

```
Mat4 BuildStereoView(const Mat4& appView, const XrView& xrView,
                    const WorldConfig& cfg)
{
    // Strip the application's own rotation if we are taking over head
    // orientation entirely (typical for full 6DoF injection), or keep it
    // if the application controls yaw (typical for a "cockpit" mod).
    Mat4 appBase = cfg.takeOverOrientation
        ? ExtractTranslationOnly(appView)
        : appView;

    // Eye pose in tracking space -> view matrix contribution.
    Mat4 eyeFromTracking = Inverse(PoseToMatrix(xrView.pose));
```

```
// Scale: application world units per metre.
Mat4 scale = Mat4Scale(1.0f / cfg.metresPerAppUnit);

return appBase * cfg.appToVr * scale * eyeFromTracking;
}
```

`metresPerAppUnit` matters enormously. Get it wrong by 2x and the world feels like a dollhouse or a giant’s playground, and — crucially — the disparity error analysis above scales with it. A miscalibrated world scale makes AER artifacts far worse, because the effective IPD relative to world geometry changes.

7.3 Capturing the depth buffer

You need depth for AER v2, for frame generation, and for depth-layer submission. Capturing it in an injected renderer requires care because the application will reuse and overwrite it.

Identification. The scene depth buffer is the depth-stencil view bound during the pass classified as `World`, with dimensions matching the main render target. Track it by the underlying resource pointer, not the view, because applications create multiple views of the same texture.

Capture timing. You want depth *after* all opaque geometry but *before* the application starts reusing the buffer for post-processing (many engines alias the depth buffer’s memory for other purposes once the depth prepass is consumed). The reliable trigger is the first draw that binds the depth buffer as a *shader resource* rather than as a depth-stencil target — that transition marks the end of depth writing.

```
// D3D11: detect the SRV bind and copy at that moment.
void STDMETHODCALLTYPE Hooked_PSSetShaderResources(
    ID3D11DeviceContext* ctx, UINT start, UINT num,
    ID3D11ShaderResourceView* const* srvs)
{
    for (UINT i = 0; i < num; ++i) {
        if (srvs[i] && ResourceOf(srvs[i]) == g_sceneDepthResource
            && !g_depthCapturedThisFrame) {
            ctx->CopyResource(g_depthCopy, g_sceneDepthResource);
            g_depthCapturedThisFrame = true;
        }
    }
    g_realPSSetShaderResources(ctx, start, num, srvs);
}
```

For D3D12 and Vulkan, the equivalent signal is the resource barrier / image layout transition from `DEPTH_WRITE` to `SHADER_RESOURCE` / `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`. Hook `ResourceBarrier` and `vkCmdPipelineBarrier` respectively and inject your copy immediately after the transition.

Format handling. Depth is commonly `D32_FLOAT`, `D24_UNORM_S8_UINT`, or `D32_FLOAT_S8X24_UINT`. You cannot `CopyResource` a typeless depth into a plain float texture without matching formats. Create your copy target with the *typeless* parent format and an SRV with the depth-plane format:

```
D24_UNORM_S8_UINT -> R24G8_TYPELESS -> SRV as R24_UNORM_X8_TYPELESS
D32_FLOAT_S8X24 -> R32G8X24_TYPELESS -> SRV as R32_FLOAT_X8X24_TYPELESS
D32_FLOAT         -> R32_TYPELESS    -> SRV as R32_FLOAT
```

MSAA. If the scene depth is multisampled you must resolve it, and depth resolve is not a standard hardware resolve — averaging depth is meaningless. Write a compute shader that takes the *minimum* (nearest, i.e. maximum value in reversed-Z) sample per pixel. Min-depth resolve preserves silhouettes correctly for reprojection purposes, whereas averaging smears them into the background.

```
// Depth resolve, reversed-Z: take the nearest sample.
Texture2DMS<float> g_depthMS;
RWTexture2D<float> g_depthOut;

[numthreads(8, 8, 1)]
void ResolveDepthMin(uint3 tid : SV_DispatchThreadID) {
    uint w, h, samples;
    g_depthMS.GetDimensions(w, h, samples);
    if (tid.x >= w || tid.y >= h) return;

    float nearest = 0.0f; // reversed-Z: larger = nearer
    for (uint s = 0; s < samples; ++s)
        nearest = max(nearest, g_depthMS.Load(int2(tid.xy), s));
}
```

```
g_depthOut[tid.xy] = nearest;
}
```

7.4 Swapchain handling for the held eye

Under AER v1, one eye’s image must persist across frames. OpenXR’s acquire/release model means you get an arbitrary image from the chain each frame, so “persisting” requires you to keep your own copy and blit it in:

```
void SubmitStereoFrame(FrameContext& fc) {
    for (int e = 0; e < 2; ++e) {
        uint32_t imageIndex = 0;
        xrAcquireSwapchainImage(g_swapchain[e], nullptr, &imageIndex);

        XrSwapchainImageWaitInfo wi{XR_TYPE_SWAPCHAIN_IMAGE_WAIT_INFO};
        wi.timeout = XR_INFINITE_DURATION;
        xrWaitSwapchainImage(g_swapchain[e], &wi);

        if (e == (int)fc.renderedEye) {
            BlitToSwapchain(fc.color, g_swapchainImages[e][imageIndex]);
            // Keep our own persistent copy for the next frame's hold.
            CopyTo(g_eyePersistent[e], fc.color);
        } else {
            BlitToSwapchain(g_eyePersistent[e],
                           g_swapchainImages[e][imageIndex]);
        }

        xrReleaseSwapchainImage(g_swapchain[e], nullptr);
    }
    // ... build XrCompositionLayerProjection and xrEndFrame
}
```

The extra full-screen copy per eye per frame is not free — at 2000x2200 per eye in RGBA16F that is roughly 35 MB of bandwidth per copy, or 6.3 GB/s at 90 Hz for two eyes. On a modern GPU with 500+ GB/s that is about 1.3% of bandwidth, which is acceptable but not nothing. If your persistence buffer and swapchain formats match exactly, some runtimes let you avoid the double copy by rendering directly into the swapchain image and keeping a separate reference; measure before optimising.

7.5 The projection layer

```
XrCompositionLayerProjectionView projViews[2]{};
for (int e = 0; e < 2; ++e) {
    projViews[e].type = XR_TYPE_COMPOSITION_LAYER_PROJECTION_VIEW;
    projViews[e].pose = g_views[e].pose;
    projViews[e].fov = g_views[e].fov;
    projViews[e].subImage.swapchain = g_swapchain[e];
    projViews[e].subImage.imageRect = {{0,0},{width,height}};
    projViews[e].subImage.imageArrayIndex = 0;
}

XrCompositionLayerProjection layer{XR_TYPE_COMPOSITION_LAYER_PROJECTION};
layer.layerFlags = XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT;
layer.space = g_appSpace;
layer.viewCount = 2;
layer.views = projViews;
```

A subtlety with AER: the pose you submit for the held eye should be the pose that eye’s image was *rendered with*, not the current pose. If you submit the current pose, the runtime will assume the image is fresh and will not correct for its staleness. If you submit the stale pose, the runtime’s reprojection — if active — will correct for the head motion since then, which is exactly what you want. Submitting per-view poses that differ between eyes is legal and correct in OpenXR.

This gives you **AER v1.5 essentially for free**: let the runtime timewarp the held eye by submitting its true (older) pose. It handles rotation correctly and, with a depth layer, approximates translation. Many implementations stop here, and for rotation-dominant content the result is genuinely good.

8 The AER Artifact Taxonomy

Every artifact below has a mechanism, a detection method, and a mitigation. Treat this chapter as a checklist during bring-up; each artifact is diagnosable in isolation with the right test scene.

8.1 A1: Temporal disparity shear

Mechanism. As derived in Chapter 6: the two eyes represent different instants, so any relative motion between head and scene appears as a disparity offset.

Appearance. The world appears to shear in depth during lateral motion. Near objects seem to slide relative to far objects in a way that does not correspond to parallax. Sustained exposure produces discomfort quickly.

Test scene. A regular grid of vertical poles at 1, 2, 4, 8, 16 m. Strafe laterally at constant speed. Under correct stereo the poles maintain fixed relative depth. Under AER v1 they will appear to rotate as a group.

Mitigation. Motion compensation (AER v2). This is the primary reason v2 exists.

8.2 A2: Inter-ocular judder (retinal rivalry)

Mechanism. Each eye's content updates at $f_a/2$, but the two update sequences are offset by half a period. For a moving high-contrast edge, one eye sees it at position x while the other sees it at $x + \delta$; on the next frame they swap roles.

Appearance. Edges appear to vibrate or "buzz". Fine detail — foliage, chain link, text — shimmers. Under strong contrast the visual system fails to fuse and you get frank binocular rivalry: the two images alternate in perception rather than merging, which is intensely uncomfortable.

Test scene. A black-and-white checkerboard plane, viewed obliquely, with slow camera pan. The high-frequency region near the horizon will rival first.

Mitigation. Three, in combination:

1. **Match the effective sharpness of both eyes.** If the held eye is stale, apply a slight low-pass to *both* eyes so the temporal difference in high frequencies is reduced. A very mild blur (0.5–0.8 px Gaussian) on the held eye alone creates a sharpness rivalry that is itself uncomfortable; blur both or neither.
2. **Reduce contrast at high spatial frequency.** A mild sharpening-inverse in the frequency band where rivalry occurs (roughly 4–12 cycles/degree).
3. **Motion compensation**, which reduces δ toward zero and thereby moves the content below rivalry threshold.

8.3 A3: Specular and view-dependent shading breakup

Mechanism. Specular highlights, Fresnel effects, parallax-mapped surfaces, refraction, and anisotropic materials all depend on the view vector. Correct stereo renders each eye with its own view vector, producing genuinely different highlight positions — which is *correct* and is a real depth cue. Under AER, the held eye's highlights are stale, so the difference includes both the correct stereo offset and a temporal error.

Appearance. Highlights on curved metallic or wet surfaces flicker or appear to detach from the surface. Water surfaces are the worst case: high-frequency specular detail combined with the animation of the water itself.

Test scene. A polished sphere under a point light, with the camera orbiting.

Mitigation. Genuinely hard, because the correct answer requires re-shading, not re-projecting. Practical options:

- **Roughness clamping.** If you can identify and modify the application's material parameters (you usually cannot, in an injected renderer), raise minimum roughness.
- **Temporal filtering of the specular term.** Requires separating specular from diffuse, which requires G-buffer access. In a deferred renderer you may be able to identify the specular buffer by its render-target signature.
- **Accept it.** In most content the affected surface area is small. This artifact is usually ranked below A1 and A2 in perceived severity.

8.4 A4: Screen-space effect inconsistency

Mechanism. SSAO, SSR, screen-space shadows, and screen-space GI all compute their result from the depth and color of the *current view*. Under AER, each eye's screen-space effects were computed from that eye's view at that eye's time. The two results are not merely offset — they are structurally different, because screen-space effects have visibility-dependent behaviour. An SSR ray that finds a reflection in the left eye may run off-screen in the right eye.

Appearance. Reflections flicker between present and absent. Ambient occlusion contact shadows shimmer. The effect is strongest near screen edges, where SSR information is most incomplete.

Test scene. A wet floor with a strong reflection of an object near the frame edge; pan so the object crosses the edge.

Mitigation.

- **Render screen-space effects at a widened FOV.** If you can force the application's SSR/SSAO passes to run with a slightly larger frustum than the display frustum, off-screen information becomes available and the eye-to-eye difference shrinks. In an injected renderer this means detecting those passes and substituting a modified projection — doable, but it requires the passes to be correctly classified.
- **Force both eyes to use the same screen-space effect result.** Since these effects are usually low-frequency, sharing the left eye's SSAO with the right eye is visually acceptable and eliminates the inconsistency entirely. This requires intercepting the effect's output and substituting it, which is very application-specific.
- **Disable the effects.** Blunt, but SSR in particular is often not worth its artifact cost under AER.

8.5 A5: Temporal antialiasing interaction

Mechanism. TAA accumulates samples across frames using motion vectors and a history buffer. Under AER, the history buffer for a given eye is two application frames old, not one, and the motion vectors the application computes assume a one-frame interval. The reprojection is therefore wrong by a factor of two.

Appearance. Ghosting trails behind moving objects, doubled in length. Or, if the TAA's disocclusion rejection triggers, aggressive flickering as history is discarded every frame.

Mitigation.

- **Maintain separate TAA history per eye.** This is the correct fix and it happens automatically if the application's history buffer is part of the per-eye state you already duplicate. Ensure your interception duplicates history buffers, not just render targets.
- **Scale motion vectors by 2.** If you can find and modify the motion vector buffer or the TAA's velocity scale constant, doubling it corrects the reprojection. Find the TAA pass by its signature: a fullscreen pass reading a history texture, a velocity texture, and the current color, writing to a target that becomes next frame's history.
- **Adjust the jitter sequence.** TAA jitters the projection matrix per frame with a Halton or similar sequence. Under AER each eye sees every *other* element of the sequence, which may have poor distribution. Provide each eye with its own full-rate jitter sequence by tracking per-eye frame counters.

This last point is subtle and frequently missed. If the application uses an 8-sample Halton sequence and you alternate eyes, the left eye gets samples 0, 2, 4, 6 and the right gets 1, 3, 5, 7. Each eye now has a 4-sample effective pattern with worse distribution than the intended 8. Override the jitter with a per-eye counter.

8.6 A6: Vergence-accommodation aggravation

Mechanism. Not caused by AER, but aggravated by it. Rapid fluctuation in apparent disparity forces the vergence system to hunt, which is fatiguing.

Mitigation. Everything that reduces A1. Additionally, consider clamping the maximum disparity error you will allow: if your motion compensation cannot correct below a threshold, fall back to half-rate mode (both eyes stale but consistent), which has zero disparity error.

8.7 A7: Flicker resonance

Mechanism. Any scene phenomenon at or near $f_a/2$ aliases against the alternation. Strobing lights, rotating machinery, flickering fire, or an application's own frame-rate-dependent effects.

Appearance. A slow beat frequency appears; a rotor that should spin appears to crawl or reverse, and does so *differently in each eye*, which is severely uncomfortable.

Mitigation. Break the periodicity. Use a non-strict schedule — for example, dither the alternation with a low-discrepancy sequence so the eye pattern is L,R,L,L,R,L,R,R,... with equal long-run distribution but no fixed period. The resonance disappears, replaced by mild stochastic noise, which is far preferable.

```
// Dithered eye schedule: maintains ~50/50 split with no fixed period.
Eye DitheredSchedule(uint64_t frameIndex, int& accumulator) {
    // van der Corput / additive recurrence with an irrational increment.
    static constexpr uint32_t kPhi = 2654435769u; // 2^32 / golden ratio
    const uint32_t h = (uint32_t)frameIndex * kPhi;
    const bool right = (h >> 31) != 0;

    // Enforce a bound on consecutive runs so no eye starves.
    if (right == g_lastWasRight) {
        if (++g_runLength >= 3) { g_runLength = 0;
                                g_lastWasRight = !right;
                                return g_lastWasRight ? Eye::Right : Eye::Left; }
    } else {
        g_runLength = 0;
    }
    g_lastWasRight = right;
    return right ? Eye::Right : Eye::Left;
}
```

8.8 Severity ranking

From bring-up experience across content types, ordered by how much they degrade the experience:

1. **A1 (disparity shear)** — disqualifying if uncorrected during motion.
2. **A2 (rivalry)** — disqualifying in high-frequency content.
3. **A7 (resonance)** — rare but severe when it occurs.

4. **A5 (TAA)** — very visible, but fully fixable.
5. **A4 (screen-space)** — visible, partially fixable.
6. **A3 (specular)** — visible, hard to fix, usually tolerable.
7. **A6** — cumulative, matters for session length rather than first impression.

Fix in that order. A1 and A2 are both addressed by the same mechanism, which is the subject of the next chapter.

9 AER v2: Motion-Compensated Synthesis

9.1 The idea

AER v1 holds the opposite eye’s previous image unchanged. AER v2 **synthesises** it, using the freshly rendered eye as the source and warping it into the other eye’s viewpoint at the current instant.

This changes the error characteristics fundamentally. Instead of a temporal error of one full frame interval, you have a *spatial* reprojection error whose magnitude depends on disocclusion, not on time. The world no longer shears; instead, small regions near depth discontinuities may be incorrect.

The transformation you need maps a pixel in the rendered eye to its position in the synthesised eye. Given a pixel at screen coordinate $\mathbf{u}$ in eye A with depth d :

$$\begin{aligned}\mathbf{p}_c^A &= (2u_x - 1, 1 - 2u_y, d, 1) \\ \mathbf{p}_w &= \mathbf{p}_c^A (V_A P_A)^{-1}, \quad \mathbf{p}_w / = w \\ \mathbf{p}_c^B &= \mathbf{p}_w (V_B P_B), \quad \mathbf{u}^B = \left(\frac{1}{2} \left(\frac{p_{c,x}^B}{w} + 1 \right), \frac{1}{2} \left(1 - \frac{p_{c,y}^B}{w} \right) \right)\end{aligned}$$

Composed, this is a single matrix $R = (V_A P_A)^{-1} (V_B P_B)$, applied in clip space. Precompute it on the CPU once per frame.

9.2 Why the naive stereo warp is nearly free

Here is the property that makes AER v2 tractable. When A and B differ *only* by the eye offset — same time, same head pose, same projection except for the frustum shift — the warp reduces to a **pure horizontal shift proportional to disparity**:

$$\Delta u_x = \frac{b \cdot p_{00}}{2z_v} = \frac{b \cdot p_{00} \cdot d}{2z_n}$$

using the infinite reversed-Z relation $z_v = z_n/d$. This is a per-pixel horizontal displacement, computable in one multiply from the depth value. No matrix inverse per pixel, no general 3D reprojection.

That is the *stereo* component. AER v2 also needs the *temporal* component, because the source frame is from time t and you are synthesising for time t as well — wait. Let us be precise about the timing, because this is where implementations diverge.

9.3 Two variants of v2, and which one to build

Variant A: same-instant stereo reprojection. Render eye A at time t . Immediately warp it to eye B at time t . Submit both.

- Both eyes now represent time t . **Disparity error is zero.**
- Neither eye is stale. Both are equally fresh.
- This eliminates artifacts A1, A2, A6, and A7 completely.
- The remaining artifact is purely disocclusion: regions visible to eye B but hidden from eye A .

Variant B: cross-time reprojection. Warp the *previous* frame’s eye- B render forward in time to t , using motion compensation.

- Uses genuinely eye- B data, so no stereo disocclusion.
- But requires temporal motion compensation, which needs optical flow or motion vectors, and introduces temporal warp artifacts.

Variant A is the correct choice, and it is worth being emphatic about this. Variant A reduces to a *known, analytic* warp with a small, well-characterised error region. Variant B requires solving a genuinely hard estimation problem to correct an error that Variant A does not have. Build Variant A.

Note what Variant A implies: AER v2 is really “render mono, synthesise stereo”. The alternation still matters, though, because *which* eye you render should still alternate — if you always render the left eye, disocclusion holes always appear on the same side of every object in the right eye, and the persistent asymmetry is perceptible. Alternating distributes the error between the eyes and the visual system averages it out.

9.4 The disocclusion problem

When you shift pixels horizontally by disparity, pixels at a depth discontinuity move by different amounts, opening a gap. The gap width in pixels is:

$$w_{\text{gap}} = \frac{b \cdot p_{00} \cdot W}{2} \left| \frac{1}{z_{\text{near}}} - \frac{1}{z_{\text{far}}} \right|$$

for a screen of width W pixels and a depth edge between z_{near} and z_{far} . With $b = 0.064$, $p_{00} \approx 1.2$ (roughly 100 deg horizontal FOV), $W = 2000$, and an edge from 0.5 m to infinity:

$$w_{\text{gap}} = \frac{0.064 \times 1.2 \times 2000}{2} \times 2 = 154 \text{ px}$$

That is a very large hole. Near-field objects — a weapon model, a cockpit frame, a hand — produce disocclusion regions over a hundred pixels wide, and no inpainting scheme will convincingly fabricate that much missing content.

This is the fundamental limitation of AER v2, and it drives the architecture.

9.4.1 Mitigation 1: render the near field in both eyes

Split the scene by depth. Geometry closer than a threshold z_{split} (typically 1–2 m) is rendered for both eyes every frame; geometry beyond it is rendered for one eye and warped. Near-field geometry is usually a small fraction of the scene’s pixel and vertex cost — a first-person weapon or cockpit covers maybe 15–25% of the screen and has low overdraw — so the cost increase is modest while the artifact reduction is enormous.

Implementation: intercept the application’s near/far plane and issue the world pass twice with different depth ranges. Concretely, for the both-eyes near pass, set the projection’s far plane to z_{split} ; for the warped far pass, set the near plane to z_{split} . Depth-composite the two.

This requires two geometry submissions for the near range, which in an injected context means replaying the command stream — expensive in D3D11, more tractable in D3D12/Vulkan where you can record once and execute twice with different constant buffers.

9.4.2 Mitigation 2: dual-source warping

Keep the *previous* frame’s opposite-eye render (which you have, since eyes alternate). Warp both the current eye and the previous opposite eye into the target view, and composite: use the current-eye warp everywhere it has valid data, and fill holes from the previous opposite-eye frame.

The previous opposite-eye frame is from the correct viewpoint for the disoccluded region — that is exactly the region it *did* have data for — so the fill is geometrically correct. It is temporally stale by one frame, but the fill regions are small and adjacent to correct data, so the staleness is much less objectionable than either a hole or an inpainted guess.

This is the single highest-value refinement in AER v2 and it costs one extra texture read plus a compare in the warp shader.

```
// Composite warp: primary from current eye, holes filled from previous
// opposite-eye frame.
Texture2D<float4> g_currentEyeWarped;
Texture2D<float> g_currentEyeWarpDepth; // depth written by the warp
Texture2D<float4> g_prevOppositeWarped;
Texture2D<float> g_prevOppositeWarpDepth;
RWTexture2D<float4> g_output;

[numthreads(8, 8, 1)]
void CompositeWarp(uint3 tid : SV_DispatchThreadID) {
    const float dCur = g_currentEyeWarpDepth[tid.xy];
    const float dPrev = g_prevOppositeWarpDepth[tid.xy];

    const bool curValid = dCur > 0.0f; // 0 = no fragment landed here
    const bool prevValid = dPrev > 0.0f;

    float4 c;
    if (curValid && prevValid) {
        // Both have data. Prefer the nearer (reversed-Z: larger d).
        // Bias toward current, which is temporally fresher.
        c = (dCur >= dPrev - kDepthBias) ? g_currentEyeWarped[tid.xy]
                                          : g_prevOppositeWarped[tid.xy];
    } else if (curValid) {
        c = g_currentEyeWarped[tid.xy];
    } else if (prevValid) {
        c = g_prevOppositeWarped[tid.xy];
    } else {
```

```

    c = InpaintFallback(tid.xy);    // see below
}
g_output[tid.xy] = c;
}

```

9.4.3 Mitigation 3: inpainting the residual

After dual-source compositing, residual holes are typically a few pixels wide. Push-pull inpainting (pyramid down with validity-weighted averaging, then pyramid up with hole-only substitution) fills these convincingly and cheaply.

```

// Push phase: build a validity-weighted mipmap pyramid.
[numthreads(8, 8, 1)]
void PushDown(uint3 tid : SV_DispatchThreadID) {
    float4 sum = 0; float wsum = 0;
    [unroll] for (int y = 0; y < 2; ++y)
    [unroll] for (int x = 0; x < 2; ++x) {
        const int2 p = int2(tid.xy) * 2 + int2(x, y);
        const float w = g_validSrc[p];        // 1 = valid, 0 = hole
        sum += g_colorSrc[p] * w;
        wsum += w;
    }
    g_colorDst[tid.xy] = (wsum > 0) ? sum / wsum : 0;
    g_validDst[tid.xy] = (wsum > 0) ? 1 : 0;
}

// Pull phase: substitute coarse data only where fine data is missing.
[numthreads(8, 8, 1)]
void PullUp(uint3 tid : SV_DispatchThreadID) {
    if (g_validFine[tid.xy] > 0) return;    // keep valid fine data
    const float2 uv = (float2(tid.xy) + 0.5f) * g_invFineSize;
    g_colorFine[tid.xy] = g_colorCoarse.SampleLevel(g_linear, uv, 0);
    g_validFine[tid.xy] = 1;
}

```

Four to five pyramid levels suffice for gaps up to ~32 px. Beyond that the fill is visibly blurry, which is the signal that you needed Mitigation 1 or 2 instead.

9.5 Implementing the warp: forward versus backward

Backward warp (gather): for each destination pixel, find the source pixel that maps to it. Requires inverting the mapping, which for a depth-dependent warp is not closed-form — you would need the destination’s depth, which you do not have. Backward warping is therefore not directly applicable here.

Forward warp (scatter): for each source pixel, compute its destination and write there. Straightforward, but needs depth-ordered resolution when multiple source pixels land on the same destination. Two implementations:

9.5.1 Forward warp via rasterisation

Render a grid mesh with one vertex per source pixel (or per 2x2 block), displacing each vertex by its disparity in the vertex shader, with depth test enabled. The hardware rasteriser handles occlusion ordering for free, and interpolation across the grid quads fills small gaps automatically.

```

struct VSOut { float4 pos : SV_Position; float2 uv : TEXCOORD0; };

VSOut WarpGridVS(uint vid : SV_VertexID) {
    const uint gw = g_gridWidth;
    const uint2 g = uint2(vid % gw, vid / gw);
    const float2 uv = (float2(g) + 0.5f) * g_invGridSize;

    const float d = g_depth.SampleLevel(g_point, uv, 0);
    const float zv = g_zNear / max(d, 1e-7f);    // view-space depth

    // Horizontal disparity shift for the target eye.
    const float shiftNdc = (g_ipd * g_projScaleX) / zv * g_eyeSign;

    VSOut o;
    o.pos = float4(uv.x * 2.0f - 1.0f + shiftNdc,

```

```

        1.0f - uv.y * 2.0f,
        d, 1.0f);

    o.uv = uv;
    return o;
}

float4 WarpGridPS(VSOut i) : SV_Target { return g_color.SampleLevel(g_linear, i.uv, 0); }

```

Draw as a triangle-strip grid with degenerate strips, or as an indexed quad list. **Set depth test to greater-equal (reversed-Z) with depth write on**, so nearer fragments win. The output depth buffer doubles as your validity mask (cleared to 0 means “no fragment landed”).

Grid density is a quality/cost dial. One vertex per pixel is exact but expensive (4.4 M vertices per eye at 2000x2200). One vertex per 2x2 block is 4x cheaper and loses very little, because the interpolation across a quad approximates the true warp well within a smooth depth region. Errors appear only at depth discontinuities inside a quad, which manifest as small stretched triangles — often *desirable*, because a stretched triangle across a disocclusion is a plausible fill.

You can go further: **adaptive tessellation** driven by local depth variance, subdividing only near depth edges. Worth it if you are vertex-bound.

9.5.2 Forward warp via compute with atomic depth

Alternative: a compute shader that scatters with `InterlockedMax` on a packed depth+index buffer.

```

RWTexture2D<uint> g_depthIndex;    // packed: depth in high bits, srcIndex in low

[numthreads(8, 8, 1)]
void ScatterPass(uint3 tid : SV_DispatchThreadID) {
    const float2 uv = (float2(tid.xy) + 0.5f) * g_invSize;
    const float d = g_depth[tid.xy];
    if (d <= 0.0f) return;

    const float zv = g_zNear / d;
    const float shiftPx = (g_ipd * g_projScaleX * g_halfWidth) / zv * g_eyeSign;
    const int2 dst = int2(tid.x + (int)round(shiftPx), tid.y);
    if (dst.x < 0 || dst.x >= (int)g_width) return;

    // Pack: 16 bits depth (reversed-Z, larger = nearer), 16 bits source x.
    const uint packed = (uint(saturate(d) * 65535.0f) << 16) | (tid.x & 0xFFFF);
    InterlockedMax(g_depthIndex[dst], packed);
}

[numthreads(8, 8, 1)]
void GatherPass(uint3 tid : SV_DispatchThreadID) {
    const uint packed = g_depthIndex[tid.xy];
    if (packed == 0) { g_out[tid.xy] = float4(0,0,0,0); return; } // hole
    const uint srcX = packed & 0xFFFF;
    g_out[tid.xy] = g_color[int2(srcX, tid.y)];
}

```

The horizontal-only nature of the stereo warp makes this especially efficient: the scatter stays within a row, so cache behaviour is excellent and you can even do the whole thing per-row in groupshared memory. For pure stereo reprojection, the compute path typically beats the raster path. For the general 3D reprojection needed in frame generation, the raster path wins because it gets interpolation for free.

9.6 Handling the near-field cutout

Combining Mitigation 1 with the warp requires a depth-split composite:

1. Render near pass ($z < z_{\text{split}}$) for BOTH eyes. cost: $2 * C_{\text{near}}$
2. Render far pass ($z > z_{\text{split}}$) for ONE eye. cost: $1 * C_{\text{far}}$
3. Warp far pass to the opposite eye. cost: warp
4. Composite: near over far, per eye, using depth.

Setting the split in the projection matrix: for the near pass, replace the far plane; for the far pass, replace the near plane. With reversed-Z infinite far, the far pass is unchanged and the near pass gets a finite far plane at z_{split} .

The composite is a simple depth-aware over:

```
[numthreads(8, 8, 1)]
void CompositeNearFar(uint3 tid : SV_DispatchThreadID) {
    const float4 nearC = g_nearColor[tid.xy];
    const float  nearD = g_nearDepth[tid.xy];
    const float4 farC  = g_farWarped[tid.xy];
    // Reversed-Z: nearD > 0 means a near-pass fragment exists.
    g_out[tid.xy] = (nearD > 0.0f) ? nearC : farC;
}
```

If the near pass has alpha-blended content (a translucent HUD element, a scope reticle) you need a proper alpha composite rather than a binary select, and you must ensure the near pass's blending is resolved before the composite.

Choosing z_{split} . The gap width formula gives you the tool. Solve for the z at which the disocclusion gap falls below your acceptable threshold w_{max} (say 16 px, which push-pull inpainting handles well):

$$z_{\text{split}} = \frac{b \cdot p_{00} \cdot W}{2w_{\text{max}}}$$

With the numbers above and $w_{\text{max}} = 16$: $z_{\text{split}} = 4.8$ m. That is higher than the 1–2 m rule of thumb, and rendering everything within 4.8 m in both eyes may eat most of your savings. The practical resolution is to accept larger gaps at intermediate depths where dual-source filling (Mitigation 2) works well, and reserve the both-eyes near pass for genuinely close geometry where dual-source filling also fails because the object moves too much between frames.

9.7 Cost accounting for AER v2

Per display frame, for one eye synthesised:

Stage	Cost (2000x2200, modern discrete GPU)
Depth resolve (if MSAA)	0.15–0.30 ms
Warp grid raster (2x2 blocks)	0.35–0.60 ms
Dual-source composite	0.10–0.15 ms
Push-pull inpaint (5 levels)	0.20–0.35 ms
Near-field both-eyes render	15–25% of C_v
Total synthesis overhead	0.8–1.4 ms + near pass

Against a saved C_v of typically 4–8 ms, this remains strongly net positive. The near-field pass is the variable that determines whether AER v2 is worth it: if your target content has a large near-field (a cockpit filling half the screen), the savings evaporate and you should consider running the near field at reduced resolution for the synthesised eye, or falling back to v1 behaviour for the near field only.

10 Frame Generation: Fundamentals and Latency

10.1 The causality constraint

To place a synthetic frame at time $t + \frac{1}{2}\Delta$ between rendered frames at t and $t + \Delta$, you must possess both. Frame $t + \Delta$ does not exist until the application finishes rendering it. Therefore the earliest you can display the synthetic frame is after frame $t + \Delta$ is complete — by which time the synthetic frame's nominal timestamp is already in the past.

The consequence: **interpolation-based frame generation displays the rendered frames themselves one full frame late.**

Without generation:

```
App renders N ----> display N ----> App renders N+1 ----> display N+1
latency = render time + compositor
```

With interpolation:

```
App renders N ----> HOLD
App renders N+1 --> generate N.5 --> display N.5 --> display N+1
latency = render time + compositor + 1 application frame interval
```

At 45 Hz application rate, that is 22 ms of added motion-to-photon latency. Recall from Chapter 2 that the detection threshold is 15–20 ms and that a 300 deg/s head turn accumulates 3 degrees per 10 ms. Adding 22 ms is not a subtle change. In flat gaming, where the latency budget is dominated by input-to-display for a mouse, this trade is often acceptable. **In VR it is usually not.**

10.2 Extrapolation as the alternative

Extrapolate instead: from frames $t - \Delta$ and t , predict frame $t + \frac{1}{2}\Delta$ and display it immediately. No frames are withheld. Latency is unchanged, or slightly improved because you are displaying content whose nominal timestamp is *ahead* of the last render.

With extrapolation:

```
App renders N ----> display N ----> extrapolate N+0.5 ----> display N+0.5
                                   (using N-1 and N)
latency = render time + compositor (unchanged)
```

The cost is accuracy. Interpolation is constrained on both sides; extrapolation has only one side and must guess. Objects that change direction, appear, or disappear between t and $t + \frac{1}{2}\Delta$ are predicted wrongly. Extrapolation artifacts are structurally different from interpolation artifacts: interpolation produces *blending* errors (ghosting, warping of correctly-placed content), while extrapolation produces *prediction* errors (content in the wrong place, overshooting on deceleration).

For VR, extrapolation is the right default. This is also what the established runtime techniques do — asynchronous spacewarp and equivalents are extrapolative, generating a new frame ahead of the last rendered one rather than between two.

10.2.1 A hybrid: partial interpolation

If you can tolerate a fraction of a frame of latency, you can interpolate at a phase close to 1.0 rather than 0.5. Generating a frame at $t + 0.9\Delta$ from frames t and $t + \Delta$ requires waiting only for frame $t + \Delta$, but since the generated frame is nearly coincident with $t + \Delta$, it does not help throughput.

The genuinely useful hybrid: **interpolate the far field, extrapolate the near field**. Far-field content has small screen-space motion and low latency sensitivity; near-field content — weapons, hands, cockpit — has large motion and high latency sensitivity, and is often head-locked or nearly so, making it cheap to extrapolate accurately. Split by depth as in Chapter 8. This is more machinery than most implementations warrant, but it is the highest-quality option.

10.3 What “generation” must actually produce

Be clear about the target. At display rate f_d and application rate f_a , you need f_d/f_a images per application frame. Common ratios:

f_a	f_d	Ratio	Generated per rendered
45	90	2	1
36	72	2	1
40	120	3	2
30	90	3	2
45	120	2.67	non-integer

Non-integer ratios are the hard case and they are common in practice, because application frame rate is not stable. Two approaches:

1. **Lock the application rate.** Throttle the application to an integer divisor of display rate. Simple, and gives you a fixed generation schedule. Costs performance headroom that the application could have used.
2. **Phase-continuous generation.** Track the phase $\phi \in [0, 1)$ of the current display time relative to the interval between the last two rendered frames, and generate at that phase. Handles arbitrary ratios but means every generated frame has a different extrapolation distance, and quality varies frame to frame.

Phase-continuous with a rate limiter (refuse to extrapolate beyond $\phi = 1.5$, falling back to a held frame or runtime reprojection) is the robust production choice.

```
struct GenerationSchedule {
    double lastRenderTime;    // wall clock of most recent app frame
    double prevRenderTime;    // the one before
    double appInterval;       // smoothed estimate

    float PhaseFor(double displayTime) const {
        // Phase relative to lastRenderTime, in units of appInterval.
        return float((displayTime - lastRenderTime) / appInterval);
    }
};

// Smooth the interval estimate; raw deltas are extremely noisy.
void UpdateInterval(GenerationSchedule& s, double now) {
    const double raw = now - s.lastRenderTime;
    // Reject outliers (hitches) before smoothing.
    if (raw > 0.25 * s.appInterval && raw < 4.0 * s.appInterval)
        s.appInterval = 0.9 * s.appInterval + 0.1 * raw;
    s.prevRenderTime = s.lastRenderTime;
```

```
s.lastRenderTime = now;
}
```

10.4 Latency budget accounting

Build an explicit budget and validate it with a photodiode or a high-speed camera. The components:

$$T_{mp} = T_{pose} + T_{sim} + T_{render} + T_{gen} + T_{submit} + T_{comp} + T_{scanout} + T_{persist}$$

- T_{pose} : age of the pose when sampled. Minimised by late-latching.
- T_{sim} : application simulation time, uncontrollable.
- T_{render} : GPU time for the application's frame.
- T_{gen} : your synthesis cost. Target < 2 ms.
- T_{submit} : copy into swapchain plus `xrEndFrame` overhead.
- T_{comp} : runtime compositor, typically 1–3 ms.
- $T_{scanout}$: panel scanout, $\approx 1/f_d$ for the last row.
- $T_{persist}$: display persistence (illumination duration), typically 1–3 ms on low-persistence OLED/LCD.

The item you control most is T_{pose} , and the single most valuable thing your generation stage can do is **re-latch the pose immediately before warping**. Because you are warping anyway, you can warp to the *newest available pose* rather than the pose the application rendered with, converting your frame generator into a timewarp at zero additional cost. This is the reason a good extrapolative frame generator can have *lower* motion-to-photon latency than the same application without generation.

```
void GenerateFrame(const RenderedFrame& prev, const RenderedFrame& curr,
                  double targetDisplayTime, GeneratedFrame& out)
{
    // Latch a fresh pose for the target display time.
    XrView views[2];
    LocateViews(targetDisplayTime, views);

    // The warp target is the composition of:
    // (a) scene motion extrapolated from prev->curr flow, and
    // (b) head motion from curr.cameraPose to views[e].pose.
    // Both are folded into a single per-pixel destination.
    for (int e = 0; e < 2; ++e) {
        const Mat4 srcVP = curr.view[e] * curr.proj[e];
        const Mat4 dstVP = ViewFromPose(views[e]) * ProjFromFov(views[e].fov);
        BuildReprojection(srcVP, dstVP, out.reproj[e]);
    }
    ExecuteWarp(prev, curr, out);
}
```

11 Motion Field Estimation

11.1 Where motion comes from

You need, per pixel, a 2D displacement describing where that pixel's content moves between frames. Four sources, in decreasing order of quality:

1. Engine motion vectors. If the application renders a velocity buffer for TAA or motion blur, it contains exactly what you want: per-pixel screen-space motion including object animation and skinning. Finding it is the challenge.

Detection heuristics: a render target of format `R16G16_FLOAT` or `R16G16_SNORM` (occasionally `R8G8`), at scene resolution, written during or immediately after the G-buffer pass, subsequently read by a fullscreen pass that also reads a history texture. The value distribution is centred near zero with a long tail.

```
bool LooksLikeVelocityBuffer(const RtDesc& d, const Stats& s) {
    const bool fmtOk = d.format == FMT_R16G16_FLOAT
        || d.format == FMT_R16G16_SNORM
        || d.format == FMT_R16G16_UNORM;
    if (!fmtOk) return false;
    if (d.width != g_sceneWidth || d.height != g_sceneHeight) return false;
    // Velocity buffers are near-zero-mean with small magnitude.
    if (s.meanAbs > 0.25f) return false; // in UV units
    if (s.fractionExactlyZero > 0.98f) return false; // probably cleared, unused
    return true;
}
```

Beware the encoding. Velocity may be stored in UV units, in NDC units, in pixels, scaled by an arbitrary constant, negated, or with the Y axis flipped. Calibrate by comparing against camera-derived motion in a static scene: force the camera to translate a known amount, sample the velocity buffer on a static object, and solve for the scale and sign. Do this automatically at startup with a two-frame probe.

2. Camera-derived reprojection from depth. For static geometry, motion is fully determined by depth plus the two camera matrices — no estimation required. This is exact, free, and covers the majority of most scenes' pixels.

$$\mathbf{u}' = \Pi(\Pi^{-1}(\mathbf{u}, d) (V_t P_t)^{-1} (V_{t+1} P_{t+1}))$$

3. Estimated optical flow. Compute the dense correspondence between two color frames directly. Handles object motion that depth reprojection misses. This is the expensive part and the subject of the rest of this chapter.

4. Global parametric motion. Fit a single homography or affine transform to the whole frame. Trivially cheap, and surprisingly adequate for rotation-dominant content, but useless for object motion.

The correct architecture uses 2 and 3 together: compute the camera-derived reprojection analytically, then estimate only the *residual* flow — the part not explained by camera motion. Residual flow is near zero over most of the frame, which makes it far easier to estimate, far more compressible in the search, and far more robust. This is the single most important structural decision in the flow stage.

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{camera}}(d, V, P) + \mathbf{F}_{\text{residual}}(\text{estimated})$$

11.2 Hierarchical block matching

The workhorse algorithm. Build a pyramid, search coarsely at the top, refine downward. This is the same structure as motion estimation in video codecs, and the implementation experience transfers directly.

11.2.1 Pyramid construction

```
// Gaussian-ish downsample, 2x2 box with a small tent for stability.
[numthreads(8, 8, 1)]
void BuildPyramidLevel(uint3 tid : SV_DispatchThreadID) {
    const int2 p = int2(tid.xy) * 2;
    float3 c = 0;
    c += g_src[p + int2(0,0)].rgb;
    c += g_src[p + int2(1,0)].rgb;
    c += g_src[p + int2(0,1)].rgb;
    c += g_src[p + int2(1,1)].rgb;
    g_dst[tid.xy] = float4(c * 0.25f, 1.0f);
}
```

Work in **luminance, not RGB**, for the matching cost — it is 3x cheaper and matching on chroma adds little. Better still, work on a **gradient-domain** representation (the magnitude of the spatial gradient, or a census transform), which is invariant to brightness changes. Brightness changes between frames are common (exposure adaptation, flickering lights, fades) and destroy naive intensity matching.

The **census transform** is particularly good here: for each pixel, compare it to its 8 neighbours and pack the comparison results into a bitmask. Matching cost is then the Hamming distance between bitmasks, which is one `countbits` instruction and is completely invariant to monotonic intensity changes.

```
uint CensusTransform(int2 p) {
    const float c = Luma(g_src[p].rgb);
    uint mask = 0;
    [unroll] for (int y = -1; y <= 1; ++y)
    [unroll] for (int x = -1; x <= 1; ++x) {
        if (x == 0 && y == 0) continue;
        mask = (mask << 1) | (Luma(g_src[p + int2(x,y)].rgb) > c ? 1u : 0u);
    }
    return mask;    // 8 bits used
}

uint MatchCost(uint a, uint b) { return countbits(a ^ b); }
```

Use a 5x5 census (24 bits) if you can afford it; the larger support is noticeably more robust in low-texture regions.

11.2.2 The search

At each pyramid level, for each block (typically 8x8 pixels at the finest level, scaling with the pyramid), search a window around the prediction inherited from the coarser level.

```
// One level of hierarchical block matching, residual flow only.
Texture2D<uint>   g_censusA;    // frame t
Texture2D<uint>   g_censusB;    // frame t+1
Texture2D<float2> g_flowCoarse; // upsampled from previous level
Texture2D<float2> g_flowCamera; // analytic camera reprojection
RWTexture2D<float2> g_flowOut;

static const int kSearchRadius = 3; // +/- 3 px at this level

[numthreads(8, 8, 1)]
void BlockMatchLevel(uint3 tid : SV_DispatchThreadID) {
    const int2 p = int2(tid.xy);

    // Prediction: camera flow plus coarse residual.
    const float2 camF = g_flowCamera[p];
    const float2 predR = g_flowCoarse[p];

    float2 bestR = predR;
    uint   bestCost = 0xFFFFFFFFu;

    // Also test zero residual and the residuals of the four neighbours;
    // neighbour propagation greatly accelerates convergence
    // (this is the PatchMatch idea, applied per level).
    float2 candidates[6];
    candidates[0] = predR;
    candidates[1] = float2(0, 0);
    candidates[2] = g_flowCoarse[p + int2(-1, 0)];
    candidates[3] = g_flowCoarse[p + int2( 1, 0)];
    candidates[4] = g_flowCoarse[p + int2( 0,-1)];
    candidates[5] = g_flowCoarse[p + int2( 0, 1)];

    [unroll] for (int c = 0; c < 6; ++c) {
        const uint cost = BlockCost(p, camF + candidates[c]);
        if (cost < bestCost) { bestCost = cost; bestR = candidates[c]; }
    }

    // Local exhaustive refinement around the best candidate.
    for (int dy = -kSearchRadius; dy <= kSearchRadius; ++dy)
    for (int dx = -kSearchRadius; dx <= kSearchRadius; ++dx) {
        const float2 r = bestR + float2(dx, dy);
        const uint cost = BlockCost(p, camF + r);
        if (cost < bestCost) { bestCost = cost; bestR = r; }
    }

    g_flowOut[p] = bestR;
}

uint BlockCost(int2 p, float2 flow) {
    const int2 q = p + int2(round(flow));
    uint sum = 0;
    [unroll] for (int y = -2; y <= 2; ++y)
    [unroll] for (int x = -2; x <= 2; ++x)
        sum += MatchCost(g_censusA[p + int2(x,y)],
                        g_censusB[q + int2(x,y)]);
    return sum;
}
```

Five to six pyramid levels at 1/2 scaling handles displacements up to roughly $3 \times 2^5 = 96$ px, which covers all but the most extreme motion at typical VR resolutions. Add a seventh level if your content includes fast lateral strafing.

11.2.3 Sub-pixel refinement

Integer flow produces visible stair-stepping in the warp. Refine to sub-pixel by fitting a parabola to the cost function around the minimum:

$$\delta_x = \frac{C(-1) - C(+1)}{2[C(-1) - 2C(0) + C(+1)]}$$

evaluated independently in x and y . Clamp $|\delta| \leq 0.5$; values outside indicate the parabola fit is invalid (usually a flat or multi-modal cost surface) and should fall back to integer.

```
float ParabolicSubpixel(float cml, float c0, float cpl) {
    const float denom = cml - 2.0f * c0 + cpl;
    if (abs(denom) < 1e-5f) return 0.0f;
    return clamp(0.5f * (cml - cpl) / denom, -0.5f, 0.5f);
}
```

11.3 Regularisation and confidence

Raw block-matched flow is noisy in low-texture regions, where the cost surface is flat and the match is arbitrary. Two remedies.

Edge-aware smoothing. Filter the flow field with a bilateral or guided filter using the color image (or the depth buffer) as the guide. This propagates flow from textured regions into flat ones while respecting object boundaries. A guided filter with the depth buffer as guide is particularly appropriate, because depth discontinuities are exactly where flow discontinuities belong.

```
// Depth-guided flow smoothing, separable approximation.
[numthreads(64, 1, 1)]
void SmoothFlowH(uint3 tid : SV_DispatchThreadID) {
    const int2 p = int2(tid.xy);
    const float dc = g_depth[p];
    const float conf_c = g_confidence[p];

    float2 sum = 0; float wsum = 0;
    [unroll] for (int i = -4; i <= 4; ++i) {
        const int2 q = p + int2(i, 0);
        const float dq = g_depth[q];
        // Depth similarity weight: reject across depth edges.
        const float wd = exp(-abs(dq - dc) * g_depthSigmaInv);
        // Spatial weight.
        const float ws = g_gaussWeights[abs(i)];
        // Confidence weight: trust confident samples more.
        const float wc = g_confidence[q];
        const float w = wd * ws * wc;
        sum += g_flowIn[q] * w;
        wsum += w;
    }
    g_flowOut[p] = (wsum > 1e-5f) ? sum / wsum : g_flowIn[p];
}
```

Confidence estimation. Produce a per-pixel confidence alongside the flow, used both for smoothing weights and for deciding whether to trust the flow at warp time. Three cheap signals:

1. **Cost margin.** The difference between the best and second-best match cost. A large margin means an unambiguous match.
2. **Forward-backward consistency.** Compute flow $t \rightarrow t+1$ and $t+1 \rightarrow t$; a pixel is consistent if $\|\mathbf{F}_{fwd}(\mathbf{u}) + \mathbf{F}_{bwd}(\mathbf{u} + \mathbf{F}_{fwd}(\mathbf{u}))\| < \tau$. This is the single best occlusion detector, and it costs a second flow pass.
3. **Local texture energy.** The variance of luminance in the block. Low variance means the match is underdetermined regardless of what cost it achieved.

```
[numthreads(8, 8, 1)]
void ComputeConfidence(uint3 tid : SV_DispatchThreadID) {
    const int2 p = int2(tid.xy);
    const float2 ff = g_flowFwd[p];
    const int2 q = p + int2(round(ff));
    const float2 fb = g_flowBwd[clamp(q, int2(0,0), g_maxCoord)];

    const float residual = length(ff + fb);
    const float cFB = saturate(1.0f - residual * g_fbSigmaInv);
}
```

```

const float margin = float(g_cost2nd[p] - g_cost1st[p]);
const float cMargin = saturate(margin * g_marginSigmaInv);

const float texture = g_lumaVariance[p];
const float cTex = saturate(texture * g_texSigmaInv);

g_confidence[p] = cFB * lerp(0.4f, 1.0f, cMargin) * lerp(0.3f, 1.0f, cTex);
}

```

The forward-backward pass doubles your flow cost, which is significant. A cheaper approximation: compute backward flow only at coarse pyramid levels, where it is 1/16th the cost, and upsample the consistency mask. Occlusion regions are large and low-frequency, so this loses very little.

11.4 Cost budget

For 2000x2200 per eye, six pyramid levels, 5x5 census, ± 3 search plus propagation:

Stage	Cost
Pyramid build (both frames, luma)	0.15 ms
Census transform (both frames)	0.20 ms
Block match, levels 5..2 (low res)	0.25 ms
Block match, level 1 (half res)	0.45 ms
Block match, level 0 (full res)	1.20 ms
Backward pass (coarse only)	0.20 ms
Confidence + smoothing	0.30 ms
Total per eye	~2.8 ms

That is too expensive for two eyes at 90 Hz. Three ways to cut it:

1. **Estimate flow at half resolution and upsample.** Flow is smooth almost everywhere; full-resolution estimation is largely wasted. Halving the resolution cuts the dominant level-0 cost by 4x. Total drops to roughly 1.1 ms per eye. **Do this by default.**
2. **Estimate flow once, for one eye, and transfer it to the other via the stereo disparity you already have.** Since you know the depth and the eye offset, a flow field valid for the left eye maps to the right eye by the same warp used in AER v2. Halves the cost again. Small errors at disocclusions, which are already low-confidence regions.
3. **Skip residual estimation where camera flow is confidently sufficient.** In a static scene with only camera motion, residual flow is exactly zero everywhere. Detect this by sampling a sparse grid, checking the residual, and skipping the full estimation if the sampled residual is below threshold. Enormous savings on static scenes, which are common.

With all three: roughly 0.5–0.7 ms for both eyes, which fits.

12 Warping, Occlusion, and Compositing

12.1 The warp target

Given flow $\mathbf{F}_{t \rightarrow t+1}$ and a phase ϕ , the extrapolated position of a pixel is $\mathbf{u} + \phi \mathbf{F}$ for interpolation with $\phi \in (0, 1)$, or $\mathbf{u} + (1 + \phi) \mathbf{F}$ for extrapolation beyond frame $t + 1$ using flow measured from t to $t + 1$.

Careful with the sign and reference frame. For extrapolation you have frames $t - \Delta$ and t , and flow between them, and you want time $t + \phi \Delta$. The flow you measured describes motion over $[t - \Delta, t]$; you extrapolate by scaling it by ϕ and applying it to frame t :

$$\mathbf{u}_{\text{dst}} = \mathbf{u} + \phi \cdot \mathbf{F}_{t-\Delta \rightarrow t}(\mathbf{u})$$

This is a **constant-velocity** model. A constant-acceleration model using three frames is possible:

$$\mathbf{u}_{\text{dst}} = \mathbf{u} + \phi \mathbf{F}_1 + \frac{1}{2} \phi^2 (\mathbf{F}_1 - \mathbf{F}_0)$$

but is **usually worse in practice**, because flow estimation noise is amplified by the differencing, and because real object motion in games is dominated by piecewise-constant velocity with abrupt changes rather than smooth acceleration. If you use it at all, heavily damp the acceleration term and clamp its magnitude.

The exception is **head rotation**, where you have exact angular velocity from the tracker and a second-order prediction is genuinely better. But head motion is handled analytically through the pose, not through the flow field, so this is already covered.

12.2 Forward warping with depth ordering

Same structure as the AER v2 warp, generalised to 2D displacement.

```
struct WarpVSOut {
    float4 pos      : SV_Position;
    float2 srcUV     : TEXCOORD0;
    float  confidence : TEXCOORD1;
    float  stretch  : TEXCOORD2;
};

WarpVSOut FlowWarpVS(uint vid : SV_VertexID) {
    const uint2 g = uint2(vid % g_gridW, vid / g_gridW);
    const float2 uv = (float2(g) + 0.5f) * g_invGridSize;

    const float2 flow = g_flow.SampleLevel(g_linear, uv, 0);
    const float  conf = g_confidence.SampleLevel(g_linear, uv, 0);
    const float  d    = g_depth.SampleLevel(g_point, uv, 0);

    // Combined displacement: scene motion (flow) + head motion (reprojection).
    const float2 sceneDisp = flow * g_phase;
    const float2 headDisp  = ReprojectDelta(uv, d, g_reprojMatrix);
    const float2 dstUV     = uv + sceneDisp + headDisp;

    WarpVSOut o;
    o.pos = float4(dstUV.x * 2.0f - 1.0f, 1.0f - dstUV.y * 2.0f, d, 1.0f);
    o.srcUV = uv;
    o.confidence = conf;
    o.stretch = 0.0f; // computed in GS or via derivative in PS
    return o;
}
```

Note `ReprojectDelta` handles head motion analytically from depth, and `flow` carries only the residual object motion. This separation matters: head motion is known exactly and should never be estimated.

Depth test greater-equal with depth write, reversed-Z, resolves overlaps in favour of nearer content. This is correct: when two source regions map to the same destination, the nearer one occludes.

12.3 Detecting and handling stretch

When a source quad's four corners map to widely separated destinations, the quad stretches across a disocclusion. The stretched triangle interpolates between foreground and background content, producing a characteristic smeared band — the signature artifact of naive warping, familiar from every frame-generation implementation that does not handle it.

Detect stretch per-quad and act on it:

```
// In the pixel shader, screen-space derivatives of the source UV tell
// you how much the quad has been stretched.
float4 FlowWarpPS(WarpVSOut i) : SV_Target {
    const float2 dx = ddx(i.srcUV);
    const float2 dy = ddy(i.srcUV);
    // Jacobian determinant: area ratio source->destination.
    const float jac = abs(dx.x * dy.y - dx.y * dy.x);
    const float nominal = g_invGridSize.x * g_invGridSize.y;
    const float areaRatio = jac / max(nominal, 1e-9f);

    // areaRatio >> 1 means the source area has been stretched thin.
    const float stretchFactor = saturate((areaRatio - 2.0f) / 6.0f);

    float4 c = g_color.SampleLevel(g_linear, i.srcUV, 0);
    // Encode stretch in alpha so the composite pass can down-weight it.
    c.a = 1.0f - stretchFactor;
    return c;
}
```

Three responses to high stretch, in order of quality:

1. **Discard.** `clip(-stretchFactor + 0.9)` — drop badly stretched fragments, leaving a hole that the fill stage handles. A hole filled by push-pull looks better than a smear.
2. **Down-weight in the composite.** Keep the fragment but blend it with a background estimate proportional to stretch.

3. **Split the quad.** In a geometry or mesh shader, subdivide quads whose corner flows differ by more than a threshold, so the discontinuity falls on a quad edge rather than across a quad interior. Highest quality, moderate cost.

12.4 Bidirectional warping for interpolation

If you have chosen interpolation despite the latency cost, use **both** source frames. Warp frame t forward by $\phi \mathbf{F}_{t \rightarrow t+1}$ and frame $t+1$ backward by $(1 - \phi) \mathbf{F}_{t+1 \rightarrow t}$, then blend.

The key advantage: **disocclusions in one direction are occlusions in the other**. A region revealed as an object moves right is not visible in frame t but *is* visible in frame $t+1$. Bidirectional warping fills these regions with correct data rather than inpainted guesses. This is the main reason interpolation produces higher spatial quality than extrapolation, and it partially justifies the latency cost in applications that can afford it.

```
[numthreads(8, 8, 1)]
void BlendBidirectional(uint3 tid : SV_DispatchThreadID) {
    const float4 fwd = g_warpedFwd[tid.xy];    // frame t warped forward
    const float4 bwd = g_warpedBwd[tid.xy];    // frame t+1 warped backward
    const float  dF  = g_warpedFwdDepth[tid.xy];
    const float  dB  = g_warpedBwdDepth[tid.xy];

    const bool vF = dF > 0.0f && fwd.a > 0.1f;
    const bool vB = dB > 0.0f && bwd.a > 0.1f;

    float4 result;
    if (vF && vB) {
        // Both valid: weight by phase, modulated by per-sample confidence
        // (alpha carries 1 - stretch).
        const float wF = (1.0f - g_phase) * fwd.a;
        const float wB = g_phase * bwd.a;
        result = (fwd * wF + bwd * wB) / max(wF + wB, 1e-4f);
    } else if (vF) {
        result = fwd;
    } else if (vB) {
        result = bwd;
    } else {
        result = float4(0, 0, 0, 0);    // hole
    }
    g_out[tid.xy] = result;
}
```

Note the phase weighting: near $\phi = 0$ trust the forward warp (it barely moved), near $\phi = 1$ trust the backward warp. This avoids the crossfade ghosting that a fixed 50/50 blend produces.

12.5 Hole filling

After warping and compositing, holes remain. Sources: disocclusion, discarded stretched fragments, and content that moved in from outside the frame.

That last category deserves attention. Content entering from off-screen has **no source data at all** — it was never rendered. During lateral head motion or fast turns, a band along the leading edge of the frame is fundamentally unfillable.

The fix is to render with a margin. Increase the application's FOV slightly beyond the display FOV, so you have source content just outside the visible region to warp in from. A 5–8% margin covers typical head rotation between frames.

$$\text{margin} = \omega_{\max} \cdot \Delta \cdot \frac{1}{\text{FOV}}$$

At $\omega_{\max} = 300 \text{ deg/s}$, $\Delta = 22 \text{ ms}$, FOV 100 deg: margin = 6.6%. The cost is $(1.066)^2 - 1 = 13.6\%$ more pixels, which is real but usually worth it — and it can be recovered by rendering the margin region at lower resolution, since it is only used as warp source, never displayed directly.

For interior holes, push-pull inpainting as in Chapter 8 works well. A refinement that matters: **fill from the background, not from the average**. A disocclusion hole should be filled with the content behind the occluder, not with a blend of foreground and background. Restrict the push phase to samples whose depth is at or behind the hole's expected depth:

```
[numthreads(8, 8, 1)]
void PushDownDepthAware(uint3 tid : SV_DispatchThreadID) {
    // Determine the hole's likely depth from valid neighbours at the
    // background side. Use the FARTHEST valid neighbour depth as the
    // reference, since disocclusion reveals background.
    float refDepth = 1e9f;    // in view-space metres; smaller = nearer
```

```

[unroll] for (int y = 0; y < 2; ++y)
[unroll] for (int x = 0; x < 2; ++x) {
    const int2 p = int2(tid.xy) * 2 + int2(x, y);
    if (g_valid[p] > 0) refDepth = min(refDepth, g_viewDepth[p]);
}

float4 sum = 0; float wsum = 0;
[unroll] for (int y = 0; y < 2; ++y)
[unroll] for (int x = 0; x < 2; ++x) {
    const int2 p = int2(tid.xy) * 2 + int2(x, y);
    if (g_valid[p] == 0) continue;
    // Weight down foreground samples: we want background content.
    const float rel = g_viewDepth[p] / max(refDepth, 1e-4f);
    const float w = saturate(rel); // rel >= 1 for background
    sum += g_color[p] * w; wsum += w;
}
g_colorDst[tid.xy] = (wsum > 0) ? sum / wsum : 0;
g_validDst[tid.xy] = (wsum > 0) ? 1 : 0;
}

```

12.6 Warping the depth buffer

If you submit a depth layer to OpenXR, or if you chain multiple generated frames, you need the warped depth to remain consistent with the warped color.

The warp shader already writes depth via `sv_Position.z` — but that is the *source* depth, which is correct only if the content did not move in depth. For head-motion reprojection you must compute the destination depth properly:

$$d_{\text{dst}} = \frac{p_{c,z}^{\text{dst}}}{p_{c,w}^{\text{dst}}}$$

from the full clip-space reprojection, not by copying d_{src} . For scene motion (residual flow) you have no depth information about the motion — the flow field is 2D — so you must either assume depth-preserving motion (usually fine; most object motion is roughly parallel to the image plane at typical distances) or estimate depth change from the depth buffers of the two source frames at corresponding points.

The latter is straightforward and worth doing:

$$d_{\text{dst}} = \text{lerp}(d_t(\mathbf{u}), d_{t+1}(\mathbf{u} + \mathbf{F}(\mathbf{u})), \phi)$$

interpolating in *view-space* depth ($1/d$ for reversed-Z), not in post-projective depth, because linear interpolation of post-projective depth is wrong.

13 The Combined Pipeline

13.1 Ordering: AER before generation

The pipeline established in Chapter 3 puts AER synthesis before frame generation. The reasoning is worth making explicit, because the opposite ordering is tempting.

Why not generate mono frames, then apply AER to each? Because AER’s synthesis needs depth, and generated frames have warped depth of degraded quality. Applying a stereo warp to an already-warped image compounds errors multiplicatively: a disocclusion in the temporal warp becomes a region of garbage that the stereo warp then displaces into a different position in one eye only — producing an inter-ocular difference in an artifact, which is exactly the rivalry condition Chapter 7 warns about.

Correct ordering:

- | | |
|--|------------------------------|
| 1. App renders eye A at time t . | [full quality color + depth] |
| 2. AER v2 warps A → B at time t . | [stereo warp, small error] |
| 3. Both eyes now exist at time t . Submit. | [display frame 1] |
| 4. Flow estimated between frames $t-1$ and t . | |
| 5. Both eyes warped to time $t + \phi$. | [temporal warp] |
| 6. Submit generated stereo pair. | [display frame 2] |
| 7. App renders eye B at time $t+1$. Go to 2. | |

Steps 2 and 5 are structurally the same operation with different reprojection matrices, so a single warp kernel serves both. That is a meaningful simplification: one well-tested warp implementation, parameterised.

13.2 The unified warp

Generalise the warp to take a source frame, a destination view, and a time phase:

```
struct WarpParams {
    // Source
    const GpuTexture* srcColor;
    const GpuTexture* srcDepth;
    Mat4              srcView, srcProj;

    // Destination
    Mat4              dstView, dstProj;

    // Temporal
    const GpuTexture* residualFlow; // null for pure stereo warp
    float             phase;        // 0 for same-instant stereo warp

    // Secondary source for hole filling (may be null)
    const GpuTexture* fillColor;
    const GpuTexture* fillDepth;
    Mat4              fillView, fillProj;
};

void ExecuteWarp(CommandContext& ctx, const WarpParams& p, GpuTexture& out);
```

- **AER v2 stereo warp:** srcView = leftView, dstView = rightView, residualFlow = null, phase = 0, fill = previous right-eye frame.
- **Frame generation:** srcView = dstView = same eye, residualFlow = estimated, phase = 0.5, fill = null or the other source frame for bidirectional.
- **Combined:** srcView = leftView at t , dstView = rightView at $t+\phi$, residualFlow = estimated, phase = 0.5.

That last case is the interesting one. **You can do the stereo warp and the temporal warp in a single pass**, saving one full-resolution round trip. The reprojection matrix composes; the flow adds. For the generated frame in an AER pipeline this is the natural formulation: you have one rendered eye at time t and you need both eyes at time $t + \phi\Delta$, which is two invocations of the unified warp from a single source.

$$\begin{aligned}\text{Frame}_{\text{gen}}^L &= \mathcal{W}(I_A^t; V_A^t P_A^t \rightarrow V_L^{t+\phi} P_L^{t+\phi}, \phi \mathbf{F}) \\ \text{Frame}_{\text{gen}}^R &= \mathcal{W}(I_A^t; V_A^t P_A^t \rightarrow V_R^{t+\phi} P_R^{t+\phi}, \phi \mathbf{F})\end{aligned}$$

Total cost per generated stereo frame: one flow estimation (shared between eyes, per Chapter 10) plus two warps. Roughly 1.5–2.0 ms at typical resolutions.

13.3 Effective rate multiplication

Combining AER (2x on the stereo axis) with generation (2x on the temporal axis) gives 4x nominal:

Configuration	App render rate needed for 90 Hz stereo
Naïve stereo	90 Hz, both eyes = 180 eye-renders/s
AER only	90 Hz, one eye = 90 eye-renders/s
Generation only	45 Hz, both eyes = 90 eye-renders/s
AER + generation	45 Hz, one eye = 45 eye-renders/s

A 4x reduction in view-dependent rendering work. Recalling the cost model from Chapter 2, though, only C_v scales this way; C_i and C_f still run at the application's frame rate. So the *end-to-end* speedup is:

$$S = \frac{C_i + 2C_v + C_f}{C_i/2 + C_v/2 + C_f/2 + C_{\text{synth}}}$$

where the denominator's halving comes from the application running at 45 Hz instead of 90. With $C_i = 4$ ms, $C_v = 6$ ms, $C_f = 2$ ms, $C_{\text{synth}} = 2.5$ ms:

$$S = \frac{4 + 12 + 2}{2 + 3 + 1 + 2.5} = \frac{18}{8.5} = 2.1$$

A 2.1x end-to-end improvement — real and substantial, but well short of the nominal 4x. Manage expectations accordingly, and note that C_{synth} is *fixed cost that does not scale down*, so on already-fast content the technique can be net negative. Always implement a bypass path.

13.4 Degradation ladder

Build an explicit ladder and descend it when you miss deadlines:

Level	Configuration	Trigger
0	Full stereo, no generation	Headroom > 40%
1	AER v2, no generation	Headroom 20–40%
2	AER v2 + generation	Headroom 5–20%
3	AER v2 + generation, half-res flow	Headroom 0–5%
4	AER v1 + generation, no inpaint	Missing deadlines
5	Hand off to runtime reprojection	Sustained misses

Transitions must be hysteretic and rate-limited — no more than one level change per 30 frames — because oscillation between levels is more perceptible than sitting at the worse level. Ascend slowly (require 60 frames of headroom), descend quickly (react within 3 frames).

```
void UpdateQualityLevel(const PerfStats& s) {
    const float headroom = 1.0f - (s.gpuFrameMs / s.displayIntervalMs);

    if (s.missedDeadlines3Frame >= 2 && g_level < 5) {
        g_level++; g_framesSinceChange = 0; return;
    }
    if (++g_framesSinceChange < 30) return;

    if (headroom > kAscendThreshold[g_level] && g_level > 0
        && g_framesSinceChange > 60) {
        g_level--; g_framesSinceChange = 0;
    } else if (headroom < kDescendThreshold[g_level] && g_level < 5) {
        g_level++; g_framesSinceChange = 0;
    }
}
```

14 UI, HUD, and Exclusion Handling

14.1 Why UI breaks everything

A head-up display is drawn in screen space with no depth. Under warping it behaves pathologically:

- It has no meaningful depth, so depth-based reprojection places it at whatever the depth buffer happens to contain — usually the far plane, or whatever geometry was behind it.
- It has zero motion in screen space, but the flow estimator sees the *world* moving behind it and, in low-confidence regions, may assign world motion to HUD pixels.
- Under AER it produces zero disparity, which is correct for a screen-locked element, but the stereo warp will displace it by whatever depth is under it.

The result is a HUD that smears, doubles, or swims. This is the most immediately visible failure in an injected VR renderer and it must be handled explicitly.

14.2 Extraction

Extract the UI into its own layer before any synthesis, and composite it back afterward.

Detection. UI draws are identifiable by the classifier from Chapter 5: alpha blending enabled, depth test disabled or always-pass, orthographic or identity projection, drawn late in the frame, small triangle counts with textured quads. Redirect these draws to a separate render target with alpha.

```
void STDMETHODCALLTYPE Hooked_DrawIndexed(
    ID3D11DeviceContext* ctx, UINT indexCount, UINT startIndex, INT baseVertex)
{
    const PassKind kind = ClassifyPass(CaptureBoundState(ctx));
    if (kind == PassKind::UI && g_config.extractUI) {
        // Redirect to the UI layer target.
        ID3D11RenderTargetView* savedRtv = nullptr;
        ID3D11DepthStencilView* savedDsv = nullptr;
        ctx->OMGetRenderTargets(1, &savedRtv, &savedDsv);
        ctx->OMSetRenderTargets(1, &g_uiLayerRtv, nullptr);
```

```

    g_realDrawIndexed(ctx, indexCount, startIndex, baseVertex);

    ctx->OMSetRenderTargets(1, &savedRtv, savedDsv);
    SafeRelease(savedRtv); SafeRelease(savedDsv);
    return;
}
g_realDrawIndexed(ctx, indexCount, startIndex, baseVertex);
}

```

Clear the UI layer to transparent black at frame start and ensure premultiplied alpha handling matches the application's blend state, or the composite will have dark fringes.

14.3 Re-presentation options

Once extracted, you choose how the UI appears in VR:

Screen-locked quad. Composite the UI layer onto both eyes at a fixed virtual distance, typically 1.5–3 m, as a flat plane facing the viewer. Locked to head orientation. Simple, always readable, but “sticky” — it follows head motion rigidly, which some users find uncomfortable during rapid movement.

Delayed head-locked. Same, but the quad's orientation lags head rotation with a critically damped spring (time constant ~150 ms) and clamps to a maximum angular offset (~15 degrees). Substantially more comfortable than rigid locking; the UI drifts to centre when you settle but does not whip around during a turn.

```

void UpdateUiQuadOrientation(Quat headOrient, float dt) {
    // Critically damped approach toward head orientation.
    const float k = 1.0f - expf(-dt / g_config.uiLagTau);
    g_uiOrient = Slerp(g_uiOrient, headOrient, k);

    // Clamp to a maximum angular deviation.
    const float ang = AngleBetween(g_uiOrient, headOrient);
    if (ang > g_config.uiMaxDeviation)
        g_uiOrient = Slerp(headOrient, g_uiOrient,
                           g_config.uiMaxDeviation / ang);
}

```

World-locked. Anchor the UI in world space — useful for cockpit instruments that ought to be attached to the vehicle. Requires knowing the anchor transform, which you generally do not in an injected renderer, unless you can identify a suitable matrix.

Native OpenXR quad layer. Submit the UI as a separate `XrCompositionLayerQuad` rather than compositing it into the projection layer. The runtime composites it at full display resolution with no warping and no resampling, which makes small text dramatically more legible. **This is the best option when available** and should be the default.

```

XrCompositionLayerQuad uiLayer{XR_TYPE_COMPOSITION_LAYER_QUAD};
uiLayer.layerFlags = XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT;
uiLayer.space = g_viewSpace; // head-relative
uiLayer.eyeVisibility = XR_EYE_VISIBILITY_BOTH;
uiLayer.subImage.swapchain = g_uiSwapchain;
uiLayer.subImage.imageRect = {{0,0},{uiW,uiH}};
uiLayer.pose.orientation = ToXrQuat(g_uiOrient);
uiLayer.pose.position = {0.0f, 0.0f, -g_config.uiDistanceMetres};
uiLayer.size = {g_config.uiWidthMetres, g_config.uiHeightMetres};

const XrCompositionLayerBaseHeader* layers[] = {
    (XrCompositionLayerBaseHeader*)&projLayer,
    (XrCompositionLayerBaseHeader*)&uiLayer,
};

```

Layer order matters: the quad must come after the projection layer to composite on top.

14.4 Exclusion masks for the warp

Some content should not be warped even when it stays in the main image: particles with additive blending, lens flares, volumetric god rays, and any effect whose screen-space appearance is not a projection of world geometry.

Maintain an **exclusion mask** — a single-channel texture written during draw interception. When a draw is classified as an excluded kind, write 1 to the mask in the region it covers. Then, in the warp, use the mask to decide behaviour:

```

float4 WarpWithExclusion(WarpVSOut i) : SV_Target {
    const float excluded = g_exclusionMask.SampleLevel(g_point, i.srcUV, 0);
    if (excluded > 0.5f) {
        // Do not warp: sample at the destination position instead of
    }
}

```

```

    // the source, effectively leaving this content screen-locked.
    return g_color.SampleLevel(g_linear, i.dstUV, 0);
}
return g_color.SampleLevel(g_linear, i.srcUV, 0);
}

```

Writing the mask requires knowing coverage, which requires either a depth-bounds trick or rendering the excluded draws twice — once into the scene, once into the mask with a trivial pixel shader. The second is simpler and cheap if the excluded draw count is small.

A cruder but often sufficient alternative: **detect additively-blended pixels post-hoc** by comparing the frame with and without... which you cannot do. In practice, if you cannot build a proper mask, the fallback is to accept that particles warp incorrectly. Additive particles warping is far less objectionable than a smeared HUD, because they lack sharp structure.

15 Synchronization and Scheduling

15.1 The concurrency problem

Your synthesis work must run on the GPU alongside the application's rendering, and must complete before the submission deadline. Three constraints collide:

1. The application owns the graphics queue and submits work you do not control.
2. Your synthesis depends on the application's output (colour, depth), so it must run *after* the application's frame completes.
3. Your synthesis must complete before `xrEndFrame`, which has a hard deadline.

The window between (2) and (3) is exactly the time between the application finishing and the display deadline. If the application uses 90% of the frame, you have 10% for synthesis — and that is when synthesis is most needed.

15.2 Async compute

The resolution is to run synthesis on a **separate queue** that executes concurrently with the application's *next* frame. Frame N 's synthesis overlaps frame $N+1$'s rendering, so the synthesis latency is hidden.

```

Graphics queue:  [App frame N      ][App frame N+1    ][App frame N+2    ]
Compute queue:   [Synth N          ][Synth N+1        ]
Submission:      ^ submit N        ^ submit N+1

```

This works because the display frame corresponding to app frame N is presented during app frame $N+1$'s rendering. You are not adding latency; you are using the pipeline depth you already have.

15.2.1 D3D12

```

// Create a dedicated compute queue.
D3D12_COMMAND_QUEUE_DESC qd{};
qd.Type      = D3D12_COMMAND_LIST_TYPE_COMPUTE;
qd.Priority  = D3D12_COMMAND_QUEUE_PRIORITY_HIGH;
device->CreateCommandQueue(&qd, IID_PPV_ARGS(&g_computeQueue));

// Per frame:
// 1. Application's graphics queue signals when its frame is done.
//    We piggyback on the application's own fence if we can observe it,
//    or insert our own signal by hooking ExecuteCommandLists.
g_appGraphicsQueue->Signal(g_crossFence.Get(), ++g_fenceValue);

// 2. Compute queue waits for that signal, then runs synthesis.
g_computeQueue->Wait(g_crossFence.Get(), g_fenceValue);
g_computeQueue->ExecuteCommandLists(1, &synthCmdList);
g_computeQueue->Signal(g_synthFence.Get(), ++g_synthValue);

// 3. Submission thread waits on the synthesis fence before xrEndFrame.
if (g_synthFence->GetCompletedValue() < g_synthValue) {
    g_synthFence->SetEventOnCompletion(g_synthValue, g_synthEvent);
    WaitForSingleObject(g_synthEvent, kDeadlineMs);
}

```

Resource state is the hazard. The application's colour and depth buffers are in graphics-queue states. To read them from a compute queue you need them in a state valid on the compute queue, and the compute queue supports only a subset of states. `D3D12_RESOURCE_STATE_NON_PIXEL_SHADER_RESOURCE` is valid on both; `PIXEL_SHADER_RESOURCE` is not valid on compute queues. Perform the transition on the graphics queue before signalling.

Also: resources in `COMMON` state can be accessed from any queue after a fence sync, which is the escape hatch when you cannot control the transitions. It costs a decompression on some hardware but it always works.

15.2.2 Vulkan

```
// Find a compute-capable queue family distinct from the graphics family.
// Async compute requires a family with COMPUTE but ideally not GRAPHICS.
uint32_t asyncFamily = FindQueueFamily(
    VK_QUEUE_COMPUTE_BIT, /*excludeFlags=*/VK_QUEUE_GRAPHICS_BIT);

// Ownership transfer is required for exclusive-sharing resources.
VkImageMemoryBarrier release{VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER};
release.srcQueueFamilyIndex = graphicsFamily;
release.dstQueueFamilyIndex = asyncFamily;
release.oldLayout = VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL;
release.newLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
release.image = appDepthImage;
// ... srcAccessMask, dstAccessMask, subresourceRange
vkCmdPipelineBarrier(graphicsCb, srcStage, dstStage, 0,
    0, nullptr, 0, nullptr, 1, &release);

// A matching ACQUIRE barrier must be recorded on the compute command buffer
// with identical family indices and layouts.
```

Queue family ownership transfer is mandatory for `VK_SHARING_MODE_EXCLUSIVE` resources and omitting it is undefined behaviour that works on some drivers and corrupts on others. If the application created its images with `VK_SHARING_MODE_CONCURRENT` you can skip it, but that is rare because concurrent sharing costs performance.

Use timeline semaphores (`VK_KHR_timeline_semaphore`, core in 1.2) for the cross-queue synchronisation — they are far easier to reason about than binary semaphores for this pattern.

15.2.3 D3D11

D3D11 has no async compute exposure. Your options:

1. **Run synthesis inline on the immediate context.** Simple, adds directly to the critical path. Acceptable if synthesis is under ~1 ms.
2. **Use a D3D11 deferred context** on a worker thread to build the command list, then execute inline. Saves CPU time, not GPU time.
3. **Interop to D3D12 or D3D11.3 with a shared resource.** Create a D3D12 device on the same adapter, share the application's textures via `IDXGIResource1::CreateSharedHandle` (requires the resource to have been created with `D3D11_RESOURCE_MISC_SHARED_NTHANDLE`, which the application's textures will not have), or copy into your own shared texture. The copy costs bandwidth but unlocks async compute.

Option 3 with a copy is the pragmatic choice for D3D11 targets when synthesis is expensive. The copy of colour+depth at 2000x2200 is roughly 0.25 ms; if it unlocks 2 ms of overlapped synthesis, it pays for itself.

15.3 Avoiding the pipeline bubble

A subtle failure: if your submission thread waits on the synthesis fence and the synthesis has not started because the graphics queue is still busy with the application's next frame, you can deadlock the pipeline into a lockstep where nothing overlaps.

Prevent this by giving the compute queue high priority and, on D3D12, considering `D3D12_COMMAND_QUEUE_PRIORITY_GLOBAL_REALTIME` (requires elevation and should be used cautiously — it can starve the compositor). More robustly: **always have a fallback ready**. If the synthesis fence has not signalled by the deadline, submit the previous frame's image rather than blocking. A repeated frame is far better than a missed `xrEndFrame`.

```
bool WaitForSynthesisOrFallback(double deadlineMs) {
    const double start = NowMs();
    while (NowMs() - start < deadlineMs) {
        if (g_synthFence->GetCompletedValue() >= g_synthValue) return true;
        _mm_pause();
    }
    g_stats.synthTimeouts++;
    return false; // caller submits the previous frame
}
```

Track `synthTimeouts` and feed it into the degradation ladder from Chapter 12. A rising timeout rate is the earliest signal that you are over budget.

16 Measurement and Validation

16.1 Instrumentation

You cannot tune what you cannot measure, and in an injected renderer the usual profilers are of limited use because they attribute all GPU work to the application. Build your own.

GPU timestamps around every synthesis stage. Use the pattern from Chapter 2, with a ring of query heaps deep enough to avoid stalling (four frames).

Frame timing histogram, not average. Report the 99th percentile and the maximum, because a single 25 ms frame is perceptually worse than a sustained 12 ms average. Track:

- Application frame interval (at `Present`)
- GPU time for the application's frame
- GPU time for each synthesis stage
- Time from `xrWaitFrame` return to `xrEndFrame` call
- Count of missed deadlines
- Count of runtime reprojections engaged (query from the runtime where available)

A HUD overlay rendered into your own composition layer, so you can see the numbers while wearing the headset. This is not optional; artifacts that are obvious in the headset are invisible on a monitor, and vice versa.

16.2 Correctness validation

16.2.1 Reference comparison

Build a mode that renders full stereo at full rate, and a mode that renders your synthesised version, and captures both from an identical deterministic camera path. Then compare per-frame.

The camera path must be deterministic and reproducible. If the application does not support demo playback, drive the camera yourself: since you already control the view matrix, you can override it entirely with a scripted path, ignoring the application's own camera. This gives you frame-exact reproducibility.

```
Mat4 ScriptedCameraOverride(uint64_t frameIndex) {
    const float t = float(frameIndex) / 90.0f;
    // A path exercising the worst cases: lateral strafe, then rotation,
    // then approach, then a near-field pass.
    if (t < 4.0f) {
        return LookAt(Vec3(sin(t * 1.5f) * 2.0f, 1.6f, 0), kOrigin, kUp);
    } else if (t < 8.0f) {
        const float yaw = (t - 4.0f) * 1.2f;
        return LookAtDir(Vec3(0, 1.6f, 0), YawToDir(yaw), kUp);
    } // ... etc
}
```

16.2.2 Metrics

Per-frame, per-eye, against the reference:

- **PSNR** — coarse, dominated by large uniform regions, but a good regression tripwire. A drop of more than 1 dB between builds warrants investigation.
- **SSIM** — better correlated with perception of structural artifacts.
- **Disocclusion area fraction** — the proportion of pixels filled by inpainting rather than warped. This is *the* number to watch for AER v2 quality; it should stay below 2–3% in typical content.
- **Inter-ocular difference metric** — compute SSIM between the left and right eyes of your *output*, and compare it to SSIM between left and right of the *reference*. Correct stereo has a characteristic inter-ocular similarity; if your output's inter-ocular similarity is much lower, you are introducing rivalry.
- **Temporal difference metric** — the mean absolute difference between consecutive output frames, compared to the reference. Higher than reference indicates judder or flicker; much lower indicates excessive smoothing or held frames.

The inter-ocular metric is the one most specific to AER and the one most worth building carefully:

$$Q_{\text{stereo}} = \frac{\text{SSIM}(I_L^{\text{out}}, I_R^{\text{out}})}{\text{SSIM}(I_L^{\text{ref}}, I_R^{\text{ref}})}$$

Values near 1.0 mean your stereo pair has the same degree of inter-ocular consistency as ground truth. Values below ~0.95 correlate with reported discomfort.

16.2.3 Latency measurement

Software measurement of motion-to-photon is unreliable. Measure it physically:

1. Mount a photodiode against the HMD lens, connected to an oscilloscope or a microcontroller ADC.
2. Rig the headset to a servo or pendulum that produces a repeatable, instrumented motion.
3. Render a scene where a specific screen region transitions black-to-white at a known head angle.
4. Measure the delay between the motion sensor's threshold crossing and the photodiode's response.

Repeat with generation enabled and disabled. The difference is your added (or removed) latency, measured rather than estimated. Do this once per major architectural change; it catches the class of bug where you believed you were extrapolating but were actually interpolating.

16.3 Artifact-specific test scenes

Build or find scenes that isolate each artifact:

Artifact	Test scene
A1 disparity shear	Depth-staggered poles, lateral strafe
A2 rivalry	Oblique checkerboard, slow pan
A3 specular	Polished sphere, orbiting camera
A4 screen-space	Reflective floor, object crossing frame edge
A5 TAA	Fine foliage, moderate camera motion
A7 resonance	Strobe light at $f_a/2$, rotating fan
Disocclusion	Near pole against distant wall, lateral motion
Flow failure	Fast-moving small object, low-texture background
Off-screen intake	Rapid 180-degree turn
HUD	Dense small text overlay during rapid motion

A synthetic test harness that renders these directly — bypassing the target application entirely — is worth building. It gives you a fast iteration loop and removes the application as a variable.

16.4 Subjective evaluation

Objective metrics do not capture discomfort. Structure subjective testing:

- **Paired comparison, blinded.** Present configuration A and B in randomised order, ask which is preferable. Far more reliable than absolute ratings.
- **Session duration to discomfort.** The most meaningful single number. Have testers report on a simple scale at fixed intervals; the time to reach a given level is a robust metric.
- **Test with people who are sensitive.** A tester who never gets uncomfortable provides no signal. Recruit for sensitivity deliberately.
- **Test the transitions.** Quality-level changes, mode switches, and hitches are often more objectionable than any steady state, and are systematically missed in short evaluations.

Stop any session immediately if a tester reports nausea. Discomfort in VR testing can persist for hours and there is no research value in pushing through it.

17 Failure Modes and Practical Hardening

17.1 The things that will break

Camera identification loses lock. The application changes rendering mode — enters a cutscene, opens a menu, switches to a different camera system — and your fingerprinted constant buffer is no longer the one in use. Symptom: the world freezes or goes wild while the head still moves.

Hardening: continuously validate. Every frame, check that the identified matrix still passes the fingerprint test and that its derived camera position moved plausibly. On failure, immediately drop to a safe state (disable stereo, submit a static or head-locked view) and re-run identification. **Never continue submitting frames from a matrix you no longer trust.**

```
bool ValidateCameraLock(const Mat4& current, const CameraTrack& track) {
    if (ScoreAsViewProjection(&current[0][0]) < 0.7f) return false;

    const Vec3 pos = ExtractCameraPosition(current);
    const float step = Length(pos - track.lastPos);
    // A camera that teleports every frame is a false lock.
    if (step > track.plausibleMaxStep) {
        if (++track.implausibleCount > 5) return false;
    } else {
```

```

    track.implausibleCount = 0;
}
track.lastPos = pos;
return true;
}

```

Resolution change. The application changes its render resolution and all your render targets are the wrong size. Hook the resize path (ResizeBuffers, swapchain recreation) and rebuild everything. Vulkan applications may recreate the swapchain without any obvious signal; watch `vkCreateSwapchainKHR`.

Device loss. GPU hangs, driver updates, and TDRs happen. Handle `DXGI_ERROR_DEVICE_REMOVED` and `VK_ERROR_DEVICE_LOST` by tearing down all your resources and re-establishing. If you hold references to application resources across device loss you will crash.

The application renders nothing. Loading screens, alt-tab, minimised windows. `frameState.shouldRender` from `xrWaitFrame` tells you the runtime’s view; also detect zero draw calls and submit a held frame or a neutral background rather than garbage.

Multi-threaded submission. The application builds command lists on many threads. All your per-context state must be thread-local, all your shared state must be properly synchronised, and your frame-boundary detection must handle out-of-order recording. This is the most common source of rare, hard-to-reproduce crashes.

Anti-cheat and anti-tamper. Injecting into a protected process may be detected and may be a terms-of-service violation. Be aware of what you are attaching to.

17.2 Numerical hazards

Depth precision. With reversed-Z float depth and an infinite far plane, precision is excellent near the camera and degrades gracefully. With forward-Z UNORM24 depth — still used by older applications — precision at distance is poor and reprojection produces visible quantisation banding in the warp. Detect the depth format and, if it is forward-Z 24-bit, reduce the maximum reprojection distance or accept the artifacts.

Denormals and infinities in the flow field. A single NaN in the flow propagates through smoothing and destroys a large region. Sanitise:

```

float2 SanitizeFlow(float2 f) {
    f = select(isfinite(f), f, float2(0, 0));
    const float m = length(f);
    return (m > g_maxFlowMagnitude) ? f * (g_maxFlowMagnitude / m) : f;
}

```

Clamp maximum flow magnitude to something physically plausible — roughly the screen width times the maximum plausible angular rate times the frame interval. Anything larger is an estimation failure, not real motion.

Matrix inversion conditioning. The reprojection matrix $(V_A P_A)^{-1} (V_B P_B)$ involves inverting a projection matrix, which is ill-conditioned when the near plane is very small relative to the far plane. Invert the view and projection separately using their known structure rather than a general 4x4 inverse:

$$V^{-1} = \begin{bmatrix} R^T & 0 \\ -tR^T & 1 \end{bmatrix}, \quad P^{-1} \text{ has a known sparse closed form}$$

This is both faster and dramatically more numerically stable than a general inverse.

17.3 Configuration surface

Expose, at minimum:

- World scale (metres per application unit) — the single most important setting.
- IPD override.
- AER mode: off / v1 / v2 / adaptive.
- Frame generation: off / interpolate / extrapolate, and target ratio.
- z_{split} for the near-field both-eyes pass.
- UI distance, size, and lag time constant.
- Quality-level floor and ceiling.
- Per-pass classification overrides, for applications the heuristics misjudge.

That last item is what turns a technique demo into something that works on more than one application. Ship a config file format that lets a user (or a community) declare “draw calls with this shader hash are UI” and “this constant buffer slot holds the view matrix”, and the general heuristics become a fallback rather than the whole system.

17.4 Ethical and legal notes

Two things worth stating plainly:

Comfort is a safety issue. VR discomfort is not merely an inconvenience; severe cases produce disorientation lasting hours and there is a real risk of injury from falls. If your technique produces artifacts you know to be uncomfortable, gate them behind an

informed setting, default to the conservative option, and make the exit path obvious.

Respect the software you inject into. Reverse engineering law varies by jurisdiction and interoperability exceptions are narrower than people assume. Modifying and distributing modifications to software you do not own is governed by that software's licence, not by convention. Build against your own targets, check the terms, and do not redistribute anything derived from another party's binary.

18 The Complete Stereo Mode Taxonomy

Chapters 6 through 9 treated alternate-eye rendering as a single technique with two variants. That framing is too coarse. In practice the design space is defined by three independent axes, and most of the confusion in this field comes from conflating them.

18.1 The three axes

Axis 1 — Tick coupling. Do the two eyes represent the same simulation state?

- **Same-tick:** both eyes are rendered from one advancement of game logic. Object positions, animation poses, particle states, and physics are byte-identical between the eyes. Only the camera differs.
- **Cross-tick:** each eye is rendered from a different advancement of game logic. The eyes differ not only in camera but in world state.

Axis 2 — Submission coupling. Are the two eye images submitted to the compositor together?

- **Paired:** both eyes are submitted in the same `xrEndFrame`, and the pair is displayed simultaneously.
- **Independent:** each eye is submitted when ready; a display frame may pair a new image for one eye with a previously submitted image for the other.

Axis 3 — Synthesis. Is either eye image synthesised rather than rendered?

- **None:** both images come from the engine.
- **Reprojected:** one image is geometrically warped from the other, or from an older frame of the same eye.
- **Interpolated / extrapolated:** a temporal synthesis stage generates additional frames.

18.2 The resulting modes

Mode	Tick	Submission	Synthesis	Engine renders per display frame
M0 Naive simultaneous stereo	Same	Paired	None	2 views
M1 Synchronized sequential	Same	Paired	None	2 views, sequentially
M2 Classic AER	Cross	Paired	None (hold)	1 view
M3 Asynchronous AER	Cross	Independent	None	1 view
M4 AER + stereo reprojection	Cross	Paired	Reprojected	1 view
M5 Alternate frame warp	Cross	Paired	Reprojected (pose)	1 view
M6 AER + temporal generation	Cross	Paired	Both	1 view per 2 display frames

Mode	Tick	Submission	Synthesis	Engine renders per display frame
M7 Half-rate paired	Same	Paired	None (hold both)	2 views per 2 display frames

Chapters 6–9 covered M2, M4, and M6. This part covers M1, M3, and M5 in full, because they are the modes that most often turn out to be the right answer, and because M1 in particular is frequently mislabelled as AER when it is the opposite of AER.

18.3 The key insight about M1

There is a widely repeated observation among practitioners that deserves to be stated precisely:

Games are always AER-shaped, because they do not have two rendering pipelines. If you lock gameplay and render both eyes so they are on the same tick, then wait for both eyes to finish their own pipeline before presenting to OpenXR, you have a synchronised sequential pair — which is basically stereo with no AER.

Unpack that. An injected renderer cannot make a single-view engine render two views *simultaneously*, because the engine has exactly one view pipeline: one frame graph, one set of transient render targets, one camera constant buffer, one G-buffer. The only thing you can do is run that pipeline **twice, sequentially**.

Running it twice sequentially is normally what produces AER’s problems — because between the two runs, the game ticks, and the eyes end up representing different moments. But **if you prevent the tick from advancing between the two runs**, the two sequential renders produce the same world state. You get true stereo. The sequential-ness costs you throughput but not correctness.

That is the whole idea, and it reframes AER: **AER is not a rendering technique. It is what you get when sequential per-eye rendering is allowed to straddle a simulation tick.** Eliminate the straddle and AER’s characteristic artifacts — disparity shear, rivalry, resonance — vanish entirely, because their common cause was the tick difference, not the sequencing.

The cost is exactly what you would expect: two full view renders per display frame, so the pair rate is half the engine’s mono frame rate. No amortisation. But the image is correct.

18.4 Choosing between the families

If your bottleneck is...	Choose
GPU, and content is rotation-dominant	M4 or M5
GPU, and content has fast lateral motion	M1 if you can afford it, else M4
GPU, and you have headroom above 2x	M1
CPU / draw submission	M1 helps least; M2/M4 help most
Latency	M5
Display rate exceeds achievable render rate	M6
Simulation determinism matters (physics, netcode)	M1, carefully; avoid tick manipulation otherwise

A production implementation should offer M1, M4, M5, and M6, with M2 as a diagnostic fallback, and should select among them adaptively.

19 Synchronized Sequential Stereo

This chapter is the longest in the document because it contains the most engine-facing work. Everything else in this guide operates on images; this operates on the engine’s control flow.

19.1 The target architecture

Display frame N:

```
[tick lock engaged]
Engine tick -----> advances world state ONCE
Render invocation 1 --> camera = LEFT eye --> full pipeline --> RT_L
Render invocation 2 --> camera = RIGHT eye --> full pipeline --> RT_R
[wait for both GPU pipelines to complete]
Single xrEndFrame with {RT_L, RT_R}, one displayTime, matched poses
[tick lock released]
```

Three requirements, each of which is a substantial engineering problem:

1. **Tick lock.** Prevent simulation advancement between the two render invocations.

2. **Render re-entrancy.** Cause the engine to execute its render pipeline a second time within one of its own frames, with a different camera.
3. **Paired submission.** Hold both results and submit them together, with correct per-eye poses derived from a single predicted display time.

Requirement 2 is the hard one and is treated separately in Chapter 25 because it also governs M0. This chapter covers 1 and 3, plus the interaction between them.

19.2 Tick locking

19.2.1 What “tick” means

Engines advance simulation in one of three patterns:

Variable timestep. Δt measured from the previous frame’s wall-clock duration; everything scales by Δt . Most common in single-player action games. The lock is easy: force $\Delta t = 0$ for the second invocation.

Fixed timestep with accumulator. The engine accumulates real elapsed time and runs N fixed steps of size h when the accumulator exceeds h , then interpolates render state between the last two steps by the leftover fraction. Common in physics-heavy titles. The lock must prevent the accumulator from advancing *and* must freeze the interpolation alpha, or the second eye will render an interpolated pose fractionally ahead of the first.

Decoupled simulation thread. Simulation runs on its own thread at its own rate; the render thread samples a snapshot. The lock must prevent snapshot rotation between the two invocations — effectively, pin the render thread to one snapshot.

19.2.2 Finding the timestep

You need the variable the engine reads to determine elapsed time. Three approaches in increasing generality:

Hook the OS timer. Most engines ultimately call `QueryPerformanceCounter`, `timeGetTime`, `GetTickCount64`, OR `std::chrono::steady_clock::now` (which lowers to `QueryPerformanceCounter` on Windows). Hook it and return a frozen value during the second render invocation.

```
static LARGE_INTEGER g_frozenQpc{};
static std::atomic<bool> g_timeFrozen{false};

BOOL WINAPI Hooked_QueryPerformanceCounter(LARGE_INTEGER* out) {
    if (g_timeFrozen.load(std::memory_order_acquire)) {
        *out = g_frozenQpc;
        return TRUE;
    }
    const BOOL r = g_realQpc(out);
    g_frozenQpc = *out;    // remember the last real value
    return r;
}
```

This is blunt and effective, and it has two hazards. First, **everything** in the process sees frozen time, including audio mixing, network keepalives, the OpenXR runtime’s own timing if it shares the process, and any watchdog that concludes the process has hung. Second, if the engine computes Δt as a difference and you freeze mid-computation, you can produce $\Delta t = 0$ in code paths that divide by it.

Mitigate by scoping the freeze as tightly as possible — engage it immediately before the second render invocation and release it immediately after — and by filtering on the calling thread:

```
BOOL WINAPI Hooked_QueryPerformanceCounter(LARGE_INTEGER* out) {
    if (g_timeFrozen.load(std::memory_order_acquire)
        && GetCurrentThreadId() == g_renderThreadId) {
        *out = g_frozenQpc;
        return TRUE;
    }
    return g_realQpc(out);
}
```

Restricting to the render thread leaves audio and networking on real time, which resolves most of the collateral damage.

Patch the delta-time variable directly. If you can locate the engine’s Δt field — by pattern scanning for a float that tracks frame duration, or by disassembling around the timer call site — write zero to it before the second invocation. Cleanest, but application-specific.

To find it generically: hook the timer, and on each call record the return address. The engine’s frame-timing call site will be the one that fires exactly once per `Present`. From that call site, the value flows into a `double` or `float` field within a few instructions; a small amount of static analysis, or a memory scan for a float that correlates with measured frame duration, locates it.

```
// Scan for a float field whose value tracks the measured frame interval.
struct DtCandidate { uintptr_t addr; float samples[16]; int n; float score; };

void ScoreDtCandidates(std::vector<DtCandidate>& cands, float measuredDt) {
    for (auto& c : cands) {
        const float v = *reinterpret_cast<float*>(c.addr);
        if (!std::isfinite(v) || v < 0.0f || v > 1.0f) { c.score = 0; continue; }
        // Correlate against the true interval; also accept ms-scaled variants.
        const float e1 = fabsf(v - measuredDt);
        const float e2 = fabsf(v - measuredDt * 1000.0f);
        c.score += (std::min(e1, e2) < measuredDt * 0.05f) ? 1.0f : -0.5f;
    }
}
```

Hook the tick function. If you can identify the engine’s Tick, Update, Or AdvanceFrame entry point, simply do not call it for the second invocation. This is the most surgical option and produces the fewest side effects, but requires identifying a specific function in a specific binary.

19.2.3 Freezing the accumulator

For fixed-timestep engines, zeroing Δt is not sufficient, because the accumulator may already hold enough time for another step. You must additionally ensure the step loop runs zero times on the second invocation. If you hooked the timer, this happens automatically — the accumulator receives zero new time and the loop condition fails. If you patched Δt directly, verify.

The interpolation alpha is the subtler issue. Render state is typically

$$\mathbf{x}_{\text{render}} = (1 - \alpha) \mathbf{x}_{n-1} + \alpha \mathbf{x}_n, \quad \alpha = \frac{\text{accumulator}}{h}$$

If α changes between the two eye renders, objects move between the eyes even though no simulation step occurred. Freezing the accumulator freezes α , so freezing time correctly handles this — but if you patched only Δt , you must freeze α separately.

19.2.4 Verifying the lock

Do not assume the lock works. Verify it empirically with a differential test:

1. Render eye L, capture the frame.
2. Engage the lock.
3. Render eye L again — the *same* eye, not the other.
4. Compare the two images.

If the lock is perfect, the two images are bit-identical. Any difference is simulation advancement or non-determinism leaking through. Run this test with a scene containing animated characters, particles, physics debris, and flowing water — the four things most likely to advance.

```
bool VerifyTickLock(FrameContext& fc) {
    RenderEyeTo(Eye::Left, g_probeA);
    EngageTickLock();
    RenderEyeTo(Eye::Left, g_probeB);
    ReleaseTickLock();

    const uint64_t hashA = HashTextureGpu(g_probeA);
    const uint64_t hashB = HashTextureGpu(g_probeB);
    if (hashA != hashB) {
        LogDiffRegions(g_probeA, g_probeB); // localise the leak
        return false;
    }
    return true;
}
```

LogDiffRegions is worth building: it tells you *which part of the screen* is advancing, which usually identifies the subsystem immediately. Particles in the upper third, a flag animating on the left, a character’s idle breathing.

19.2.5 Residual non-determinism

Some things advance regardless of Δt :

- **Per-frame counters** used to drive noise, dithering, TAA jitter, or stochastic effects. These increment on frame count, not time. The second eye gets the next index in the sequence. For TAA jitter this is actually *correct* — you want different jitter per eye anyway — but for blue-noise dithering it produces inter-ocular noise differences that read as film grain in one eye only.
- **GPU-driven simulation.** Particle systems, cloth, and hair simulated in compute shaders advance when their dispatch runs, independent of CPU-side Δt . Two render invocations means two dispatches. You must either skip the simulation dispatch on the second invocation, or accept the difference.
- **Random number generators** consumed during rendering.
- **Streaming and LOD transitions** driven by frame counters or by distance hysteresis with per-frame updates.

For GPU simulation the fix is to classify those dispatches (they write to buffers they also read, and they run before the geometry pass) and skip them on the second invocation:

```
void STDMETHODCALLTYPE Hooked_Dispatch(
    ID3D12GraphicsCommandList* cl, UINT x, UINT y, UINT z)
{
    if (g_secondEyeInvocation && IsSimulationDispatch(g_currentPso)) {
        return;           // skip: state already advanced for eye 1
    }
    g_realDispatch(cl, x, y, z);
}
```

Identifying `IsSimulationDispatch` generically is hard. A workable heuristic: a dispatch whose bound UAVs are also bound as SRVs in the same or an adjacent dispatch, and which does not write to any texture used as a render target that frame. Build a list at runtime, verify with the differential test above, and cache by PSO hash.

For frame counters, hook the counter increment or simply accept the difference — it is usually below perceptual threshold for everything except heavy dithering.

19.3 The double render invocation

Chapter 25 covers the general problem. For the same-tick case specifically, the sequence within one engine frame is:

```
Engine frame begins
Engine tick (simulation advances)
Engine render pass tree executes          <- invocation 1, eye L
Engine calls Present                      <- YOUR HOOK
|
+-- capture RT_L
+-- engage tick lock
+-- set camera to eye R
+-- re-invoke the render pass tree        <- invocation 2, eye R
+-- capture RT_R
+-- release tick lock
+-- fence: wait for both invocations' GPU work
+-- xrEndFrame({RT_L, RT_R})
+-- suppress the real Present (or present a mirror view)
Engine frame ends
```

Re-invoking from inside the `Present` hook is the most reliable placement, because at that point the engine has finished all its rendering for the frame and its render state is in a known, quiescent condition. Re-invoking mid-frame is possible but requires far more care about state restoration.

What does “re-invoke the render pass tree” mean concretely? Four techniques, in increasing order of invasiveness:

1. **Call the engine’s render entry point again.** If you can identify a function like `RenderFrame(SceneView*)` OR `FrameGraph::Execute()`, call it directly with your modified camera. Requires the function to be re-entrant, which is the whole difficulty.
2. **Command list replay.** Record the engine’s command lists, then submit them again with a patched camera constant buffer. Fails on most modern engines for reasons detailed in Chapter 25 (transient resources, aliased memory, one-shot command lists, async compute dependencies).
3. **Deferred present.** Do not re-invoke at all. Instead, let the engine render its next frame naturally, but hold the tick lock across the *whole* frame so the second frame is a same-tick render of the other eye. This turns a re-entrancy problem into a pacing problem, and it is by far the most robust technique for an injected renderer.
4. **Native second view.** Cause the engine to register a second view in its own view list, so it renders both in one frame graph execution. Highest quality, allows the engine to share view-independent work, and is usually blocked by engine architecture. Chapter 25.

19.3.1 Technique 3 in detail: the held-tick frame pair

This deserves elaboration because it is the practical answer for most targets.

```

Engine frame N:    tick advances, render eye L, Present hooked -> capture, no submit
                  [engage tick lock]
Engine frame N+1:  tick does NOT advance, render eye R, Present hooked -> capture
                  [release tick lock]
                  -> both eyes captured from the same simulation state
                  -> submit paired to OpenXR
Engine frame N+2:  tick advances, render eye L, ...

```

You never re-invoke anything. The engine runs its normal frame loop; you simply prevent simulation advancement on every other frame and pair up the results. The engine's own frame rate is $2f_{\text{pair}}$, and the pair rate — the rate at which new stereo content reaches the display — is half that.

The camera must be set to the correct eye at the *start* of each engine frame, which means deciding at the previous *Present*, exactly as in Chapter 7.

```

void OnPresent_SyncSequential(FrameContext& fc) {
    if (g_phase == Phase::FirstEye) {
        CaptureEye(g_eyeOrder[0], fc);
        EngageTickLock();
        g_pendingCameraEye = g_eyeOrder[1];
        g_phase = Phase::SecondEye;
        SuppressPresent(fc);      // nothing to show yet
    } else {
        CaptureEye(g_eyeOrder[1], fc);
        ReleaseTickLock();
        WaitForBothEyeFences();
        SubmitPairedStereo(fc);
        g_pendingCameraEye = g_eyeOrder[0];
        g_phase = Phase::FirstEye;
    }
}

```

The critical property: because the tick is locked across the boundary, the two captures share simulation state exactly. This is genuine stereo. The disparity error of Chapter 6 is identically zero.

The cost: the pair rate is half the engine's frame rate, and the engine's frame rate is roughly its mono rate (each frame renders one view). So

$$f_{\text{pair}} \approx \frac{f_{\text{mono}}}{2}$$

which is the same throughput as naive simultaneous stereo. You have not saved anything. What you have gained is that you never needed the engine to be re-entrant.

19.3.2 Sharing view-independent work

There is one optimisation available and it is smaller than intuition suggests. During the second (locked) frame, view-independent work — shadow map rendering, GPU skinning, particle simulation, environment probe updates — is recomputed identically. If you can skip it, you save C_i .

Measured across representative deferred-renderer content, view-independent passes account for roughly **6–12%** of frame GPU time when shadow maps are cached and skinning is reused. The reason it is so low is that in a modern renderer the overwhelming majority of cost is in view-dependent lighting, screen-space effects, and post-processing, and shadow maps — the biggest genuinely shareable item — are often already cached across frames by the engine itself.

Concretely: caching shadow maps for the second eye means detecting the shadow passes (Chapter 5's classifier) and skipping them, reusing the previous frame's depth targets. This works only if the engine does not alias that memory for something else in between, which you must verify. The payoff is real but modest; implement it after everything else works.

```

bool ShouldSkipPassOnSecondEye(PassKind kind, const BoundState& s) {
    if (!g_secondEyeInvocation) return false;
    switch (kind) {
        case PassKind::ShadowMap:
            return g_config.shareShadowMaps && !ShadowTargetAliased(s);
        case PassKind::Capture:
            return g_config.shareProbes;      // cubemap / probe updates
        default:
            return false;
    }
}

```

19.4 Paired submission

Once both eyes are captured, submission is straightforward but has three subtleties.

One display time, two poses. Call `xrLocateViews` once, with the single `predictedDisplayTime` for the pair. Both eyes get poses from that one call, so they are consistent. Do *not* locate views separately for each eye at different times — that reintroduces exactly the temporal disparity error you eliminated.

```
void SubmitPairedStereo(FrameContext& fc) {
    XrViewLocateInfo li{XR_TYPE_VIEW_LOCATE_INFO};
    li.viewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
    li.displayTime = fc.predictedDisplayTime;    // ONE time for both eyes
    li.space = g_appSpace;

    XrViewState vs{XR_TYPE_VIEW_STATE};
    uint32_t n = 2;
    XrView views[2] = {{XR_TYPE_VIEW},{XR_TYPE_VIEW}};
    xrLocateViews(g_session, &li, &vs, 2, &n, views);

    for (int e = 0; e < 2; ++e) {
        AcquireWaitBlitRelease(g_swapchain[e], g_eyeCapture[e]);
        g_projViews[e].pose = views[e].pose;
        g_projViews[e].fov = views[e].fov;
    }
    // ... xrEndFrame with both
}
```

The camera used to render must match the pose submitted. This is the part that goes wrong most often. You rendered eye L with a camera derived from a pose located at some earlier time — probably the previous frame’s `predictedDisplayTime`, because you had to set the camera before the frame began. If you submit the *newly located* pose, the runtime believes the image corresponds to a pose it does not.

Two correct resolutions:

1. **Submit the render pose.** Store the pose you actually rendered with and submit that. The runtime’s reprojection then corrects the difference. Simple and correct.
2. **Warp to the new pose.** Reproject both eyes from the render pose to the freshly located pose before submission, then submit the new pose. This is Chapter 26’s alternate frame warp applied to both eyes, and it reduces latency. Better, more work.

Do not do the third thing — render with pose A, submit pose B, no warp — which produces a world that lags and swims by exactly the pose difference.

Wait for both pipelines, not just the second. The first eye’s GPU work was submitted an engine frame earlier and has almost certainly completed, but “almost certainly” is not a synchronisation primitive. Take a fence value after each eye’s capture and wait on both.

```
struct EyeCapture {
    GpuTexture    color;
    GpuTexture    depth;
    uint64_t      fenceValue;
    XrPosef       renderPose;
    Mat4          view, proj;
};

void WaitForBothEyeFences() {
    const uint64_t target = std::max(g_capture[0].fenceValue,
                                     g_capture[1].fenceValue);
    if (g_fence->GetCompletedValue() < target) {
        g_fence->SetEventOnCompletion(target, g_fenceEvent);
        WaitForSingleObject(g_fenceEvent, 100);
    }
}
```

In D3D11, where you cannot signal a fence explicitly on the immediate context pre-11.3, use `ID3D11Query` of type `D3D11_QUERY_EVENT` after each capture and poll it. In Vulkan, use a timeline semaphore signalled on each eye’s submission.

19.5 Latency analysis

Same-tick sequential stereo has a specific latency structure worth deriving, because it is worse than it first appears.

Let τ be the engine’s per-frame time (one view). The first eye is rendered during $[0, \tau]$, the second during $[\tau, 2\tau]$, and submission happens at 2τ . The simulation state both eyes represent is from time 0.

$$T_{\text{sim-to-submit}} = 2\tau$$

Compare to naive simultaneous stereo, where both views render concurrently within one frame of duration $\approx 2\tau$ (same total work), and submission is also at 2τ . **The same.** Sequential rendering does not cost extra latency relative to simultaneous rendering of the same total work, because the work is the same; it is merely serialised differently.

But compare to mono: mono submits at τ . So stereo of either kind doubles simulation-to-submit latency relative to mono, which is the unavoidable cost of rendering twice as much.

The important consequence: **the pose you render eye L with is 2τ old at submission time.** At $\tau = 11$ ms, that is 22 ms of pose staleness. This is why combining M1 with a final warp to the freshly-located pose (resolution 2 above) is not a refinement but close to a necessity. The warp costs 0.3 ms and removes 22 ms of rotational latency.

19.6 When same-tick sequential is the right choice

- The engine renders fast enough that half its mono rate still meets or exceeds a comfortable display rate. If mono runs at 120 fps, the pair rate is 60 — usable with generation to reach 90 or 120.
- Content has significant lateral motion or high-frequency detail, where AER's artifacts are worst.
- The engine tolerates tick manipulation without breaking physics or netcode.
- You cannot solve engine re-entrancy, which rules out M0.

It is the wrong choice when the engine is CPU-bound, because you double CPU frame cost with no amortisation whatsoever, and when tick locking causes side effects you cannot suppress.

19.7 Combining with temporal generation

M1 pairs naturally with Chapter 10's frame generation, and this is the most practically important combination in the whole document:

```
Engine frame 2N:    tick advances, render eye L
Engine frame 2N+1: tick locked,   render eye R
                  -> submit true stereo pair at display frame k
                  -> generate one extrapolated stereo pair -> display frame k+1
```

Engine at 90 fps mono $\rightarrow$ 45 Hz true stereo pairs $\rightarrow$ 90 Hz display via one generated frame per pair. You get geometrically correct stereo with zero disparity error, at full display rate, for the cost of one engine frame per eye plus roughly 1.5 ms of synthesis.

The generation stage is *easier* here than under AER, because both eyes share a timestamp: the flow field computed for one eye transfers cleanly to the other, and there is no eye-alternation phase to reason about.

This configuration — **same-tick sequential stereo plus extrapolated frame generation** — is the recommended default for content that can sustain it, and it should be the first target you build toward once the basic pipeline works. # Asynchronous Alternate-Eye Rendering

19.8 Definition and motivation

Classic AER (M2) renders one eye per engine frame and submits a paired stereo image every display frame, holding the stale eye. Asynchronous AER (M3) removes the pairing constraint: **each eye is treated as an independent stream**, submitted when it is ready, and the compositor is handed whatever the freshest image for each eye happens to be.

The motivation is throughput smoothing. Under classic AER, submission is gated on a fixed alternation schedule; if one eye's render overruns, the whole pair is late. Under async AER, an overrun in one eye does not delay the other. In practice this converts a hard hitch into a brief per-eye staleness, which is much less objectionable.

The second motivation is that some engines produce frames at an irregular cadence — a heavy frame followed by two light ones is common with streaming and shader compilation. Async AER absorbs that irregularity into per-eye staleness rather than into pair-rate jitter.

19.9 Architecture

```
Engine thread:  [render L]---[render R]---[render L]-[render R]--[render L]---...
                |           |           |           |           |
                v           v           v           v           v
            ring[L][0]  ring[R][0]  ring[L][1]  ring[R][1]  ring[L][2]

Submit thread:  xrWaitFrame -> pick newest complete per eye -> xrEndFrame
                |-----|-----|-----|-----|-----|-----| (display rate)
```

Two independent per-eye ring buffers. The submission thread runs at display rate and picks, for each eye independently, the newest ring entry whose GPU work has completed.

```
struct EyeSlot {
    GpuTexture color;
    GpuTexture depth;
```

```

XrPosef    renderPose;
Mat4       view, proj;
double     captureTime;      // monotonic seconds
uint64_t   tickId;          // engine simulation tick identity
uint64_t   fenceValue;
std::atomic<bool> published;
};

class EyeStream {
    static constexpr int N = 3;
    EyeSlot slots[N];
    std::atomic<uint32_t> writeIdx{0};
public:
    EyeSlot* BeginWrite() { return &slots[writeIdx.load() % N]; }
    void EndWrite(uint64_t fence) {
        slots[writeIdx.load() % N].fenceValue = fence;
        slots[writeIdx.load() % N].published.store(true,
            std::memory_order_release);
        writeIdx.fetch_add(1, std::memory_order_acq_rel);
    }
    // Newest slot whose GPU work has completed.
    const EyeSlot* NewestComplete(uint64_t completedFence) const {
        const uint32_t w = writeIdx.load(std::memory_order_acquire);
        for (int back = 1; back <= N; ++back) {
            const EyeSlot& s = slots[(w - back) % N];
            if (s.published.load(std::memory_order_acquire)
                && s.fenceValue <= completedFence)
                return &s;
        }
        return nullptr;
    }
};

```

Three slots per eye is the minimum: one being written, one complete and submittable, one still referenced by the compositor.

19.10 The skew problem

Async AER's characteristic failure is **unbounded eye skew**. If the engine happens to render three left-eye frames in a row — because your scheduler is adaptive, or because a right-eye frame was dropped — the right eye's newest complete image becomes arbitrarily old, and the disparity error of Chapter 6 grows without bound.

Define the skew as the difference in capture time between the two submitted eyes:

$$\sigma = |t_{\text{capture}}^L - t_{\text{capture}}^R|$$

Under strict alternation with a stable engine, $\sigma \approx \tau$, one engine frame. Under async operation it can spike.

Recall the disparity-error ratio from Chapter 6, now expressed in terms of skew:

$$\frac{\epsilon}{\theta} = \frac{v \sigma}{b}$$

With $b = 0.064$ m and a tolerance of, say, 15% disparity error at $v = 1$ m/s, the maximum acceptable skew is $\sigma_{\text{max}} = 0.15 \times 0.064 / 1.0 = 9.6$ ms. That is *less than one engine frame at 90 fps*, which tells you immediately that async AER needs active skew management, not merely a hope that the engine behaves.

19.10.1 Bounded-skew scheduling

Enforce the bound in the eye scheduler:

```

Eye ScheduleEyeAsync(const EyeStream& L, const EyeStream& R, double now) {
    const double ageL = now - L.NewestPublishedTime();
    const double ageR = now - R.NewestPublishedTime();

    // Always render whichever eye is staler. This self-corrects skew
    // without any explicit alternation rule.
    if (ageL > ageR + g_config.skewHysteresis) return Eye::Left;
    if (ageR > ageL + g_config.skewHysteresis) return Eye::Right;
}

```

```
// Tie: alternate.
return g_lastRendered == Eye::Left ? Eye::Right : Eye::Left;
}
```

“Render the staler eye” is a simple and effective policy: it degenerates to strict alternation when the engine is stable and self-corrects automatically when it is not. The hysteresis term prevents thrashing when the ages are nearly equal.

19.10.2 Skew-driven degradation

When measured skew exceeds tolerance — because a hitch was long enough that no scheduling policy could recover — you must do something other than submit a badly-skewed pair. Options, in order of preference:

1. **Warp the stale eye forward.** You have its depth and its render pose; reproject it to the current predicted pose. This is Chapter 26’s alternate frame warp and it converts temporal skew into a reprojection error, which is far more benign.
2. **Synthesise the stale eye from the fresh one.** Chapter 9’s stereo warp. Removes skew entirely at the cost of disocclusion artifacts.
3. **Fall back to half-rate paired.** Submit the *older* image for *both* eyes, so the pair is temporally consistent even though both are stale. Zero disparity error, whole-image judder. Often the least objectionable option during a hitch.

```
SubmissionPlan PlanSubmission(const EyeStream& L, const EyeStream& R, double now) {
    const EyeSlot* sl = L.NewestComplete(g_completedFence);
    const EyeSlot* sr = R.NewestComplete(g_completedFence);
    if (!sl || !sr) return SubmissionPlan::RepeatPrevious();

    const double skew = fabs(sl->captureTime - sr->captureTime);

    if (skew <= g_config.skewOk)         return SubmissionPlan::Direct(sl, sr);
    if (skew <= g_config.skewWarpable)    return SubmissionPlan::WarpStale(sl, sr);
    if (g_config.allowStereoSynthesis)     return SubmissionPlan::SynthesiseStale(sl, sr);
    return SubmissionPlan::HalfRatePaired(OlderOf(sl, sr));
}
```

Thresholds that work as starting points: skewOk = 6 ms, skewWarpable = 25 ms.

19.11 Tick identity and matched poses

Async AER makes explicit something classic AER hides: the two eyes represent different simulation ticks. Recording a tickId per slot lets you reason about it.

If the engine exposes a frame counter, or if you can derive one from the number of `Present` calls, tag each capture. Then at submission you know exactly how many ticks apart the eyes are, which is a more reliable staleness measure than wall-clock time — it is immune to frame-time jitter.

19.11.1 Forced matched eye poses

A related option worth implementing: when synthesising or warping the stale eye, you can choose to force both eyes to use **poses derived from a single head pose**, differing only by the eye offset, rather than each eye’s own render pose.

- **Independent poses:** each eye is submitted with the pose it was rendered with. The runtime reprojects each independently. Preserves whatever the engine actually drew, but the two eyes are reprojected differently, which can introduce relative distortion.
- **Matched poses:** both eyes are warped to a single common head pose (plus their respective eye offsets) before submission. Guarantees that the stereo geometry at display time is internally consistent, at the cost of a warp on both eyes.

Matched poses are almost always the better choice when you are warping anyway, because the whole point is to eliminate inter-ocular inconsistency. Independent poses are appropriate only when neither eye is being warped.

```
void ApplyMatchedPoses(EyeCapture& L, EyeCapture& R,
    const XrPosef& commonHeadPose,
    const XrView views[2])
{
    // Warp each eye from its own render pose to the pose implied by the
    // common head pose plus that eye's offset.
    for (int e = 0; e < 2; ++e) {
        EyeCapture& c = (e == 0) ? L : R;
        const Mat4 srcVP = c.view * c.proj;
        const Mat4 dstV = Inverse(PoseToMatrix(views[e].pose));
        const Mat4 dstP = ProjectionFromFov(views[e].fov, g_zNear);
```

```

WarpParams p{};
p.srcColor = &c.color; p.srcDepth = &c.depth;
p.srcView = c.view; p.srcProj = c.proj;
p.dstView = dstV; p.dstProj = dstP;
p.phase = 0.0f; p.residualFlow = nullptr;
ExecuteWarp(g_ctx, p, g_submitTarget[e]);
}
}

```

19.12 Interaction with the runtime

Async AER submits at display rate with independently-aged eyes. Because you always submit *something* for both eyes every frame, the runtime never sees a missed frame and never engages its own reprojection — which is what you want, since your per-eye staleness is invisible to it and its reprojection would be based on the wrong assumption.

Submit each eye with its own render pose, though. Even if you are not warping, the runtime can then correct rotational staleness per eye, which recovers a good part of the skew penalty for free.

19.13 When async AER is the right choice

It is a good default for content with irregular frame pacing and rotation-dominant motion. It is a poor choice for content with sustained lateral motion, where any skew is directly visible as disparity shear.

The honest summary: **async AER is classic AER with better hitch behaviour and worse worst-case disparity error.** If you implement the skew management described above, including warp-based correction, it strictly dominates classic AER. If you implement it naively — two rings, newest wins, no skew bound — it is worse.

20 Engine Re-entrancy: Making a Renderer Run Twice

This chapter addresses the problem underlying modes M0 and M1: causing an engine that renders one view to render two. It is the single largest source of failed VR injection projects, and the failure modes are consistent enough to be catalogued.

20.1 The three families

Family A — Replay. Capture the engine’s GPU command stream for the frame, then submit it again with the camera constant buffer patched.

Family B — Re-invoke. Call the engine’s own render entry point a second time, with a different view.

Family C — Native second view. Register an additional view with the engine so its own renderer produces both in one pass.

Each family fails for a distinct structural reason on modern engines. Understanding *why* saves months.

20.2 Why Family A (replay) fails

Command list replay is superficially attractive: in D3D12 and Vulkan, command lists are recorded objects that can, in principle, be submitted multiple times. In practice:

Transient and aliased resources. Modern frame graphs allocate render targets from a transient heap with memory aliasing — the G-buffer’s memory becomes the bloom chain’s memory later in the frame. Replaying the command list re-executes those aliasing barriers against a heap whose contents have moved on. The second execution reads garbage.

One-shot command lists. D3D12 command lists recorded with `D3D12_COMMAND_LIST_FLAG_NONE` are re-submittable only if the application did not record any state that depends on execution order. Bundles are explicitly re-submittable; direct command lists in practice are not, because engines rely on resources being in the state the *previous* command list left them.

Descriptor heap churn. If the engine writes descriptors into a ring buffer during recording and the ring has wrapped by replay time, the replayed list references stale descriptors.

Async compute dependencies. Work submitted on the compute queue and synchronised by fence has already consumed its fence values. Replaying the graphics list without replaying and re-fencing the compute work produces a race or a hang.

Non-idempotent GPU state. Any pass that reads and writes the same buffer — particle simulation, temporal accumulation, GPU culling with persistent state — advances on every execution. Replaying advances it twice per tick.

Upload buffer reuse. The engine’s per-frame upload allocator has already been reset and partially overwritten by the time you replay.

The general statement: **replay assumes the frame is a pure function of its inputs, and modern frame graphs are not.** Every mechanism engines use to reduce memory footprint and CPU cost — aliasing, transient allocation, ring buffers, persistent GPU state — breaks the purity assumption.

There is a narrow case where replay works: a forward renderer with static render targets, no async compute, no temporal accumulation, and per-draw constant buffers that you can shadow. Older engines and some simple ones qualify. Test for it before assuming it is impossible, but do not plan around it.

20.2.1 Diagnosing replay viability

```
// Empirical viability probe: replay the frame and diff the result.
bool ProbeReplayViability(FrameContext& fc) {
    CaptureCommandStream(fc);
    const uint64_t h1 = HashTextureGpu(fc.finalColor);

    ReplayCommandStream(fc, /*patchCamera=*/false); // identical camera
    const uint64_t h2 = HashTextureGpu(fc.finalColor);

    // With an identical camera, a viable replay produces an identical image.
    if (h1 != h2) { LogReplayDivergence(); return false; }

    // Second test: does replay corrupt the NEXT frame?
    // (Detects resource-state and aliasing damage.)
    return !DetectCorruptionNextFrame();
}
```

Run this before writing any camera-patching code. A failed probe saves you from debugging “why does the second eye look wrong” when the answer is “replay was never going to work.”

20.3 Why Family B (re-invoke) fails

Calling the engine’s render function again is architecturally cleaner but runs into engine state assumptions:

Frame-scoped allocators. The render function assumes a fresh linear allocator, which was reset at frame start and is now partially consumed. A second invocation either overflows it or overwrites the first invocation’s data.

Global “current view” state. Engines commonly store the active view in a global or in a manager singleton, with construction and teardown around the frame. Invoking twice either leaves the manager in a torn state or requires you to reconstruct it, which means understanding its full lifetime.

Render graph build state. Frame-graph engines build a graph, compile it, then execute. Re-invoking execution without rebuilding gives you the same graph writing to the same (now-consumed) transient resources. Re-invoking from graph *build* requires the build to be re-entrant, which it typically is not because it registers resources in a per-frame arena with slot identity.

The arena slot collision. This one is worth naming specifically because it is so common. Engines allocate per-frame view data from a fixed-size arena indexed by view slot. A single-view engine has exactly one slot. Constructing a second view by copying the first produces two objects that both believe they own slot 0; the first teardown frees resources the second still references, and you get a use-after-free that manifests as a crash several frames later, in an unrelated system.

Thread affinity. The render function may assert or implicitly depend on running on the render thread at a specific phase. Calling it from a Present hook may be on the right thread but the wrong phase.

20.3.1 When re-invoke does work

Re-invocation succeeds when the engine already has a concept of rendering multiple views per frame — for scene captures, mirrors, planar reflections, security cameras, or picture-in-picture. If such a path exists, it is by construction re-entrant, and hijacking it is the highest-value discovery you can make in a given target.

Search for it: look for render targets that are not the backbuffer and not shadow-map-shaped, being written by a full scene pass. A mirror or a scene-capture render target has the signature of a main view at a different resolution.

```
bool LooksLikeSecondaryScenePass(const BoundState& s) {
    return s.depthTestEnabled
        && s.rtIsColor
        && !s.rtIsBackbuffer
        && s.drawCallCount > g_config.sceneDrawThreshold
        && s.projectionIsPerspective
        && !s.rtIsCubeCompatible;
}
```

If found, the path forward is: drive that capture’s camera to your second eye pose, raise its resolution to match, ensure its render feature set matches the main view (scene captures often run at reduced quality — no screen-space reflections, lower shadow quality, sometimes a different shading mode entirely), and route its output into your second eye swapchain.

The quality-parity check matters. A scene capture rendered in an unlit or simplified mode is useless as a stereo counterpart; verify feature parity before investing in the path.

20.4 Why Family C (native second view) fails

Registering a genuine second view with the engine is the ideal — the engine then shares view-independent work automatically, allocates correct per-view resources, and renders both views in one frame graph execution.

It fails because view registration is typically not a public, reachable operation:

- The view list is populated during a specific initialisation phase from configuration data, not dynamically.
- View objects are allocated from the arena described above, with slot identity baked in.
- Inserting into the engine’s view registry requires the engine’s own hashing and insertion logic, which may be inlined and not separately callable.
- Attach/detach of render proxies is often a one-time setup event, not a per-frame iteration you can piggyback on.

A common and instructive discovery pattern: you find the function that attaches a render proxy for a view, hook it intending to piggyback on a per-frame iteration, and observe that it fires **zero times** across hundreds of frames. It is a one-time setup event. The path then becomes “trigger the setup event once for your own view” rather than “join the per-frame loop” — a very different and usually harder problem.

20.4.1 The spawn approach

One workaround worth knowing: if the game world contains entities that themselves own a render view — security cameras, monitors, scrying pools, mirrors, sniper scopes — then spawning such an entity causes the engine to run its own full view construction path, including whatever GPU backing and registration you could not perform manually.

The technique: spawn the entity, locate its view object, then per frame drive its camera transform to your second eye pose and its resolution to your target, and route its render target into your submission. The engine renders both views per tick through its native fan-out.

This is legitimate and, when available, is the highest-quality path. Its difficulties are finding an entity template that uses the right producer, achieving resolution and quality parity, and dealing with the entity’s world presence (it may be visible, may collide, may be culled when far from the player).

20.5 The deferred renderer wall

There is one more approach that deserves treatment because it works beautifully on one class of engine and catastrophically on another.

Geometry-shader stereo injection — injecting a geometry shader into every scene pipeline state object, duplicating each primitive into two views with a per-eye clip-space offset, either into a texture array or packed side-by-side in a double-width target. The canonical clip-space transform is:

$$x'_{\text{clip}} = x_{\text{clip}} + S(w_{\text{clip}} - C)$$

where S is separation and C is convergence. Applying the offset in clip space, before the perspective divide, produces correct depth-dependent parallax: the shift scales with w (which is view-space depth), so distant geometry shifts more in NDC terms than near geometry, exactly as stereo requires. Objects at depth C land on the screen plane with zero disparity.

For side-by-side packing you then compress each eye into half of NDC:

$$x'' = P \cdot x' \mp \frac{1}{2}w$$

with P a pack scale. **A subtlety that bites everyone:** if the engine aspect-adjusts its horizontal FOV to the render target’s aspect ratio, and you have doubled the target width, the engine has *already* halved the effective horizontal clip extent. Applying an additional $\times 0.5$ pack double-squeezes the image and produces an apparent zoom. Make P tunable and calibrate empirically; for a fully aspect-adjusted engine at 2:1, $P = 1.0$ is correct.

Why this works on forward renderers. In a forward renderer, one shader does geometry and lighting per draw. Duplicating the geometry duplicates everything. Screen-space effects are minimal. The result is genuine stereo.

Why this fails on deferred renderers. Geometry-shader injection controls only the G-buffer fill. Everything afterwards — deferred lighting, shadow resolve, screen-space ambient occlusion, screen-space reflections, volumetrics, fog, temporal antialiasing, upscaling, ray-traced effects — runs as full-buffer screen-space or compute passes with a **single mono camera**. Those passes divide into two groups:

- **Pure per-pixel passes** (read G-buffer at $\mathbf{u}$, write lit result at $\mathbf{u}$) inherit the packing correctly and work.
- **Passes that reconstruct world position from the camera** (volumetrics, screen-space reflections, fog, ray-traced effects) use the mono camera and therefore compute wrong world positions for both halves.
- **Passes that sample neighbouring pixels** (blur, ambient occlusion, temporal antialiasing, upscalers) read across the centre seam and blend one eye’s content into the other.

The result is not “slightly wrong.” It is unfusable — one eye showing structurally different content from the other, with texture chaos near the seam. The failure is structural, not a tuning problem.

To make it work on a deferred renderer you would have to per-eye-ify every screen-space pass: patch each pass’s view constant buffer per half, add seam guard bands, and split the temporal and upscaling passes. That is dozens of pass families, some of which (UI, menus, loading screens) are not geometry at all and render full-buffer regardless. It is a multi-month project with an uncertain outcome, and the ray-traced and upscaler passes are the hardest tail.

Historical note on why this used to work: driver-level stereo solutions on deferred engines did not do geometry duplication. They duplicated *every draw* into two render targets with per-eye constant buffers — effectively full per-pass duplication, which is Family A performed by the driver with full knowledge of resource lifetimes. From outside the engine, that path is the replay path, which is closed for the reasons above.

20.6 The decision procedure

1. Does the engine already render a secondary full-quality scene view (mirror, scene capture, portal)?
 YES -> Family B via that path. Highest value. Verify quality parity.
 NO -> continue.
2. Is the renderer forward, with minimal screen-space effects?
 YES -> geometry-shader injection is viable. Consider it.
 NO -> continue.
3. Does the replay viability probe pass?
 YES -> Family A. Rare on modern engines but cheap to confirm.
 NO -> continue.
4. Can you locate and trigger the engine's view registration?
 YES -> Family C. Highest quality, highest effort.
 NO -> continue.
5. Can you lock the simulation tick reliably?
 YES -> M1 synchronized sequential (Chapter 23). This is the answer for most targets. No re-entrancy required.
 NO -> M4 / M5 / M6: AER with synthesis. Accept the artifacts and invest in the warp pipeline instead.

Step 5 is the important one. **The synchronized sequential approach exists precisely because steps 1–4 usually fail**, and it converts an intractable engine-internals problem into a tractable pacing problem. If you are early in a project and steps 1–4 look difficult, go to step 5 immediately rather than spending months proving they are impossible.

20.7 Cost of true simultaneous stereo

Worth stating so the effort is calibrated correctly. When both views render in one engine tick with full sharing of view-independent work, measured GPU cost across representative deferred content is approximately **188–200%** of mono — so each eye costs about 88–100% of a mono frame, and sharing saves roughly **6–12%** relative to two fully independent renders.

That number is the honest answer to “is native simultaneous stereo worth the engineering?” A 6–12% GPU saving is not, on its own, worth a multi-month engine-internals project. The reasons to pursue Family B or C are correctness and quality — correct per-eye screen-space effects, correct temporal accumulation, correct upscaling — not performance. If you only need same-tick correctness, Chapter 23's approach gets you there for a fraction of the effort. # Alternate Frame Warp

20.8 What it is

Alternate Frame Warp (AFW) is the refinement that makes alternate-eye rendering genuinely usable. The idea is narrow and precise:

On each display frame, one eye has a freshly rendered image and the other does not. Rather than holding the stale eye unchanged, **reproject it from the pose it was rendered with to the pose predicted for the current display time**.

It is per-eye asynchronous timewarp applied specifically to the alternate (stale) eye, integrated into the injected renderer rather than left to the runtime.

Distinguish it clearly from the two neighbouring techniques:

Technique	Source	Destination	Corrects
Runtime timewarp	Last submitted frame	Current pose	Rotation (and translation with depth)
AFW	Stale eye's own last render	Current predicted pose	Head motion since that eye rendered
AER v2 stereo warp (Ch. 9)	<i>Other</i> eye, same instant	This eye's viewpoint	Eye offset
Frame generation (Ch. 10–11)	Two frames of same eye	A future/intermediate instant	Scene and head motion

AFW and the AER v2 stereo warp solve different problems and can be combined. AFW uses genuine data for the correct eye

(no stereo disocclusion) but is temporally stale (scene motion is uncorrected). The stereo warp is temporally fresh but has disocclusion. **Compositing both is strictly better than either**, and that combination is the highest-quality AER configuration short of same-tick rendering.

20.9 Why it works so well

The staleness of the alternate eye decomposes into two components:

$$\text{staleness} = \underbrace{\text{head motion}}_{\text{large, known exactly}} + \underbrace{\text{scene motion}}_{\text{smaller, must be estimated}}$$

Head motion dominates in most content. A head rotating at 200 deg/s over a 22 ms staleness produces 4.4 degrees of angular error — which, at 1.7 arcmin/pixel, is roughly 155 pixels of image displacement. Scene motion over the same interval is typically a few pixels to a few tens of pixels for anything but a fast-moving foreground object.

And head motion is known exactly. You have the tracker. There is no estimation, no flow field, no confidence. It is a closed-form reprojection from the depth buffer and two poses. So AFW removes the overwhelmingly dominant error term at very low cost and with no estimation risk.

This is why AFW should be implemented before any optical-flow work. It is roughly 90% of the achievable quality improvement for roughly 10% of the effort.

20.10 The math

Given the stale eye's render pose $\mathbf{P}_r$ (position $\mathbf{t}_r$, orientation $\mathbf{q}_r$) and the predicted display pose $\mathbf{P}_d$, plus the eye's projection P , construct the reprojection.

View matrix from a pose (row-vector convention, view = inverse of the pose transform):

$$V(\mathbf{P}) = \begin{bmatrix} R(\mathbf{q})^T & 0 \\ -\mathbf{t} R(\mathbf{q})^T & 1 \end{bmatrix}$$

Then the clip-to-clip reprojection is

$$R_{r \rightarrow d} = P^{-1} V(\mathbf{P}_r)^{-1} V(\mathbf{P}_d) P$$

For each pixel with UV $\mathbf{u}$ and depth d :

$$\mathbf{p}_c = (2u_x - 1, 1 - 2u_y, d, 1), \quad \mathbf{p}'_c = \mathbf{p}_c R_{r \rightarrow d}, \quad \mathbf{u}' = \left(\frac{p'_x/p'_w + 1}{2}, \frac{1 - p'_y/p'_w}{2} \right)$$

20.10.1 Decomposing into rotation and translation

It is worth separating the two because they behave completely differently:

Rotation-only reprojection is depth-independent. If $\mathbf{t}_r = \mathbf{t}_d$, the mapping reduces to a homography — every pixel moves according to a single 3×3 transform regardless of depth. No disocclusion is possible; the image simply rotates within a wider frustum. This is why classical timewarp is so robust.

$$H = P^{-1} R(\mathbf{q}_r)^T R(\mathbf{q}_d) P$$

Translation reprojection is depth-dependent and does produce disocclusion. The screen-space displacement from a translation $\Delta \mathbf{t}$ is approximately

$$\Delta \mathbf{u} \approx \frac{1}{z_v} \Pi(\Delta \mathbf{t})$$

— inversely proportional to depth, exactly like stereo disparity. Indeed **translational timewarp is mathematically identical to a stereo warp** with the “IPD” replaced by the translation vector. Everything Chapter 9 says about disocclusion gap widths applies directly, with b replaced by $\|\Delta \mathbf{t}\|$.

Practical consequence: over a 22 ms staleness, head translation is at most a few millimetres to a couple of centimetres. That is comparable to or smaller than the IPD, so disocclusion from AFW's translational component is comparable to or smaller than the stereo warp's — and usually much smaller, since sustained head translation is rare.

Implement both, but structure the code so that if depth is unavailable or untrustworthy you can fall back to rotation-only, which is always safe.

```
// AFW warp, rotation + optional translation.
cbuffer AFWParams : register(b0) {
    float4x4 gRotOnly;           // homography for rotation component
```

```

float4x4 gFullReproj;    // full clip-to-clip including translation
float    gZNear;
float    gTranslationScale; // 0 = rotation only, 1 = full
float2    gInvSize;
};

[numthreads(8, 8, 1)]
void AFWarpVS_Grid(uint3 tid : SV_DispatchThreadID) { /* see grid path below */ }

// Gather-style rotation-only path: exact and hole-free.
[numthreads(8, 8, 1)]
void AFWRotationOnly(uint3 tid : SV_DispatchThreadID) {
    const float2 uv = (float2(tid.xy) + 0.5f) * gInvSize;
    float4 c = float4(uv.x * 2.0f - 1.0f, 1.0f - uv.y * 2.0f, 1.0f, 1.0f);
    // Inverse homography maps destination pixel -> source pixel.
    float4 s = mul(c, gRotOnly);
    const float2 srcUv = float2((s.x / s.w) * 0.5f + 0.5f,
                                0.5f - (s.y / s.w) * 0.5f);
    gOut[tid.xy] = gSrc.SampleLevel(gLinear, srcUv, 0);
}

```

Note the rotation-only path is a **backward** (gather) warp, which is possible precisely because it is depth-independent — you can invert the mapping in closed form. That makes it hole-free, exact, and cheaper than the forward path. Always use the gather path when translation is negligible; a good heuristic is $\|\Delta \mathbf{t}\| < 2$ mm.

For the translational component you need the forward scatter or grid-raster path from Chapter 9, with the same depth ordering and hole handling.

20.11 Pose prediction

AFW warps to the pose at *display time*, which is in the future. You need a prediction. OpenXR gives you one via `xrLocateViews` at `predictedDisplayTime`, and for most purposes that is sufficient and correct — the runtime has the best tracking data.

However, there are cases where you want your own predictor:

- You are warping to a time the runtime was not asked about (an intermediate generated frame).
- You want velocity estimates for scheduling decisions (Chapter 7’s motion-adaptive policy).
- The runtime’s prediction is conservative and you want to extrapolate further.

20.11.1 A practical predictor

Position and orientation want different treatments.

Position: least-squares linear fit over a sliding window. Head position is smooth and low-frequency; a linear fit over ~100 samples (roughly one second at display rate) tracks it well and rejects tracker noise.

For each axis independently, maintain running sums and solve

$$\text{slope} = \frac{\sum xy - n\bar{x}\bar{y}}{\sum x^2 - n\bar{x}^2}, \quad \text{intercept} = \bar{y} - \text{slope} \cdot \bar{x}$$

```

class ScalarPredictor {
    static constexpr uint32_t kRing = 100;
    double m_t[kRing]{}, m_v[kRing]{};
    uint32_t m_w = 0, m_n = 0;
    double m_sx = 0, m_sy = 0, m_sxx = 0, m_sxy = 0;
    double m_slope = 0, m_intercept = 0;
    bool m_valid = false;

public:
    void AddSample(double t, double v) {
        if (m_n >= kRing) { // evict the oldest
            const double ot = m_t[m_w], ov = m_v[m_w];
            m_sx -= ot; m_sy -= ov; m_sxx -= ot*ot; m_sxy -= ot*ov;
        }
        m_t[m_w] = t; m_v[m_w] = v;
        m_sx += t; m_sy += v; m_sxx += t*t; m_sxy += t*v;
        m_w = (m_w + 1) % kRing;
        if (m_n < kRing) ++m_n;
    }
}

```

```

const double n = double(m_n);
const double mx = m_sx / n, my = m_sy / n;
const double sxx = m_sxx - n * mx * mx;
const double sxy = m_sxy - n * mx * my;
if (fabs(sxx) > 1e-12) {
    m_slope = sxy / sxx;
    m_intercept = my - m_slope * mx;
    m_valid = true;
} else {
    m_valid = false;
}
}
bool IsValid() const { return m_n >= kRing && m_valid; }
double Predict(double t) const { return m_intercept + m_slope * t; }
double Slope() const { return m_slope; }
};

```

Note the eviction arithmetic: subtracting the outgoing sample's contributions keeps the update $O(1)$. Recomputing the sums from scratch each frame is also fine at 100 samples, and is more numerically stable over long sessions — floating-point cancellation in the running sums can drift after hours. If you use the incremental form, periodically recompute from the ring to resynchronise.

Orientation: angular-velocity integration. A least-squares fit on quaternion components is wrong (quaternions do not live in a vector space). Instead, estimate angular velocity from the two most recent samples and integrate.

Given consecutive orientations $\mathbf{q}_0$ at t_0 and $\mathbf{q}_1$ at t_1 :

$$\Delta \mathbf{q} = \mathbf{q}_1 \mathbf{q}_0^{-1}, \quad \omega = \frac{2 \log(\Delta \mathbf{q})}{t_1 - t_0}$$

where $\log$ of a unit quaternion $(\mathbf{v}, w)$ is $\frac{\mathbf{v}}{\|\mathbf{v}\|} \arccos(w)$. Then predict:

$$\mathbf{q}(t) = \exp\left(\frac{1}{2}\omega(t - t_1)\right) \mathbf{q}_1$$

```

Quat QuatLog(const Quat& q) {
    const float vlen = sqrtf(q.x*q.x + q.y*q.y + q.z*q.z);
    if (vlen < 1e-8f) return Quat{0, 0, 0, 0};
    const float a = atan2f(vlen, q.w);
    const float s = a / vlen;
    return Quat{q.x * s, q.y * s, q.z * s, 0.0f};
}

Quat QuatExp(const Quat& v) {
    const float a = sqrtf(v.x*v.x + v.y*v.y + v.z*v.z);
    if (a < 1e-8f) return Quat{0, 0, 0, 1};
    const float s = sinf(a) / a;
    return Quat{v.x * s, v.y * s, v.z * s, cosf(a)};
}

Quat PredictOrientation(const Quat& q0, double t0,
                       const Quat& q1, double t1, double t)
{
    const double dt = t1 - t0;
    if (dt < 1e-6) return q1;
    Quat dq = Mul(q1, Conjugate(q0));
    if (dq.w < 0.0f) dq = Negate(dq); // shortest arc
    const Quat logDq = QuatLog(dq);
    const float scale = float((t - t1) / dt);
    return Normalize(Mul(QuatExp(Scale(logDq, scale)), q1));
}

```

Clamp the prediction horizon. Extrapolating more than about 30–40 ms produces overshoot that is worse than the staleness it corrects, because head motion changes direction on roughly that timescale. Clamp both the horizon and the maximum angular velocity used.

```
double ClampHorizon(double now, double target) {
    return std::min(target, now + g_config.maxPredictionSeconds); // e.g. 0.035
}
```

20.12 Combining AFW with the stereo warp

The full alternate-eye pipeline with both corrections:

Fresh eye A (rendered this frame at pose P_r):

```
-> warp from  $P_r$  to  $P_d$  (AFW, usually small since it just rendered)
-> submit as eye A
```

Stale eye B (rendered one frame ago at pose $P_{r'}$):

```
candidate 1: warp eye B from  $P_{r'}$  to  $P_d$            [AFW: correct eye,
                                                    stale content]

candidate 2: stereo-warp eye A from A's view to B's [fresh content,
                                                    disocclusion holes]

-> composite: candidate 2 where valid, candidate 1 in its holes
-> submit as eye B
```

Candidate 2 is preferred where it has data, because it is temporally fresh. Candidate 1 fills the disocclusion regions with genuinely correct geometry — eye B *did* see behind that occluder, one frame ago. This is Chapter 9's dual-source warping, now with the fill source properly pose-corrected.

```
[numthreads(8, 8, 1)]
void CompositeAfwStereo(uint3 tid : SV_DispatchThreadID) {
    const float4 stereo = g_stereoWarped[tid.xy]; // from fresh eye
    const float dS      = g_stereoWarpDepth[tid.xy];
    const float4 afw     = g_afwWarped[tid.xy];   // stale eye, pose-corrected
    const float dA      = g_afwWarpDepth[tid.xy];

    const bool vS = dS > 0.0f && stereo.a > 0.15f; // alpha carries 1-stretch
    const bool vA = dA > 0.0f;

    float4 c;
    if (vS)    c = stereo;
    else if (vA) c = afw;
    else      c = float4(0,0,0,0); // push-pull fills

    // Soften the boundary between the two sources to avoid a visible seam.
    const float edge = g_sourceBoundaryDistance[tid.xy];
    if (vS && vA && edge < g_blendWidth) {
        const float w = saturate(edge / g_blendWidth);
        c = lerp(afw, stereo, w);
    }
    g_out[tid.xy] = c;
}
```

The boundary softening matters more than it looks. A hard switch between two sources with slightly different temporal content produces a visible outline around every occluder — a “halo” that is one of the more recognisable artifacts of naive dual-source warping. Two to four pixels of blend removes it.

20.13 Warping the fresh eye too

Should you warp the freshly rendered eye as well? Yes, and for the same reason as Chapter 23's paired submission: the fresh eye was rendered with a pose that is already one full engine frame old by submission time. Warping it to the predicted display pose removes that latency.

The alternative is to submit each eye with its own render pose and let the runtime correct. That is simpler and works, but it means the two eyes are reprojected independently by the runtime, using its own algorithm, which may not be depth-aware. Doing both warps yourself with matched destination poses gives you a guaranteed-consistent stereo pair at display time.

Recommendation: warp both eyes to a single common predicted head pose (plus their respective eye offsets), and submit that pose. This is the “forced matched eye poses” configuration and it is the correct default whenever you are warping at all.

20.14 Cost

Stage	Cost at 2000x2200/eye
Rotation-only gather warp	0.18–0.25 ms
Full depth-aware forward warp (grid, 2x2)	0.35–0.60 ms
Dual-source composite + boundary blend	0.12–0.18 ms
Push-pull inpaint	0.20–0.35 ms
AFW total, both eyes	~1.0–1.6 ms

Against a staleness correction of 20+ ms of head motion, this is the best cost/benefit ratio available anywhere in the pipeline.

21 Hardware Optical Flow and Vendor Interop

Chapter 10 developed optical flow from scratch in compute shaders. Modern GPUs expose dedicated hardware for this, and using it changes the cost equation substantially — the flow estimation moves off the shader cores entirely and runs on a fixed-function block concurrently with rendering.

21.1 What the hardware provides

The hardware optical flow engines exposed by current GPU vendors share a common shape:

- **Input:** two images, typically 8-bit single-channel or 4:2:0, at a supported resolution.
- **Output:** a **grid-based** flow field, not per-pixel. Grid sizes of 1, 2, 4, or 8 pixels per cell are typical; 4 is a common default. Each cell holds a 2D vector, usually as two 16-bit fixed-point values with 5 fractional bits (i.e. `short2` at 1/32-pixel precision).
- **A cost map:** a per-cell confidence or matching-cost value, usually 8-bit. Lower cost means a better match. This is extremely valuable and is the single most useful thing the hardware gives you beyond the flow itself.
- **Optional hints:** a previous flow field can be supplied to seed the search, which both improves quality and reduces latency for temporally coherent content.

The engine is a fixed-function block. It does not consume shader cores, so its work genuinely overlaps rendering. Its latency, however, is not zero, and the API is typically synchronous at some level — which is the main integration hazard.

21.2 Integration architecture

The flow engine is generally exposed through a compute API (CUDA or equivalent) rather than through the graphics API, which means you need **cross-API interop**: share your D3D12 or Vulkan textures with the compute context, synchronise across the API boundary, and get the results back.

D3D12 render target (BGRA8)

- > CreateSharedHandle
- > import as external memory in compute API
- > map to a compute-side array / texture object
- > hardware flow engine consumes it
- > flow buffer (device pointer)
- > warp kernel consumes flow + colors + depth
- > writes into a shared output texture
- > D3D12 copies that into the swapchain image

Two synchronisation objects are needed: a shared fence/semaphore so the compute side waits for the graphics side's writes, and the reverse for the results.

```
// D3D12 side: create a shared fence usable by the compute API.
ComPtr<ID3D12Fence> fence;
device->CreateFence(0, D3D12_FENCE_FLAG_SHARED, IID_PPV_ARGS(&fence));
HANDLE fenceHandle = nullptr;
device->CreateSharedHandle(fence.Get(), nullptr, GENERIC_ALL,
    nullptr, &fenceHandle);
// Import fenceHandle as an external semaphore on the compute side.

// Per frame:
graphicsQueue->Signal(fence.Get(), ++fenceValue);
// compute side: waitExternalSemaphore(sem, fenceValue) on its stream
// ... run flow + warp on the compute stream ...
// compute side: signalExternalSemaphore(sem, ++fenceValue)
graphicsQueue->Wait(fence.Get(), fenceValue);
// ... graphics copies the result ...
```

21.2.1 The import-cost trap

This is the single most important practical lesson in vendor interop, and it is non-obvious enough to be worth stating loudly.

Importing a texture into the compute API is expensive — creating a shared handle, importing the external memory, mapping it to an array, and creating texture and surface objects costs on the order of a few milliseconds *per texture*. With five or six textures per eye (previous colour, current colour, previous depth, current depth, motion vectors, output) across two eyes, doing this every frame costs on the order of **25 ms per frame**. That alone caps you at under 40 fps and will look like a mysterious performance wall.

The fix: import once per resource, cache by pointer, never release until shutdown.

```
class ImportCache {
    std::unordered_map<ID3D12Resource*, ExternalImage> m_map;
public:
    bool GetOrImport(ID3D12Resource* res, bool writable, ExternalImage* out) {
        auto it = m_map.find(res);
        if (it != m_map.end()) { *out = it->second; return true; }

        ExternalImage img{};
        if (!ImportD3D12Texture(res, writable, &img)) return false;
        m_map.emplace(res, img);
        *out = img;
        return true;
    }
    void Clear() { // on resize / device loss only
        for (auto& [res, img] : m_map) DestroyExternalImage(img);
        m_map.clear();
    }
};
```

This works only if your textures are stable across frames — which is why you should ring-buffer a **fixed set** of capture and output textures rather than allocating per frame. Entries live until shutdown or resolution change, so the compute side can always reference them and there is no per-frame release and no use-after-free risk.

21.2.2 Blocking versus non-blocking

The flow engine's execute call typically synchronises on its output stream, meaning it blocks the calling thread until the flow is ready. If you call it from the engine's present thread, you inject that latency directly into the game's frame loop, producing jitter that is often worse than the artifact you were fixing.

Two mitigations, both necessary:

1. Do not block on the warp. Queue the warp kernel and signal the shared fence, then return immediately. GPU ordering guarantees the consumer's copy happens after the warp, because both are ordered on the same queue after the cross-API wait. No CPU wait is required.

```
bool ProcessFrameNonBlocking(/* ... */) {
    // graphics queue signals; compute stream waits
    m_graphicsQueue->Signal(m_fence.Get(), ++m_fenceValue);
    ComputeWaitSemaphore(m_sem, m_fenceValue, m_stream);

    LaunchWarpKernel(args, m_stream); // async

    ComputeSignalSemaphore(m_sem, ++m_fenceValue, m_stream);
    m_graphicsQueue->Wait(m_fence.Get(), m_fenceValue);
    return true; // no CPU wait anywhere
}
```

2. Run flow estimation on a separate high-priority stream per eye, so the two eyes' flow computations do not serialise behind each other and so flow does not block the warp of the *other* eye.

21.2.3 Flow caching: compute once per capture, warp every frame

This is the structural optimisation that makes hardware flow viable at high refresh rates.

Observe: the flow field between a given pair of frames (I_{t-1} , I_t) is **fixed**. If you are generating multiple display frames between two engine renders, or if an eye has not received a new capture this frame, recomputing the flow gives you the identical answer at full cost.

So: **key the flow cache on the (previous, current) source texture pair**. Recompute only when an eye receives a genuinely new capture. Warp every display frame, at a new blend phase, reusing the cached flow.


```

float2 v10 = LoadFlowCell(clamp(c0 + int2(1,0), int2(0,0), maxCell),
                          strideBytes, flowRaw);
float2 v01 = LoadFlowCell(clamp(c0 + int2(0,1), int2(0,0), maxCell),
                          strideBytes, flowRaw);
float2 v11 = LoadFlowCell(clamp(c0 + int2(1,1), int2(0,0), maxCell),
                          strideBytes, flowRaw);

return lerp(lerp(v00, v10, f.x), lerp(v01, v11, f.x), f.y);
}

```

Do not interpolate across depth discontinuities. Use the depth buffer to gate the interpolation, as in Chapter 10's guided smoothing. Naive bilinear interpolation of flow across an object boundary produces a band of intermediate motion that smears the silhouette — trading the cell-boundary shimmer for a softer but broader artifact.

21.3.2 Cost map as confidence

Convert the cost map to a confidence weight and use it everywhere Chapter 10 uses `g_confidence`:

```

float ConfidenceFromCost(uint cost) {
    // Lower cost = better match. Typical useful range 0..64.
    return saturate(1.0f - float(cost) * (1.0f / 64.0f));
}

```

Regions with high cost — low texture, occlusion boundaries, illumination changes — get down-weighted in the blend, which is precisely where you want to fall back to pose-derived reprojection or to hole filling.

21.3.3 Spatial smoothing with outlier rejection

Per-frame optical flow is noisy on textured and animated surfaces, and that noise shimmers visibly in the synthesised eye. A 3x3 low-pass over the flow grid reduces it substantially, at the cost of slightly trailing motion boundaries.

The refinement that matters: **drop the most deviant neighbour before averaging**. A plain mean lets a single spurious cell — and flow engines produce them regularly at occlusion boundaries — drag the whole neighbourhood. Excluding the neighbour with the largest squared deviation from the centre value gives you most of the robustness of a median at a fraction of the cost, since you are already loading all eight neighbours.

```

float2 SmoothFlowRobust(int2 pixel, uint gridSize, float strength, uint2 outSize)
{
    const float2 centre = SampleFlowBilinear(pixel, gridSize, ...);
    if (strength <= 0.001f) return centre;

    float2 sum = centre;
    int count = 1;
    float worstDev2 = 0.0f;
    float2 worst = centre;

    const int g = int(gridSize);
    [unroll] for (int dy = -g; dy <= g; dy += g) {
        [unroll] for (int dx = -g; dx <= g; dx += g) {
            if (dx == 0 && dy == 0) continue;
            const int2 q = pixel + int2(dx, dy);
            if (any(q < 0) || any(q >= int2(outSize))) continue;

            const float2 nf = SampleFlowBilinear(q, gridSize, ...);
            sum += nf; ++count;

            const float2 d = nf - centre;
            const float dev2 = dot(d, d);
            if (dev2 > worstDev2) { worstDev2 = dev2; worst = nf; }
        }
    }

    // Reject the single most deviant neighbour.
    if (count > 2) { sum -= worst; --count; }

    const float2 avg = sum / float(count);
    return lerp(centre, avg, saturate(strength));
}

```

```
}

```

Useful strength range is 0 to about 0.6; beyond that the trailing becomes visible on fast motion. Expose it as a slider labelled for what it does perceptually (“stale-eye shimmer reduction”) rather than for what it is.

A depth-gated variant, which preserves motion boundaries better at slightly higher cost:

```
float2 SmoothFlow3x3(int2 cell, float strength) {
    if (strength <= 0.001f) return LoadFlow(cell);
    const float dc = LoadCellDepth(cell);
    float2 sum = 0; float wsum = 0;
    [unroll] for (int y = -1; y <= 1; ++y)
    [unroll] for (int x = -1; x <= 1; ++x) {
        const int2 c = cell + int2(x, y);
        const float dq = LoadCellDepth(c);
        const float w = exp(-abs(dq - dc) * g_depthSigmaInv);
        sum += LoadFlow(c) * w; wsum += w;
    }
    const float2 smoothed = (wsum > 1e-4f) ? sum / wsum : LoadFlow(cell);
    return lerp(LoadFlow(cell), smoothed, saturate(strength));
}
```

21.4 Vendor-specific practical hazards

Three issues that cost significant debugging time and are not documented anywhere obvious.

Texture objects may silently return zeros for imported external memory. On some driver and runtime combinations, creating a texture object over memory imported from a graphics API and sampling it through the normal texture path yields all zeros — no error, no warning, just black. The workaround is to use the **surface** read path instead (`surf2Dread` or the equivalent) and implement bilinear filtering manually.

```
// Manual bilinear over a surface-read path, RGBA8 source.
float4 SurfaceBilinearRGBA8(SurfaceHandle s, float x, float y, int w, int h) {
    const int x0 = clamp(int(floor(x)), 0, w - 1);
    const int y0 = clamp(int(floor(y)), 0, h - 1);
    const int x1 = min(x0 + 1, w - 1);
    const int y1 = min(y0 + 1, h - 1);
    const float fx = saturate(x - float(x0));
    const float fy = saturate(y - float(y0));

    uchar4 c00, c10, c01, c11;
    SurfRead(&c00, s, x0 * 4, y0); // byte offset on x, row index on y
    SurfRead(&c10, s, x1 * 4, y0);
    SurfRead(&c01, s, x0 * 4, y1);
    SurfRead(&c11, s, x1 * 4, y1);

    const float inv = 1.0f / 255.0f;
    float4 a = lerp(ToFloat4(c00) * inv, ToFloat4(c10) * inv, fx);
    float4 b = lerp(ToFloat4(c01) * inv, ToFloat4(c11) * inv, fx);
    return lerp(a, b, fy);
}
```

Note the x coordinate is a **byte** offset in the surface API while y is a row index. Passing a texel index for x produces a horizontally compressed read that looks like a corrupted texture rather than an obvious failure.

Sample at $uv * \text{size} - 0.5$ to align with texel centres. Omitting the half-texel offset shifts the whole warp by half a pixel, which is not obviously wrong on its own but compounds with everything else and shows up as a persistent softness.

External memory mappings expose mip 0 only. Do not assume a full mip chain is available on an imported resource; if you need lower resolutions, generate them on the graphics side before import, or downsample explicitly in the compute kernel.

Performance-level selection is a real tradeoff, not a free quality dial. Flow engines typically expose fast/medium/slow presets that trade density and accuracy against latency. The slow preset produces noticeably denser and more accurate flow, but at high per-eye resolutions it will not fit an 11 ms budget. The correct resolution is to run the *slow* preset on **downscaled input** rather than the fast preset at full resolution — flow is smooth almost everywhere (Chapter 10), so input downscaling costs far less quality than preset reduction does. Implement the downscale path before exposing the preset as a user setting, or users will select the slow preset and conclude the feature is broken.

Do not share a flow session between eyes. Beyond the cache-thrashing point made above, flow engines maintain internal temporal state — previous-frame hints used to seed the search. Feeding alternating left and right eye images into one session leaks each eye’s hints into the other’s search, which degrades both. The two eyes are similar enough that the damage is subtle

rather than obvious, which makes it worse: you get quietly degraded flow and no error. Allocate one session per eye.

21.5 Graceful fallback

Hardware flow is not universally available: the vendor SDK may be absent, the driver may be too old, the GPU may not have the block, or the resolution may be unsupported. Structure the pipeline so the flow *source* is an interface with three implementations:

1. Hardware flow engine (best quality/cost).
2. Compute-shader block matching from Chapter 10 (portable).
3. Camera-derived reprojection only, no residual flow (always available).

Level 3 is not a degenerate fallback — for static scenes it is exactly correct, and it is what AFW uses. Make it a first-class mode, not an error path.

```
class IFlowSource {
public:
    virtual ~IFlowSource() = default;
    virtual bool IsAvailable() const = 0;
    virtual bool Compute(int eye, const FrameRef& prev, const FrameRef& curr,
                        FlowResult* out) = 0;
    virtual const char* Name() const = 0;
};

std::unique_ptr<IFlowSource> SelectFlowSource() {
    for (auto& factory : {MakeHardwareFlow, MakeComputeFlow, MakeNullFlow}) {
        auto s = factory();
        if (s && s->IsAvailable()) {
            Log("Flow source: %s", s->Name());
            return s;
        }
    }
    return MakeNullFlow();
}
```

22 Engine Motion Vectors and Upscaler Integration

22.1 Why engine motion vectors are worth the trouble

Hardware and estimated optical flow both infer motion from images. Engine motion vectors *know* it. They carry the exact screen-space displacement of every pixel, including:

- Skinned character animation, which optical flow handles poorly because the surface deforms.
- Fast small objects, where flow's search range fails.
- Motion in low-texture regions, where flow has no signal at all.
- Motion of transparent and alpha-tested surfaces (sometimes).

Any modern title using temporal antialiasing or a temporal upscaler produces them. Finding and using them is the highest-quality motion source available.

22.2 Finding them via the upscaler

Titles integrating a temporal upscaler pass their inputs through a well-defined interface — typically a tagging API where the application declares “this resource is the depth buffer, this is the motion vector buffer, this is the colour input.” That tagging call is the ideal interception point, because the resource identity is declared explicitly rather than inferred.

```
// Generic shape of an upscaler resource-tagging hook.
Result Hooked_SetTag(const ResourceTag* tags, uint32_t count,
                    CommandBuffer cmd, const FrameToken* frame)
{
    for (uint32_t i = 0; i < count; ++i) {
        switch (tags[i].type) {
            case kTagMotionVectors:
                g_engineMv.resource = tags[i].resource;
                g_engineMv.extent   = tags[i].extent;
                g_engineMv.format    = tags[i].format;
                g_engineMv.valid     = true;
                break;
            case kTagDepth:
                g_engineDepth.resource = tags[i].resource;
```

```

        g_engineDepth.valid = true;
        break;
    case kTagColorInput:
        g_engineColor.resource = tags[i].resource;
        break;
    default: break;
}
}
return g_realSetTag(tags, count, cmd, frame);
}

```

Also intercept the upscaler’s *evaluate* call to learn the scale factors and jitter offsets it was given — you need both, as explained below.

If no upscaler is present, fall back to the render-target signature heuristics from Chapter 10.

22.3 The three conventions you must resolve

Engine motion vectors are stored in one of several conventions, and getting any of them wrong produces motion in the wrong direction or at the wrong magnitude — which looks like severe ghosting and is easy to misdiagnose as a flow quality problem.

- 1. Units.** Common encodings: normalised UV displacement, NDC displacement (which is 2x UV), pixels at render resolution, or pixels at display resolution. The upscaler interface usually reports a scale factor to convert to its expected space; capture it.
- 2. Direction.** Motion vectors may encode “where this pixel was in the previous frame” (backward, the TAA convention) or “where this pixel will be next frame” (forward). TAA needs backward vectors, so backward is far more common. A warp that extrapolates forward needs to negate them.
- 3. Y orientation.** Depending on whether the engine works in UV space (Y down) or NDC (Y up), the vertical component’s sign differs.

22.3.1 Automatic calibration

Do not guess. Calibrate at runtime with a controlled probe.

The procedure: on a frame where the camera translates by a known amount and the scene is (mostly) static, the motion vector at a pixel should equal the camera-derived reprojection displacement, which you can compute exactly from depth and the two view-projection matrices. Solve for scale and sign by least squares over a sparse sample of high-confidence pixels.

```

struct MvCalibration { float scaleX, scaleY; bool valid; };

MvCalibration CalibrateMotionVectors(const FrameRef& prev, const FrameRef& curr)
{
    // Sample a sparse grid, skipping pixels near depth discontinuities.
    std::vector<float2> engineMv, cameraMv;
    for (int y = 8; y < H - 8; y += 32)
        for (int x = 8; x < W - 8; x += 32) {
            if (DepthEdgeNearby(curr.depth, x, y)) continue;
            const float d = SampleDepth(curr.depth, x, y);
            if (d <= 0.0f) continue;
            cameraMv.push_back(CameraReprojectDelta(x, y, d, prev.viewProj,
                                                    curr.viewProj));
            engineMv.push_back(SampleMv(curr.mv, x, y));
        }
    if (engineMv.size() < 64) return {1,1,false};

    // Robust per-axis scale: median of ratios, which tolerates the
    // moving-object outliers that ruin a plain least-squares fit.
    std::vector<float> rx, ry;
    for (size_t i = 0; i < engineMv.size(); ++i) {
        if (fabsf(engineMv[i].x) > 1e-4f) rx.push_back(cameraMv[i].x / engineMv[i].x);
        if (fabsf(engineMv[i].y) > 1e-4f) ry.push_back(cameraMv[i].y / engineMv[i].y);
    }
    if (rx.size() < 32 || ry.size() < 32) return {1,1,false};
    return { Median(rx), Median(ry), true };
}

```

Use the **median of ratios**, not a mean or a least-squares fit. Moving objects in the scene are legitimate outliers whose engine motion vectors correctly differ from the camera-derived value; a median ignores them, a mean does not.

Run the calibration over several frames with differing camera motion and require consistent results before accepting. Cache per

application.

22.4 Jitter

Temporal upscalers jitter the projection matrix each frame by a sub-pixel offset from a low-discrepancy sequence. This means:

- The rendered image is offset by up to half a pixel in a per-frame-varying way.
- Motion vectors may or may not include the jitter delta, depending on the engine's convention.

If jitter is not accounted for, your warp inherits a sub-pixel oscillation that reads as a fine shimmer. Capture the jitter offsets from the upscaler's evaluate call and subtract the jitter delta:

$$\mathbf{F}_{\text{corrected}} = \mathbf{F}_{\text{engine}} - (\mathbf{j}_t - \mathbf{j}_{t-1})$$

expressed in whatever units your flow is in. If you cannot determine whether the engine's vectors already include jitter, test empirically: with a completely static camera and scene, correctly de-jittered motion vectors are exactly zero. Any residual oscillation at the jitter sequence's period tells you the correction is wrong or doubled.

22.5 Combining engine vectors with estimated flow

Neither source is complete. Engine vectors are exact where they exist but are frequently absent or wrong for: particles, transparent surfaces, decals, non-rasterised effects, and anything drawn after the motion vector pass. Estimated flow covers everything visible but is noisy and fails in low-texture regions.

Combine by confidence:

```
float2 CombinedFlow(int2 p, float2 cameraFlow) {
    const float2 engineF = SampleEngineMv(p) * g_mvScale;
    const bool   engineValid = HasEngineMv(p);
    const float2 hwF      = SampleGridFlow(p);
    const float  hwConf    = ConfidenceFromCost(SampleCost(p));

    // Residual relative to camera-derived motion.
    const float2 engineResidual = engineF - cameraFlow;
    const float2 hwResidual     = hwF      - cameraFlow;

    if (engineValid) {
        // Engine vectors dominate; use hardware flow only where the engine's
        // residual is implausibly large (usually means the pixel is a
        // particle or transparency the MV pass missed).
        const float disagreement = length(engineResidual - hwResidual);
        const float w = saturate(1.0f - disagreement * g_disagreeScale);
        return cameraFlow + lerp(hwResidual, engineResidual,
                                lerp(0.5f, 1.0f, w));
    }
    return cameraFlow + hwResidual * hwConf;
}
```

The residual formulation matters here as much as in Chapter 10: comparing two sources' *residuals* relative to a common analytic baseline is far more meaningful than comparing raw vectors, because it removes the large shared camera component that would otherwise dominate the disagreement metric.

22.6 Resolution mismatch

Engine motion vectors are at *render* resolution, which under an upscaler is lower than display resolution. Your warp operates at display resolution. Sample the motion vector texture with a bilinear filter in normalised coordinates and scale the result by the resolution ratio — but again, gate the bilinear filtering across depth edges, or you will smear motion boundaries by the upscale factor.

An alternative worth considering: **run the entire warp at render resolution and upscale afterwards**. This is cheaper and avoids the mismatch entirely, at the cost of softening the generated frame relative to the rendered one — which produces a sharpness alternation between real and generated frames that some viewers find more objectionable than the cost saving is worth. Measure both. # Comparative Analysis of All Modes

This chapter puts every mode on a common footing so the choice can be made quantitatively rather than by intuition.

22.7 The cost model

Extend Chapter 2's decomposition. Per engine frame:

- C_i — view-independent GPU work (shadows, skinning, particle simulation, probes).
- C_v — view-dependent GPU work (geometry, lighting, screen-space, post).

- C_f — fixed CPU and submission overhead.
- C_s — your synthesis cost per display frame.

Define $\rho = C_i / (C_i + C_v)$, the view-independent fraction. Typical values: 0.15–0.25 for a modern deferred renderer with cached shadows; 0.30–0.45 for a renderer with expensive per-frame shadow rendering and heavy GPU simulation.

22.7.1 Per-mode engine cost per display frame

Let E be the engine cost of one mono frame, $E = C_i + C_v + C_f$.

Mode	Engine GPU per display frame	Notes
M0 simultaneous	$C_i + 2C_v + C'_f$	$C'_f \approx 1.6C_f$
M1 synced sequential	$2(C_i + C_v + C_f)$	halved by shadow sharing: $\rightarrow (2 - \rho)(C_i + C_v) + 2C_f$
M2 classic AER	$C_i + C_v + C_f$	one view
M3 async AER	$C_i + C_v + C_f$	one view
M4 AER + stereo warp	$C_i + C_v + C'_f + C_s$	
M5 alternate frame warp	$C_i + C_v + C_f + C_s$	C_s smaller than M4
M6 AER + generation	$\frac{1}{2}(C_i + C_v + C_f) + C_s$	engine at half display rate
M1+M6 combined	$(C_i + C_v + C_f) + C_s$	engine at display rate, pairs at half
M7 half-rate paired	$\frac{1}{2}(C_i + 2C_v + C'_f)$	

The row worth staring at is **M1+M6**: same-tick sequential stereo with one generated frame per pair. Its engine cost per display frame equals one mono frame — the same as classic AER — while delivering geometrically correct same-tick stereo. The catch is that the pair rate is half the display rate, so half of what you see is synthesised. That is the fundamental trade of this configuration, and for most content it is a better trade than AER's inter-ocular inconsistency.

22.7.2 Worked example

Take a representative deferred title: $C_i = 3.0$ ms, $C_v = 8.0$ ms, $C_f = 1.5$ ms, so mono $E = 12.5$ ms (80 fps mono), $\rho = 0.27$. Target 90 Hz display (11.1 ms budget). Synthesis $C_s = 1.6$ ms for warp-only modes, 2.6 ms with flow-based generation.

Mode	Cost/display frame	Achieved	Verdict
M0	$3.0 + 16.0 + 2.4 = 21.4$ ms	47 Hz	Misses badly
M1	$(2 - 0.27)(11.0) + 3.0 = 22.0$ ms	45 Hz pairs	Misses
M2/M3	12.5 ms	80 Hz	Just misses
M4	$12.5 + 1.6 = 14.1$ ms	71 Hz	Misses
M5	$12.5 + 1.2 = 13.7$ ms	73 Hz	Misses
M6	$6.25 + 2.6 = 8.9$ ms	90 Hz	Meets
M1+M6	$11.0 + 2.6 = 13.6$ ms	73 Hz	Misses
M7	$10.7 + 0.3 = 11.0$ ms	90 Hz	Meets, whole-frame judder

For this hardware/content combination, M6 is the only configuration that reaches 90 Hz, and the honest conclusion is that the content is too heavy for same-tick stereo on this GPU. Reduce render resolution by 20% ($C_v \rightarrow 5.1$ ms) and re-run:

Mode	Cost	Achieved
M1	$(1.73)(8.1) + 3.0 = 17.0$ ms	59 Hz pairs
M1+M6	$8.1 + 2.6 = 10.7$ ms	90 Hz, half generated
M4	$9.6 + 1.6 = 11.2$ ms	89 Hz

Now M1+M6 fits, and it is clearly the better choice over M4: correct stereo with generated intermediate frames beats incorrect stereo at full render rate.

This is the general shape of the answer. Resolution is the lever that moves same-tick stereo from impossible to comfortable, and once it fits, it wins.

22.8 Quality model

Cost is only half the decision. Quantify the artifacts.

22.8.1 Disparity error

From Chapter 6, with σ the inter-ocular temporal skew and v the lateral head velocity:

$$\frac{\epsilon}{\theta} = \frac{v \sigma}{b}$$

Mode	σ	Error at $v = 1$ m/s, $\tau = 11$ ms
M0, M1, M7	0	0%
M2, M3	τ	17%
M4 (stereo warp)	0	0% (replaced by disocclusion)
M5 (AFW)	τ , but head component removed	~2–4% residual
M6 (on M2 base)	τ	17% plus generation error
M1+M6	0	0% plus generation error

M4 and M1/M1+M6 are the only configurations with structurally zero disparity error. M4 achieves it by synthesis (trading in disocclusion); M1 achieves it by construction (trading in throughput).

22.8.2 Disocclusion area

For M4, the fraction of the synthesised eye that must be filled. From Chapter 9's gap-width relation, integrated over a scene's depth-edge distribution. A useful approximation:

$$A_{\text{disocc}} \approx \frac{b P_{00}}{2} \cdot \frac{L_{\text{edge}}}{W} \cdot \left\langle \left| \frac{1}{z_1} - \frac{1}{z_2} \right| \right\rangle$$

where L_{edge} is the total length of depth discontinuities in pixels and the bracket denotes the average inverse-depth jump across them. In practice measure it directly (Chapter 17) rather than estimating; typical values are 0.5–2% for outdoor scenes and 3–8% for interiors with strong near-field occluders.

For M5, replace b with $\|\Delta \mathbf{t}\|$, the head translation over one frame — typically 0.5–5 mm, i.e. an order of magnitude smaller than IPD. **AFW's disocclusion is roughly 10–30x smaller than the stereo warp's.** This is the quantitative reason AFW should be implemented first.

22.8.3 Temporal error

For generated frames, the extrapolation distance $\phi \Delta$ multiplied by unmodelled acceleration:

$$\text{error} \approx \frac{1}{2} \|\mathbf{a}\| (\phi \Delta)^2$$

At $\Delta = 22$ ms, $\phi = 0.5$, and a screen-space acceleration of 2000 px/s² (a fast-moving object changing direction), the error is $\frac{1}{2}(2000)(0.011)^2 = 0.12$ px. Negligible. At $\phi = 1.5$ (deep extrapolation during a hitch) and 10,000 px/s², it is 5.4 px — visible. This is why the phase clamp in Chapter 10 matters.

22.8.4 A composite quality score

For automated regression testing, combine:

$$Q = w_1 \left(1 - \frac{\epsilon/\theta}{0.20} \right) + w_2 \left(1 - \frac{A_{\text{disocc}}}{0.05} \right) + w_3 \frac{Q_{\text{stereo}} - 0.90}{0.10} + w_4 \left(1 - \frac{T_{mp} - T_{mp}^{\text{ref}}}{0.020} \right)$$

each term clamped to $[0, 1]$, with weights summing to 1. Reasonable weights from subjective correlation: $w_1 = 0.35$, $w_2 = 0.20$, $w_3 = 0.25$, $w_4 = 0.20$. The disparity term dominates because disparity error correlates most strongly with reported discomfort.

22.9 Decision table

Content / constraint	Recommended	Second choice
Mono headroom > 2.2x display rate	M1	M0 if re-entrancy solved
Mono headroom 1.6–2.2x	M1 + M6	M4 + M5
Mono headroom 1.1–1.6x	M4 + M5 + M6	M6
Mono headroom < 1.1x	M6 with reduced resolution	M7
CPU-bound	M4 / M5 (M1 doubles CPU cost)	M6
Fast lateral motion content	M1 family	M4
Rotation-dominant content	M5	M4
Cannot lock tick safely	M4 + M5 + M6	M2
Cannot capture depth	M5 rotation-only	M2

22.10 Summary of what each technique buys

- **Tick locking** buys correctness (zero disparity error). Costs throughput.
- **Stereo warp** buys correctness. Costs disocclusion artifacts.
- **AFW** buys latency and removes the dominant staleness term. Costs very little. **Always on.**
- **Frame generation** buys throughput. Costs temporal artifacts and, if interpolating, latency.
- **Hardware flow** buys generation quality at low shader cost. Costs integration complexity and vendor dependence.
- **Engine motion vectors** buy generation quality for animated content. Costs discovery and calibration effort.

23 The Mode-Switching Runtime

A production implementation does not pick one mode. It runs a state machine.

23.1 State and transitions

```
enum class StereoMode {
    Mono,                // fallback: same image both eyes
    SyncSequential,      // M1
    SyncSequentialGen,   // M1 + M6
    AlternateEye,        // M2
    AlternateEyeAsync,   // M3
    AlternateEyeWarp,    // M4 + M5
    AlternateEyeWarpGen, // M4 + M5 + M6
    HalfRatePaired,     // M7
};

struct ModePolicy {
    StereoMode mode;
    float minHeadroom;    // fraction of display interval free
    float maxLateralSpeed; // m/s above which this mode is unsuitable
    bool requiresTickLock;
    bool requiresDepth;
    bool requiresFlow;
};

static constexpr ModePolicy kPolicies[] = {
    { StereoMode::SyncSequential,    0.55f, 99.0f, true, false, false },
    { StereoMode::SyncSequentialGen, 0.25f, 99.0f, true, true, true },
    { StereoMode::AlternateEyeWarpGen, 0.12f, 1.5f, false, true, true },
    { StereoMode::AlternateEyeWarp,   0.20f, 1.2f, false, true, false },
    { StereoMode::AlternateEyeAsync,  0.15f, 0.6f, false, false, false },
    { StereoMode::HalfRatePaired,     0.05f, 99.0f, false, false, false },
    { StereoMode::Mono,               0.00f, 99.0f, false, false, false },
};
```

Selection walks the list in preference order and picks the first policy whose requirements are satisfied by current capabilities and current measured conditions.

```
StereoMode SelectMode(const Capabilities& cap, const PerfStats& perf,
                    const HeadState& head)
{
    const float headroom = 1.0f - perf.p99FrameMs / perf.displayIntervalMs;
    const float lateral  = LateralSpeed(head);
```

```

for (const ModePolicy& p : kPolicies) {
    if (p.requiresTickLock && !cap.tickLockVerified) continue;
    if (p.requiresDepth && !cap.depthCaptureOk) continue;
    if (p.requiresFlow && !cap.flowAvailable) continue;
    if (headroom < p.minHeadroom) continue;
    if (lateral > p.maxLateralSpeed) continue;
    if (!g_config.modeEnabled[(int)p.mode]) continue;
    return p.mode;
}
return StereoMode::Mono;
}

```

23.2 Transition safety

Mode changes are visible. Some are more visible than others, and some are unsafe to perform at arbitrary times.

Safe at any frame boundary: M4 ↔ M5, enabling/disabling generation, changing flow source.

Requires an even frame boundary: anything entering or leaving M1, because M1 operates on frame *pairs*. Switching mid-pair leaves the tick lock engaged with no matching second eye. Track pair phase and defer the transition.

Requires a quiescent frame: changing render resolution, reallocating swapchains, or re-initialising the flow engine. Perform during a loading screen or a menu if you can detect one; otherwise accept a single dropped frame and submit the previous image.

```

bool CanTransitionNow(StereoMode from, StereoMode to, const FrameContext& fc) {
    const bool fromPaired = IsPairedMode(from);
    const bool toPaired = IsPairedMode(to);
    if ((fromPaired || toPaired) && fc.pairPhase != PairPhase::Complete)
        return false;
    if (RequiresRealloc(from, to) && !fc.quiescent)
        return false;
    return true;
}

```

Rate limiting. A mode change every frame is far worse than sitting in the wrong mode. Require a minimum dwell time (30–60 display frames), use asymmetric thresholds (descend fast, ascend slow), and add a cooldown after any change.

Crossfade where possible. For transitions that change the character of the image — entering or leaving generation, switching between warped and rendered second eye — a 4–8 frame crossfade in the composite hides the discontinuity. Do not crossfade across a tick-lock boundary; the two sources represent different world states and crossfading them produces a visible double image.

23.3 Capability probing at startup

Determine what is available before the first frame is submitted, and re-probe on resolution or device change.

```

Capabilities ProbeCapabilities() {
    Capabilities c{};

    c.cameraLockOk = RunCameraIdentification(kProbeFrames);
    c.depthCaptureOk = ProbeDepthCapture();
    c.depthIsReversed = ProbeDepthConvention();
    c.mvBufferFound = ProbeEngineMotionVectors();
    c.mvCalibration = c.mvBufferFound ? CalibrateMotionVectors(...)
                                     : MvCalibration{};
    c.flowAvailable = SelectFlowSource()->IsAvailable();
    c.replayViable = ProbeReplayViability();
    c.secondaryViewFound = ProbeSecondaryScenePass();
    c.tickLockVerified = VerifyTickLock(...);
    c.asyncComputeOk = ProbeAsyncCompute();
    c.depthLayerSupported = RuntimeSupportsDepthLayer();

    LogCapabilities(c);
    return c;
}

```

Log the results prominently. When a user reports a problem, the capability block is the first thing you want to see, because most reported issues resolve to “the depth capture failed on this configuration and the implementation silently fell back.”

Never silently fall back. Surface the active mode and the reason for it in the in-headset overlay.

24 Bring-Up Order

The order in which you build this determines whether the project succeeds. Each stage should be independently verifiable before the next begins.

24.1 Stage 1 — Mono in the headset

Hook the graphics API, capture the final backbuffer, submit it to OpenXR as a head-locked quad or as a projection layer with the application's own FOV. No camera manipulation.

Verification: the game is visible in the headset, at full rate, with correct colours and no flicker. Head motion does nothing.

This stage validates: hooking, frame boundary detection, swapchain management, format conversion, gamma handling, and OpenXR submission. Do not proceed until it is rock solid, because every later bug will otherwise be ambiguous.

24.2 Stage 2 — Camera control

Find and rewrite the view matrix. Drive it from the head pose. Still mono — both eyes get the same image.

Verification: head rotation moves the world correctly and with the right gain. Translation moves the world correctly and with the right scale. Nothing is inside-out or mirrored.

This is where world scale gets calibrated. Stand a known-size object (a doorway, a character) in view and verify it subtends the right angle. Getting this wrong poisons everything downstream, including all the artifact analysis.

24.3 Stage 3 — Projection control

Replace the application's projection with one built from `XrFovf`, including the frustum shift. Still mono.

Verification: the FOV matches the headset. Straight lines at the frustum edges are straight. Moving your head does not cause the image to shear.

24.4 Stage 4 — Depth capture

Locate, copy, and resolve the scene depth buffer. Visualise it as a debug overlay.

Verification: the depth visualisation matches the scene geometry, has correct near/far behaviour, and does not flicker or contain stale content. Verify the reversed-Z convention explicitly.

24.5 Stage 5 — Naive AER (M2)

Alternate eyes, hold the stale one. This will look bad. That is expected and it is the point — you now have a working stereo pipeline whose artifacts you can measure.

Verification: stereo depth is perceptible. The artifacts of Chapter 7 are present and identifiable against the test scenes.

24.6 Stage 6 — Alternate frame warp (M5)

Add per-eye pose reprojection of the stale eye. Rotation-only first, then add the translational component.

Verification: the disparity shear during head rotation is essentially eliminated. Measure with the depth-staggered-pole test scene. This is the largest single quality jump in the project.

24.7 Stage 7 — Stereo warp (M4)

Add same-instant stereo reprojection with dual-source filling.

Verification: disocclusion area fraction below 3% on representative content. Inter-ocular SSIM ratio above 0.95.

24.8 Stage 8 — Tick lock and M1

Implement and verify the tick lock with the differential test. Then implement the held-tick frame pair.

Verification: the differential test passes bit-exactly. Stereo pairs show zero disparity error under the strafe test.

24.9 Stage 9 — Frame generation (M6)

Add flow estimation and temporal warping. Camera-derived reprojection first (free, exact for static scenes), then residual flow.

Verification: generated frames are visually indistinguishable from rendered ones in static scenes; artifacts appear only around fast-moving objects.

24.10 Stage 10 — Mode switching and hardening

Add the state machine, capability probing, degradation ladder, and the failure handling of Chapter 19.

Verification: a two-hour session across varied content with no crashes, no mode oscillation, and no unexplained quality changes.

Do not reorder. In particular, do not attempt Stage 9 before Stage 6; frame generation built on an uncorrected AER base produces artifacts that are extremely hard to attribute, and you will spend weeks debugging the flow estimator for problems that are actually pose staleness.

25 Extended Failure Catalogue

Chapter 19 covered failures common to all modes. These are specific to the modes introduced in Part VI onward.

25.1 Tick-lock failures

F1: Audio and network stall. Global time freezing halts audio mixing and network timeouts. *Symptom:* stuttering audio, disconnections. *Fix:* thread-scoped freeze (Chapter 23).

F2: Physics explosion on release. Some integrators, when handed a zero timestep followed by a normal one, produce a discontinuity — particularly constraint solvers with warm-starting. *Symptom:* ragdolls launching, vehicles jumping. *Fix:* verify with the differential test *and* a release test: lock and release repeatedly in a physics-heavy scene and watch for divergence from an unlocked baseline.

F3: Animation blend-tree corruption. Blend trees with time-based state machines may transition on a zero-dt frame if the transition check is $\geq$ rather than $>$. *Symptom:* animations skipping or resetting on alternate frames. *Fix:* if the tick lock cannot be made safe here, fall back to hooking the animation update specifically and skipping it.

F4: Watchdog termination. Some engines have a hang detector that fires if the simulation does not advance. *Symptom:* the process exits or displays a “not responding” state. *Fix:* advance time by a tiny epsilon (say 10^{-6} s) rather than exactly zero. The differential test will show a difference that small as identical in practice for rendering purposes, but it keeps monotonic-time assertions happy. Verify the epsilon does not accumulate into visible drift.

F5: Frame-counter-driven effects diverge. As in Chapter 23. *Symptom:* dithering or noise patterns differ between eyes; TAA jitter sequence misaligned. *Fix:* hook the counter or accept.

F6: Deadlock on locked tick with a job system. If the engine’s job system waits for a simulation job that you have prevented from being scheduled, the frame never completes. *Symptom:* hard hang on enabling M1. *Fix:* the timer-freeze approach avoids this because jobs still run, they just compute zero-delta results. Direct tick-function skipping is what triggers this; prefer timer freezing on job-system engines.

25.2 Async AER failures

F7: Unbounded skew after a hitch. *Symptom:* severe depth distortion for a second or two after a stall. *Fix:* the staler-eye scheduling policy plus the skew-driven degradation ladder from Chapter 24.

F8: Ring exhaustion. If the submission thread never finds a complete slot — because GPU work is backing up — it repeats the previous frame indefinitely. *Symptom:* frozen image while the game is clearly still running. *Fix:* detect consecutive repeats, and on exceeding a threshold, force a synchronous wait for the next completion and log it.

F9: Torn slot publication. A slot read while being written. *Symptom:* rare single-frame corruption, hard to reproduce. *Fix:* the acquire/release publication protocol in Chapter 24, and never reuse a slot that could still be referenced by the compositor — three slots minimum, four if your runtime holds images longer.

25.3 Warp and generation failures

F10: Warp using the wrong depth convention. *Symptom:* the reprojection moves in the wrong direction, or nearby objects move less than distant ones. *Fix:* verify the reversed-Z probe. A quick check: sample depth at the centre of the screen while walking toward a wall; in reversed-Z the value increases.

F11: Flow cache never invalidating. If your cache key is stale — for example, you ring-buffer capture textures and the same pointer comes back with new contents — the cache returns flow computed from different images. *Symptom:* ghosting that gets worse over time and never corrects. *Fix:* key on (pointer, capture sequence number), not pointer alone.

F12: Motion vector sign inverted. *Symptom:* moving objects ghost with a trail in front of them rather than behind. *Fix:* the calibration procedure in Chapter 28.

F13: Jitter double-correction. *Symptom:* fine shimmer at the jitter sequence period, worse than with no correction. *Fix:* the static-scene test in Chapter 28.

F14: Generated frames submitted with the rendered frame’s pose. *Symptom:* the world appears to stutter between smooth and jumpy at half the display rate — generated frames are spatially correct but temporally mis-timestamped. *Fix:* each submission carries the pose it was warped to, not the pose of its source.

F15: Interop import leak. Importing without caching, or caching without clearing on resize. *Symptom:* progressive memory growth and eventual allocation failure, or a hard performance cliff. *Fix:* Chapter 27’s import cache with explicit lifetime tied to resolution.

F16: Cross-API fence value regression. Signalling the shared fence with a value lower than one already signalled. *Symptom:* hangs or immediate-return waits that allow reads of incomplete data. *Fix:* a single monotonically incremented counter owned by one component; never let two subsystems signal the same fence independently.

25.4 Mode-switching failures

F17: Transition mid-pair. *Symptom:* a single frame with one eye black or duplicated when switching into or out of M1. *Fix:* pair-phase gating.

F18: Oscillation at a threshold. *Symptom:* visible quality pulsing at 1–2 Hz. *Fix:* asymmetric thresholds plus dwell time plus

cooldown.

F19: Capability regression not detected. The depth buffer becomes unavailable mid-session (the engine changed rendering path) but the mode does not step down. *Symptom:* the warp produces garbage. *Fix:* validate capabilities continuously, not only at startup, and gate each mode’s per-frame execution on its inputs actually being present this frame.

```
bool ModeInputsValid(StereoMode m, const FrameContext& fc) {
    switch (m) {
    case StereoMode::AlternateEyeWarpGen:
        return fc.depthValid && fc.flowValid && fc.prevFrameValid;
    case StereoMode::AlternateEyeWarp:
        return fc.depthValid;
    case StereoMode::SyncSequential:
        return fc.pairComplete && fc.tickLockHeld;
    default:
        return true;
    }
}
```

Call this every frame, and on failure fall through to the next mode down rather than executing with missing inputs. Executing a depth-dependent warp with a stale or cleared depth buffer is the single most common cause of “it worked yesterday” reports.

The Render Pass Census

Everything in Parts II through IX depends on knowing what the application’s renderer is doing. This chapter describes how to build that knowledge systematically rather than by guesswork, and how to persist it so it does not have to be rediscovered every session.

25.5 Why a census

Chapter 5’s classifier makes a per-draw decision from immediate state. That is sufficient for the common cases but fails on ambiguous ones, and gives you no visibility into what is actually happening. A census is the systematic alternative: enumerate every distinct pass in a frame, characterise it, and store the result in a database keyed by stable identity.

The payoffs are large:

- You can compute the C_i/C_v split exactly rather than estimating it.
- You know which passes to skip on the second eye of a same-tick pair.
- You know which passes to exclude from warping.
- You can detect when the renderer changes behaviour (entering a cutscene, a different weather system, a menu) because the census changes.
- You can ship per-application census data so users do not pay the discovery cost.

25.6 Stable pass identity

The identity must survive across frames and across sessions, and must not depend on pointer values or allocation order.

```
struct PassIdentity {
    uint64_t vsHash;           // vertex/mesh shader bytecode hash
    uint64_t psHash;           // pixel shader bytecode hash
    uint64_t csHash;           // compute shader bytecode hash (0 if graphics)
    uint32_t rtFormatMask;     // packed formats of bound render targets
    uint32_t rtCountAndFlags;  // count, depth bound, MSAA sample count
    uint16_t widthClass;       // quantised: full, half, quarter, eighth, other
    uint16_t heightClass;
    uint32_t blendStateHash;
    uint32_t depthStateHash;

    uint64_t Key() const {
        uint64_t h = 1469598103934665603ull;
        auto mix = [&h](uint64_t v) {
            h ^= v; h *= 1099511628211ull;
        };
        mix(vsHash); mix(psHash); mix(csHash);
        mix((uint64_t)(rtFormatMask) << 32 | rtCountAndFlags);
        mix((uint64_t)(widthClass) << 48 | (uint64_t)(heightClass) << 32
            | (uint64_t)(blendStateHash) ^ depthStateHash);
        return h;
    }
};
```

Shader bytecode hashing is the anchor. In D3D11 you get the bytecode at `CreateVertexShader/CreatePixelShader`; in D3D12 it is in

the PSO description; in Vulkan it is in `VkShaderModuleCreateInfo`. Hash at creation time and store in a map keyed by the API object pointer, so per-draw lookup is a hash-map hit rather than a re-hash.

Quantise the resolution rather than storing it exactly. An engine using dynamic resolution scaling changes render target dimensions every frame; without quantisation every frame produces new pass identities and the census never converges.

```
uint16_t ClassifyDimension(uint32_t d, uint32_t reference) {
    const float r = float(d) / float(reference);
    if (r > 0.90f) return 0; // full
    if (r > 0.45f) return 1; // half
    if (r > 0.22f) return 2; // quarter
    if (r > 0.11f) return 3; // eighth
    return 4; // other / small
}
```

25.7 What to record

For each identity, accumulate across many frames:

```
struct PassRecord {
    PassIdentity id;

    // Frequency
    uint32_t invocationsPerFrame; // mean
    uint32_t framesSeen;

    // Cost
    double gpuMsMean, gpuMsP99;
    double gpuMsAtFullRes, gpuMsAtHalfRes; // for the scaling probe

    // Geometry
    uint64_t drawCalls, primitives;

    // Behaviour flags, learned
    bool writesDepth;
    bool readsDepth;
    bool readsHistory; // samples a texture it also writes
    bool wroteToBackbufferChain; // reaches the final image
    bool cameraDependent; // determined by the perturbation probe
    bool resolutionScaling; // cost scales with pixel count

    // Classification
    PassKind kind;
    float kindConfidence;
};
```

25.8 The two probes that classify everything

25.8.1 Probe 1: resolution scaling

Run the application at two resolutions and compare per-pass cost. Passes whose cost scales with pixel count are view-dependent (or at least screen-space); those whose cost is invariant are view-independent or CPU-bound.

$$s = \frac{\log(t_{\text{full}}/t_{\text{half}})}{\log 4}$$

$s \approx 1$ means fully pixel-bound; $s \approx 0$ means invariant. Values in between indicate mixed or vertex-bound work.

You cannot always change the application's resolution externally, but you often can — via its own settings, via a hooked `ResizeBuffers`, or by intercepting the resolution it reads from its config. If none of those work, use the dynamic-resolution variation the engine produces naturally and correlate.

25.8.2 Probe 2: camera perturbation

This is the definitive test for view dependence, and it is more reliable than the resolution probe.

Apply a small perturbation to the camera — a rotation of a few degrees — for a single frame, and hash each pass's output. Passes whose output changes are camera-dependent; those whose output is identical are not.

```

void RunCameraPerturbationProbe(FrameContext& fc) {
    // Frame A: normal camera. Hash every pass output.
    g_probePhase = ProbePhase::Baseline;
    RenderOneFrame();
    auto baseline = CollectPassOutputHashes();

    // Frame B: tick locked, camera rotated by a small angle.
    EngageTickLock();
    g_cameraPerturbation = MakeYawRotation(kProbeAngleRadians);
    g_probePhase = ProbePhase::Perturbed;
    RenderOneFrame();
    auto perturbed = CollectPassOutputHashes();
    g_cameraPerturbation = Mat4Identity();
    ReleaseTickLock();

    for (auto& [key, h] : baseline) {
        auto it = perturbed.find(key);
        if (it == perturbed.end()) continue;
        g_census[key].cameraDependent = (it->second != h);
    }
}

```

Note this probe **requires the tick lock**, because otherwise simulation advancement changes every pass’s output and the probe returns “everything is camera-dependent.” This is a nice example of infrastructure paying for itself: the tick lock built for Chapter 23 is also the enabling mechanism for accurate pass classification.

Hashing a pass’s output requires reading back its render target, which is expensive. Do it for one probe frame only, and hash a downsampled version (say 64x64) rather than the full target — sufficient to detect a camera change, and cheap.

25.9 Deriving the classification

With both probes, classification becomes mostly mechanical:

cameraDependent	resolutionScaling	writesDepth	Likely kind
No	Yes	Yes	Shadow map
No	No	No	Skinning / simulation / probe update
Yes	Yes	Yes	World geometry
Yes	Yes	No	Deferred lighting / screen-space effect
Yes	No	No	Small screen-space pass, or vertex-bound
No	Yes	No	Post-process independent of camera (tonemap, bloom)

Combine with the immediate-state heuristics from Chapter 5 and take the higher- confidence answer. Where they disagree, log it — disagreements are where the interesting per-application quirks live.

25.10 Persisting and shipping the census

Serialise the census to a file keyed by application identity (executable name plus version or file hash) plus GPU vendor. On startup, load it; if the loaded census matches what you observe in the first few frames, skip the probes entirely.

```

[application]
name = <executable>
version_hash = <hash of main module>

[pass.0x9f3a...]
kind = ShadowMap
camera_dependent = false
resolution_scaling = true
skip_on_second_eye = true
gpu_ms_mean = 1.84

[pass.0x2b71...]
kind = UI
camera_dependent = false
exclude_from_warp = true
extract_to_layer = true

```

Also expose the file for manual override. A user or community maintainer who can say “pass 0x2b71 is the compass, exclude it from warping” turns a technique demo into something that works broadly. The heuristics should be the fallback, not the whole system.

25.11 Census-driven decisions

Once you have it, the census answers questions you previously guessed at:

```
float ComputeViewIndependentFraction(const Census& c) {
    double indep = 0, total = 0;
    for (const auto& [key, r] : c.records) {
        const double cost = r.gpuMsMean * r.invocationsPerFrame;
        total += cost;
        if (!r.cameraDependent) indep += cost;
    }
    return total > 0 ? float(indep / total) : 0.0f;
}

bool ShouldSkipOnSecondEye(const PassRecord& r) {
    if (r.cameraDependent) return false;           // must re-render
    if (r.readsHistory) return false;              // stateful; skipping changes it
    if (!r.writesDepth && !r.wroteToBackbufferChain) return false; // unknown effect
    return true;
}
```

ComputeViewIndependentFraction gives you ρ exactly, which feeds the cost model in Chapter 29 and therefore the mode selection in Chapter 30. Without it you are guessing at the single most important parameter in the whole system.

26 Temporal Upscaling and Antialiasing Under Stereo Amortisation

Chapter 7 identified TAA interaction as artifact A5. This chapter treats it fully, because on any modern title the temporal upscaler is the largest single interaction surface between your stereo scheme and the engine.

26.1 The structural problem

A temporal upscaler maintains a history buffer, reprojects it using motion vectors, validates it against the current frame, and accumulates. Its correctness depends on three assumptions:

1. The history corresponds to the previous frame of **the same view**.
2. Motion vectors describe the displacement over **one frame interval**.
3. The jitter sequence advances by **one element per frame**.

Every stereo amortisation scheme violates at least one of these.

Mode	Violation
M2–M6 (AER family)	History is 2 engine frames old for a given eye; motion vectors describe 1
M1 (same-tick)	Two renders per tick; history alternates between eyes unless separated
M0 (simultaneous)	Generally fine if the engine has per-view history

26.2 Per-eye history separation

The correct fix, and the one that makes everything else work, is to give each eye its own complete set of temporal state:

- History colour buffer
- History depth (if used)
- Accumulated sample count / confidence
- Any exponential moving averages (luminance for auto-exposure, for instance)
- The upscaler’s internal state, if it exposes a context object

For an engine-integrated upscaler with an explicit context handle, create two contexts and swap the active one based on the current eye. For a hand-rolled TAA, identify the history texture and swap it.

```
// Redirect the history texture binding based on the current eye.
void STDMETHODCALLTYPE Hooked_PSSetShaderResources_TaaAware(
    ID3D11DeviceContext* ctx, UINT start, UINT num,
    ID3D11ShaderResourceView* const* srvs)
{
    if (g_currentPassKind == PassKind::TemporalResolve) {
        std::vector<ID3D11ShaderResourceView*> patched(srvs, srvs + num);
```

```

    for (UINT i = 0; i < num; ++i) {
        if (patched[i] && ResourceOf(patched[i]) == g_engineHistoryResource)
            patched[i] = g_perEyeHistorySrv[(int)g_currentEye];
    }
    g_realPSSetShaderResources(ctx, start, num, patched.data());
    return;
}
g_realPSSetShaderResources(ctx, start, num, srvs);
}

```

You must also redirect the *write* — the pass that produces next frame’s history — to the same per-eye resource, or you will read per-eye history and write shared history.

Memory cost: one extra full-resolution history buffer, plus whatever auxiliary state. At 2000x2200 RGBA16F that is 35 MB. Acceptable.

26.3 Motion vector scaling

Under AER, a given eye’s history is two engine frames old. The engine’s motion vectors describe one frame. The reprojection therefore under-shoots by exactly a factor of two, producing history that is misaligned by one frame of motion — which the upscaler’s validation will reject, discarding history and causing aggressive flicker, or will accept, causing doubled ghost trails.

Fix by scaling the motion vectors the upscaler consumes:

$$\mathbf{F}_{\text{taa}} = k \cdot \mathbf{F}_{\text{engine}}$$

with k the number of engine frames between this eye’s current and previous render. For strict alternation $k = 2$. For async AER, k varies and you should compute it from the recorded tick identities.

Implementing this requires either patching the upscaler’s velocity-scale constant (if it has one — most do, for exactly this kind of situation) or running a pre-pass that multiplies the motion vector texture.

```

[numthreads(8, 8, 1)]
void ScaleMotionVectors(uint3 tid : SV_DispatchThreadID) {
    g_mvOut[tid.xy] = g_mvIn[tid.xy] * g_frameSpanScale;
}

```

Important caveat: this is only correct for motion that is constant over the two frames. For an object that changed direction, doubling the one-frame vector points somewhere wrong. In practice the error is small and the alternative (a persistently mis-scaled reprojection) is worse. Accumulate two frames of motion vectors and sum them if you want the exact answer and can afford the storage.

26.4 Jitter sequence management

The jitter sequence must advance once per *render of that eye*, not once per engine frame, and each eye should traverse the full sequence rather than every other element.

```

float2 JitterForEye(Eye eye, uint64_t eyeFrameCounter, uint32_t phaseCount) {
    // Per-eye counter, not global frame counter.
    const uint32_t idx = uint32_t(eyeFrameCounter % phaseCount);
    return float2(HaltonSequence(idx + 1, 2) - 0.5f,
                  HaltonSequence(idx + 1, 3) - 0.5f);
}

```

Additionally, **offset the two eyes’ sequences relative to each other**. If both eyes use the same jitter at the same index, they sample identical sub-pixel positions, which wastes the opportunity to average out sampling noise stereoscopically. Offset the right eye by half the sequence length:

```

const uint32_t idx = uint32_t((eyeFrameCounter + (eye == Eye::Right
    ? phaseCount / 2 : 0)) % phaseCount);

```

For same-tick stereo (M1), this is particularly valuable: the two eyes render the same instant with different jitter, and the visual system averages the two sampling patterns, producing a genuine perceived antialiasing improvement over mono.

26.5 Upscaling and the warp

If the engine upscales, you have a choice about where your warp runs.

Warp after upscaling (at display resolution): highest quality, most expensive, and the motion vectors need resampling to display resolution.

Warp before upscaling (at render resolution): cheaper, motion vectors are native resolution, but the warp’s artifacts get upscaled along with everything else, and the upscaler’s temporal validation will see the warped image as inconsistent with its history and may reject it.

Recommendation: warp after upscaling. The temporal upscaler’s history validation is not designed to cope with an externally warped input, and feeding it one produces history rejection that manifests as flicker. Keep the engine’s temporal chain intact and operate downstream of it.

The exception is the same-tick mode M1, where you are not warping the engine output at all — both eyes are genuinely rendered — and only the generated frames are warped. There, warping generated frames at display resolution is unambiguously correct.

26.6 Sharpness alternation

A subtle artifact specific to mixing rendered and warped images: a warped image is slightly softer than a rendered one, because it has been resampled with a bilinear or better filter. If every other display frame is warped, viewers perceive a sharpness pulsing at half the display rate.

Two mitigations:

1. **Match the filtering.** Apply an equivalent very mild resample to the rendered frames so both have been filtered once. Wasteful but effective, and cheap (a single bilinear pass at 0.1 ms).
2. **Sharpen the warped frames.** A light unsharp mask tuned to restore the resampling loss. Risk: over-sharpening produces ringing that is itself a temporal artifact.

Option 1 is more robust. Measure the difference in high-frequency energy between rendered and warped frames and tune until they match:

$$E_{hf}(I) = \sum_{\mathbf{u}} \|\nabla^2 I(\mathbf{u})\|^2$$

Aim for $|E_{hf}^{\text{warp}}/E_{hf}^{\text{render}} - 1| < 0.05$.

27 Canted Displays and Non-Standard Frusta

27.1 The problem

Several headset designs angle the two display panels inward or outward rather than mounting them parallel. This gives a wider combined field of view but means the two eyes’ view directions are **not parallel** — each eye’s view matrix includes a rotation, not merely a translation.

Everything in Chapters 6 through 9 that assumed the stereo transform is a pure horizontal translation is invalid on canted hardware. Specifically:

- The stereo disparity is no longer a pure horizontal shift (Chapter 9’s key simplification).
- The single-multiply disparity computation is wrong.
- Row-wise scatter optimisation for the stereo warp is wrong, because pixels move vertically as well as horizontally.

27.2 Detection

`xrLocateViews` returns a full pose per eye, including orientation. Compare the two eyes’ orientations:

```
bool DetectCantedDisplay(const XrView views[2], float* outAngleRadians) {
    const Quat qL = ToQuat(views[0].pose.orientation);
    const Quat qR = ToQuat(views[1].pose.orientation);
    Quat rel = Mul(qR, Conjugate(qL));
    if (rel.w < 0.0f) rel = Negate(rel);
    const float angle = 2.0f * acosf(std::clamp(rel.w, -1.0f, 1.0f));
    *outAngleRadians = angle;
    return angle > 0.5f * kDegToRad;    // more than half a degree: canted
}
```

Typical canted designs are in the 5–15 degree range per eye.

27.3 Handling it correctly

The general warp already works. Chapter 9’s full clip-space reprojection $R = (V_A P_A)^{-1} (V_B P_B)$ makes no assumption about the relationship between the two views. If you implemented the general form, canted displays work with no change.

What breaks is the optimisation. So:

```
enum class StereoWarpPath { HorizontalShift, GeneralReprojection };

StereoWarpPath SelectStereoWarpPath(const XrView views[2]) {
    float angle;
    if (DetectCantedDisplay(views, &angle)) return StereoWarpPath::GeneralReprojection;
    // Also require matching FOVs; asymmetric-but-parallel is still shifttable
    // only if the horizontal tangent extents match.
```

```

const float wL = tanf(views[0].fov.angleRight) - tanf(views[0].fov.angleLeft);
const float wR = tanf(views[1].fov.angleRight) - tanf(views[1].fov.angleLeft);
if (fabsf(wL - wR) > 0.01f * wL) return StereoWarpPath::GeneralReprojection;
return StereoWarpPath::HorizontalShift;
}

```

Implement both, select at runtime, and default to the general path when uncertain. The general path costs perhaps 15% more in the warp kernel — a full 4x4 transform per vertex rather than a single multiply — which is a small price for correctness across hardware you cannot test on.

27.4 Parallel projection mode

Most runtimes for canted hardware offer a “parallel projection” compatibility mode that renders with parallel frusta wide enough to cover the canted panels, then warps to the panel geometry in the compositor. This makes the hardware behave like a conventional headset from your perspective.

The cost is significant — typically 20–30% more pixels, because the parallel frustum must be wide enough to cover both canted frusta. But it means everything works.

Recommendation: support the general reprojection path so parallel projection is not required, but detect when it is enabled and switch to the cheaper horizontal path, since in that mode the views genuinely are parallel. Detecting it is exactly the `DetectCantedDisplay` call above returning false on hardware you know to be canted.

27.5 Other non-standard frusta

Asymmetric but parallel. The common case. Fully handled by Chapter 4’s `ProjectionFromFov` including the frustum shift terms. The horizontal-shift warp optimisation remains valid provided the two eyes’ tangent *extents* match even if their centres do not; the shift is then still purely horizontal, just with different NDC offsets per eye.

Different FOV per eye. Some hardware has slightly different panels or optics per eye. The horizontal-shift path is invalid; use general reprojection.

Off-centre optical axes. Handled by the frustum shift. No special case needed beyond using the runtime’s reported values rather than assuming symmetry.

Quad views / foveated view configurations. Some runtimes expose four views — a wide low-resolution view and a narrow high-resolution inset per eye. Everything in this document generalises, but you now have four warps rather than two, and the inset/outset composition must happen before or after your warp consistently. Treat each of the four as an independent view for warping purposes, and take care that the inset’s disocclusion filling can draw from the outset view, which covers the same region at lower resolution — an excellent fill source that costs nothing extra.

28 Foveation and Peripheral Quality

28.1 Why it interacts with everything here

Synthesis cost scales with pixel count. Every technique in this document — warping, flow estimation, inpainting, compositing — can be run at reduced quality in the periphery, where the visual system cannot resolve the difference. Because peripheral regions are the majority of the pixel count in a wide-FOV headset, the savings are large.

The fraction of pixels within a given eccentricity for a 100-degree FOV, assuming a circular high-quality region:

Inner radius (fraction of half-FOV)	Fraction of pixels
0.3	9%
0.4	16%
0.5	25%
0.6	36%

So running full-quality synthesis in the inner 50% by radius and half-quality outside cuts synthesis cost by roughly 37%.

28.2 Two-zone foveation: the cheapest effective form

Before reaching for graduated quality levels, consider the simplest scheme that actually works, which is more aggressive than most implementers expect and looks better than they expect too.

Divide the frame into two zones by radius from the optical centre.

- **Sweet spot (inner):** the full synthesis path — flow sampling, fusion, weight cascade, depth-aware blending, refinement.
- **Periphery (outer):** skip synthesis entirely. Copy the temporally nearest real frame, at reduced effective resolution, by snapping the sample coordinate to a block grid.

The periphery path is one read and one write. No flow, no depth, no fusion, no refinement. It is the cheapest possible per-pixel operation, and because peripheral acuity is low it hides warp artifacts rather than creating them — a peripheral region showing

a slightly stale but *coherent* frame is less noticeable than one showing a fresh frame with warp tears.

Two details make it work rather than look broken.

Grow the block size with eccentricity. A fixed 2x2 snap is imperceptible once the compositor’s own downscaling and lens distortion have been applied, which means you paid nothing for it. Scale the block size from roughly 4 pixels just outside the sweet spot up to 16 pixels at the lens edge, so the saving actually materialises where acuity is lowest.

Feather the boundary. A hard radial cut produces a visible aliased ring, which is far more noticeable than either zone’s content. Blend across a band — around 8% of the radius — fading the warped result toward the reduced-resolution raw sample.

```
// Returns true if this pixel was handled by the periphery fast path.
bool PeripheryFastPath(uint2 p, float2 uv, float foveation,
                      SurfaceHandle nearestFrame, SurfaceHandle outSurf)
{
    if (foveation <= 0.01f) return false;

    const float2 c = (uv - g_opticalCentreUv) * g_aspectCorrect;
    const float r = length(c) * 2.0f;          // 0 at centre, ~1 at edge
    const float inner = 1.0f - foveation;
    const float feather = 0.08f;

    const float foveBlend = saturate((r - inner) / feather);
    if (foveBlend < 1.0f) return false;        // sweet spot or feather band

    // Full periphery: block-snapped copy of the nearest real frame.
    const float depth = saturate((r - inner) / max(0.30f, 1.0f - inner));
    const int blk = 4 + int(depth * 12.0f);    // 4..16 px
    const int2 s = int2(p) - (int2(p) % blk);

    WritePixel(outSurf, p, ReadPixel(nearestFrame, s));
    return true;
}
```

and in the feather band, after computing the warped colour normally:

```
if (foveBlend > 0.0f) {
    const int2 s = int2(p) & ~3;               // 4 px blocks in the band
    const float3 raw = ReadPixel(nearestFrame, s).rgb;
    warped.rgb = lerp(warped.rgb, raw, foveBlend);
}
```

Choose “nearest real frame” by phase, not arbitrarily: at $\beta \geq 0.5$ the current frame is temporally closer, below that the previous one. Using the wrong one puts a temporal discontinuity at the zone boundary that the feather cannot hide.

This scheme composes with the graduated approach below — use two zones as the coarse structure and graduate quality within the sweet spot if you need finer control.

28.3 Graduated foveation in the warp

A more granular form: reduce the warp grid density and skip the refinement stages progressively rather than switching between two zones.

```
float FoveationWeight(float2 uv, float peripheryFraction) {
    if (peripheryFraction <= 0.001f) return 1.0f;
    // Normalised radial distance from the eye's optical centre.
    const float2 d = (uv - g_opticalCentreUv) * g_aspectCorrect;
    const float r = length(d) * 2.0f;          // 0 at centre, ~1 at edge
    const float inner = 1.0f - peripheryFraction;
    return 1.0f - saturate((r - inner) / max(peripheryFraction, 1e-3f));
}
```

Use the weight to modulate:

- **Warp grid density.** Emit a coarser grid outside the inner region. A geometry or mesh shader can do this adaptively; a simpler approach is two draws with two grids and a stencil mask.
- **Flow refinement.** Skip the finest pyramid level in the periphery. Flow error in the periphery is far less visible.
- **Inpainting quality.** Fewer push-pull levels.
- **Occlusion-boundary sharpening.** Skip entirely outside the inner region.

```
struct WarpQualityLevels {
    uint32_t gridStrideInner = 2;    // pixels per grid vertex
```

```
uint32_t gridStrideOuter = 6;
uint32_t flowLevelsInner = 6;
uint32_t flowLevelsOuter = 4;
uint32_t inpaintLevelsInner = 5;
uint32_t inpaintLevelsOuter = 3;
};
```

Use the optical centre, not the screen centre. In an asymmetric frustum the optical axis is off-centre, and foveating around the geometric centre of the render target puts the high-quality region in the wrong place. Compute it from the FOV:

$$u_{\text{centre}} = \frac{-t_L}{t_R - t_L}, \quad v_{\text{centre}} = \frac{t_U}{t_U - t_D}$$

28.4 Eye-tracked foveation

If the runtime exposes gaze (via an eye tracking extension), centre the high-quality region on the gaze point rather than the optical axis. The savings increase substantially because the high-quality region can be much smaller — the fovea subtends only about 5 degrees.

Two cautions specific to this document’s techniques:

Latency. Gaze moves in saccades at up to 900 deg/s. If your quality region lags the gaze by more than about 50 ms, the user will occasionally look at a low-quality region before it upgrades. Predict gaze with the same machinery as Chapter 26’s pose prediction, and size the high-quality region with a margin for prediction error.

Do not foveate the flow estimation asymmetrically between eyes. If the left eye’s flow is computed at higher quality than the right eye’s, the two eyes’ generated frames differ in a structured way, which is a rivalry condition. Foveate identically in both eyes, centred on each eye’s own projection of the gaze point.

28.5 Peripheral degradation as a degradation-ladder step

Foveation slots naturally into the ladder from Chapter 12 as an intermediate step between full quality and dropping a whole technique:

Level	Configuration
2	AER v2 + generation, no foveation
2.5	AER v2 + generation, fixed foveation 0.35
3	AER v2 + generation, fixed foveation 0.5, half-res flow

The half-step is valuable because it degrades gracefully in a region the user is unlikely to notice, rather than degrading uniformly.

29 Near-Field Content: Weapons, Hands, and Cockpits

29.1 Why the near field dominates artifact severity

Chapter 9 derived the disocclusion gap width:

$$w_{\text{gap}} = \frac{b P_{00} W}{2} \left| \frac{1}{z_1} - \frac{1}{z_2} \right|$$

The $1/z$ dependence means a first-person weapon at 0.4 m against a background at 10 m produces a gap roughly 25 times wider than an object at 10 m against a background at infinity. Near-field content is where every synthesis technique fails worst, and it is simultaneously the content the user looks at most.

It is also the content most likely to be:

- **Rendered with a separate projection.** Many engines render the first-person weapon with a narrower FOV and a compressed depth range to prevent clipping into walls. Its depth values are not comparable to the world’s.
- **Drawn in a separate pass** after the main scene, sometimes after a depth clear.
- **Excluded from motion vectors** or given incorrect ones.

29.2 Detecting the near-field pass

The signature is distinctive:

```
bool LooksLikeFirstPersonPass(const BoundState& s, const PassRecord& r) {
    // Rendered after the main world pass, into the same colour target.
    if (!s.rtMatchesSceneSize) return false;
    if (r.drawCalls > 200) return false;           // few draws
```

```
// Depth cleared or depth range compressed just before.
if (s.depthWasClearedThisFrame || s.depthRangeCompressed) return true;
// Or: a distinct projection matrix with a much narrower FOV.
if (s.projectionFovRatioToMain < 0.85f) return true;
return false;
}
```

The depth-range trick is the giveaway. An engine rendering a weapon that must not clip into geometry typically sets a viewport depth range of, say, $[0.0, 0.1]$, compressing the weapon into the nearest tenth of the depth buffer. Hook `RSSetViewports / vkCmdSetViewport` and watch for a mid-frame depth range change.

29.3 Handling strategy by mode

Same-tick modes (M1, M0). No special handling needed. Both eyes are genuinely rendered, so the near field is correct by construction. This is a significant and underappreciated advantage of M1: it is the only mode where a first-person weapon looks exactly right.

AER with stereo warp (M4). The near field must be rendered for both eyes. This is Chapter 9’s Mitigation 1, and for first-person content it is not optional.

Implementation: after the world pass completes and you have captured it, re-issue *only* the near-field pass with the second eye’s camera, into a separate target with alpha, and composite. Because the near-field pass has few draw calls and low overdraw, re-issuing it is cheap — typically 5–12% of frame cost rather than the 100% a full second view would cost.

```
void RenderNearFieldSecondEye(FrameContext& fc) {
    if (!g_census.nearFieldPassIdentified) return;

    BindTarget(g_nearFieldTarget[1]);           // second eye, cleared to transparent
    ClearRenderTarget(g_nearFieldTarget[1], {0,0,0,0});
    ClearDepth(g_nearFieldDepth[1]);

    SetCameraForEye(Eye::Right);                // or whichever is synthesised
    g_replayNearFieldOnly = true;
    ReplayCapturedDrawRange(g_census.nearFieldDrawRange);
    g_replayNearFieldOnly = false;
}
```

Note this is a replay — the narrow kind that Chapter 25 says sometimes works. It works here specifically because the near-field pass is small, self-contained, usually does not use transient aliased resources, and does not participate in temporal accumulation. Probe it with the viability test; if it passes, this path is by far the best answer for first-person content.

If replay fails, fall back to rendering the near field only in the dominant eye and accepting monoscopic weapon rendering. Surprisingly, this is tolerable — a monoscopic weapon at a fixed apparent depth is far less objectionable than a weapon with a 150-pixel disocclusion tear down one side. Many users do not notice.

Frame generation (M6). The near field must be excluded from flow-based warping or given its own treatment, because:

- Its motion is dominated by weapon sway and view-model animation, which the engine’s motion vectors may not cover.
- It occupies the region of highest visual attention.
- Its extreme depth means pose-driven reprojection displaces it enormously.

The cleanest approach is to **composite the near field after generation**: generate the world frame, then draw or warp the near field on top using its own, correct transform. If the near field is rendered each engine frame anyway (as in the both-eyes strategy above), you can simply re-composite the most recent near-field render onto the generated world frame with a pose-only warp.

29.4 Depth-range unification

To use near-field content in any depth-based technique you must convert its compressed depth into the same space as the world. If you captured the viewport depth range $[d_{\min}, d_{\max}]$ used for the near-field pass, and the near-field projection’s near and far planes, you can reconstruct view-space depth:

$$d_{\text{ndc}} = \frac{d_{\text{buffer}} - d_{\min}}{d_{\max} - d_{\min}}, \quad z_v = \frac{P_{32}^{nf}}{d_{\text{ndc}} - P_{22}^{nf}}$$

then re-encode into the world’s depth convention. Do this in a small compute pass over the near-field region only, producing a unified depth buffer that all downstream techniques can use.

```
[numthreads(8, 8, 1)]
void UnifyNearFieldDepth(uint3 tid : SV_DispatchThreadID) {
    const float dbuf = g_depth[tid.xy];
    if (dbuf < g_nfRangeMin || dbuf > g_nfRangeMax) return; // not near-field
```

```

const float dn = (dbuf - g_nfRangeMin) / (g_nfRangeMax - g_nfRangeMin);
const float zv = g_nfP32 / (dn - g_nfP22);

// Re-encode into the world projection (reversed-Z infinite).
g_unifiedDepth[tid.xy] = g_worldZNear / max(zv, g_worldZNear);
}

```

This unification is worth the effort: with a correct unified depth buffer, the near field participates properly in warping, occlusion ordering, and depth-layer submission, and a whole family of “the weapon looks wrong” bugs disappears.

29.5 Cockpits and large near-field geometry

Vehicle and aircraft content is the hard case: the near field is not a small weapon model but a cockpit occupying 40–60% of the screen at 0.3–0.8 m. Consequences:

- The both-eyes near-field render is no longer cheap; it is most of the frame.
- Disocclusion from any warp is enormous over a large area.
- The cockpit is static relative to the head in a way that makes any error highly visible, because the viewer has a strong expectation of its rigidity.

For this content, **same-tick stereo (M1) is close to mandatory**. The synthesis approaches all fail. The good news is that cockpit content is often less demanding on the world render — the visible world is a fraction of the screen — so the 2x cost of M1 is more affordable than it would be in a dense outdoor scene.

If M1 is genuinely unaffordable, the fallback ordering is:

1. Render the cockpit for both eyes, warp only the through-canopy world.
2. Reduce the cockpit’s render resolution for the synthesised eye rather than warping it.
3. Accept a monoscopic cockpit at a fixed convergence distance.

Option 3 is more usable than it sounds for content where the cockpit is largely static, and it is far better than a warped cockpit with tears along every strut. # Gather-Based Warp Synthesis

Chapters 9 and 11 developed the warp as a **forward scatterer**: for each source pixel, compute where it lands and write there, with depth ordering resolving collisions. That formulation is geometrically exact and handles occlusion correctly through the depth buffer, but it is expensive — it needs a grid mesh or atomic scatter, a depth target, and a separate hole-filling pass.

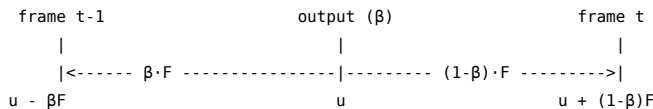
There is an alternative formulation that is cheaper, simpler, and in many cases produces better results for temporal synthesis specifically. It deserves its own treatment because the design reasoning is entirely different.

29.6 The gather formulation

Instead of asking “where does this source pixel go?”, ask “where did this destination pixel come from?” For each output pixel at UV $\mathbf{u}$, with a flow field $\mathbf{F}$ describing motion from frame $t-1$ to frame t , and a temporal phase $\beta \in [0, 1]$:

$$\mathbf{u}_{\text{prev}} = \mathbf{u} - \beta \mathbf{F}, \quad \mathbf{u}_{\text{curr}} = \mathbf{u} + (1 - \beta) \mathbf{F}$$

Sample both source frames at those positions and blend. One thread per output pixel, two bilinear reads, one write. No mesh, no depth target, no atomics, no separate inpainting pass.



The geometric assumption is that the flow field is locally invertible — that tracing backward from the output position by $\beta \mathbf{F}(\mathbf{u})$ lands on the pixel that will arrive at $\mathbf{u}$. This is exactly true where the flow is locally constant and approximately true where it varies smoothly. It is **wrong** at motion boundaries, where the flow sampled at the destination belongs to whichever surface happens to occupy that destination.

That is the trade. Forward scatter is geometrically correct and needs explicit occlusion handling; gather is geometrically approximate and needs explicit *disagreement* handling. For temporal synthesis, where displacements are small and motion boundaries are a minority of pixels, gather wins on cost and is competitive on quality — provided you handle the boundaries, which is the rest of this chapter.

29.7 Bidirectional sampling and why it matters

The single most important property of the gather formulation is that it naturally samples **both** source frames. This is not an optimisation; it is what makes the technique work.

A region that is disoccluded between $t-1$ and t — revealed as an object moves — has no valid data in frame $t-1$ but does have it in frame t . A region occluded over the same interval has the reverse. Sampling both sides and choosing per-pixel means the overwhelming majority of disocclusion is filled with **real data** rather than inpainted guesses.

This is why bidirectional gather produces noticeably fewer of the torn regions that plague unidirectional warping. It is also why, in the extrapolation case (below), where only one source exists, quality is inherently lower and different handling is needed.

29.8 The weight cascade

Everything interesting is in how you choose the blend weight w between the two samples. Rather than a single formula, build it as a **cascade of corrections**, each a lerp toward a “pick” value by a strength factor. This structure is worth adopting wholesale: each stage is independently tunable, independently disableable, and degrades to the previous stage’s answer when its condition is not met.

29.8.1 Stage 0 — Phase

$$w \leftarrow \beta$$

The baseline. At $\beta = 0$ take the previous frame entirely, at $\beta = 1$ the current frame, linearly in between. This is correct for perfectly interpolable content.

29.8.2 Stage 1 — Bounds

If one sample position falls outside the frame, that side has no data. Take the other side entirely.

$$w \leftarrow \begin{cases} 0 & \text{if } \mathbf{u}_{\text{prev}} \text{ in bounds and } \mathbf{u}_{\text{curr}} \text{ out} \\ 1 & \text{if } \mathbf{u}_{\text{curr}} \text{ in bounds and } \mathbf{u}_{\text{prev}} \text{ out} \\ w & \text{otherwise} \end{cases}$$

This is a hard switch, not a blend, because a clamped out-of-bounds sample is edge smear and blending it in is strictly harmful.

29.8.3 Stage 2 — Confidence

Define $\text{pick}_{\text{near}}$ as the temporally nearest real frame: 0 if $\beta < 0.5$, else 1. When flow confidence is low, the sample positions are untrustworthy, and the best fallback is the nearest real frame **unwarped in spirit** — i.e. weight fully toward it rather than blending two badly-placed samples.

$$w \leftarrow w + (1 - c) (\text{pick}_{\text{near}} - w)$$

with $c \in [0, 1]$ the per-pixel confidence from the flow engine’s cost map (Chapter 27) or from the margin/consistency estimate (Chapter 10).

This stage is what makes the whole scheme robust. A completely failed flow estimate degrades gracefully to “show the nearest real frame”, which is judder — unpleasant but never structurally wrong. Without it, a failed estimate produces a smear, which is much worse.

29.8.4 Stage 3 — Depth-aware occlusion

At a genuine depth discontinuity, the two sample positions see different surfaces. Blending them produces a semi-transparent halo where foreground and background mix. The correct choice is the **nearest** surface, because it is the one that occludes.

Sample depth at both source positions. In reversed-Z, larger is nearer.

$$\Delta_d = |d_{\text{prev}} - d_{\text{curr}}|, \quad \text{pick}_d = \begin{cases} 0 & d_{\text{prev}} > d_{\text{curr}} \\ 1 & \text{otherwise} \end{cases}$$

$$o = \text{clamp}\left(\frac{\Delta_d - \delta_0}{\delta_1}, 0, 1\right), \quad w \leftarrow w + o (\text{pick}_d - w)$$

The two thresholds matter. δ_0 is the depth difference below which the surfaces are considered the same (absorbing depth-buffer noise and shading gradients on a continuous surface); δ_1 controls how sharply the pick engages. Reasonable starting values in reversed-Z normalised depth are $\delta_0 = 0.02$ and $\delta_1 = 0.05$, but these are scene-scale dependent — express them in **view-space** metres if you can, where $\delta_0 \approx 0.1$ m and $\delta_1 \approx 0.3$ m are more portable.

Guard the whole stage on both depths being valid. Sky and cleared regions have degenerate depth and will otherwise pick arbitrarily.

29.8.5 Stage 4 — Photometric disagreement

Depth catches geometric occlusion. It misses cases where the two samples differ for other reasons: a moving shadow, a specular highlight that moved, an alpha-blended effect, an object whose motion the flow field missed entirely. In all of these, a 50/50 blend produces a **ghost** — a semi-transparent double image.

Detect disagreement photometrically and sharpen toward the nearest real frame:

$$\Delta_L = |L(\mathbf{c}_{\text{prev}}) - L(\mathbf{c}_{\text{curr}})|, \quad s = \text{clamp}\left(\frac{\Delta_L - \lambda_0}{\lambda_1}, 0, 1\right) \cdot k_{\text{sharp}}$$

$$w \leftarrow w + s (\text{pick}_{\text{near}} - w)$$

with L the luminance. Starting values: $\lambda_0 = 0.12$, $\lambda_1 = 0.28$ in normalised luminance. k_{sharp} is the user-facing strength.

The trade this stage makes is explicit and worth understanding: **sharp edges at the cost of judder in disagreeing regions**. Where the two frames agree, you keep the smooth interpolation that hides judder; where they disagree, you snap to one frame, which judders but stays crisp. Since disagreement regions are exactly where a blend would look wrong, this is almost always the right trade. Expose it as a slider, because the correct value is content-dependent.

29.8.6 Putting the cascade together

```
float ComputeBlendWeight(
    float phase,          // beta
    bool prevInBounds, bool currInBounds,
    float confidence,      // 0..1 from cost map
    float depthPrev, float depthCurr, bool depthValid,
    float3 colorPrev, float3 colorCurr,
    float occlusionSharpStrength)
{
    // Stage 0: phase
    float w = phase;
    const float pickNear = (phase < 0.5f) ? 0.0f : 1.0f;

    // Stage 1: bounds (hard)
    if (prevInBounds && !currInBounds) w = 0.0f;
    else if (!prevInBounds && currInBounds) w = 1.0f;

    // Stage 2: confidence
    w += (1.0f - confidence) * (pickNear - w);

    // Stage 3: depth-aware occlusion
    if (depthValid) {
        const float dd = abs(depthPrev - depthCurr);
        if (dd > kDepthSame) {
            const float pickD = (depthPrev > depthCurr) ? 0.0f : 1.0f; // reversed-Z
            const float occ = saturate((dd - kDepthSame) / kDepthEdge);
            w += occ * (pickD - w);
        }
    }

    // Stage 4: photometric disagreement
    const float lp = dot(colorPrev, float3(0.299f, 0.587f, 0.114f));
    const float lc = dot(colorCurr, float3(0.299f, 0.587f, 0.114f));
    const float disagree = abs(lp - lc);
    const float sharpen = saturate((disagree - kLumaSame) / kLumaEdge)
        * saturate(occlusionSharpStrength);
    w += sharpen * (pickNear - w);

    return saturate(w);
}
```

Note that every stage after Stage 1 has the same shape: $w += \text{strength} * (\text{pick} - w)$. That uniformity is deliberate — it makes the cascade easy to reason about, easy to extend with a new signal, and safe to disable any stage by zeroing its strength.

29.9 Extrapolation in the gather formulation

When $\beta > 1$ you are extrapolating past the most recent real frame and there is no second source. The bidirectional machinery does not apply and the handling must change fundamentally.

Let $\epsilon = \beta - 1$ be the extrapolation distance. Gather from the current frame only:

$$\mathbf{u}_{\text{src}} = \mathbf{u} - \epsilon \mathbf{F}_{\text{obj}} \gamma - \mathbf{F}_{\text{cam}} \kappa$$

The crucial insight is that the two motion components must be treated asymmetrically.

29.9.1 Object motion: fade out as it grows

Residual object flow is an estimate. Extrapolating an estimate produces error proportional to the extrapolation distance, and a large estimated displacement is precisely the case most likely to be wrong. So attenuate it:

$$\gamma = c \cdot \begin{cases} 1 & \|\epsilon \mathbf{F}_{\text{obj}}\| \leq m \\ \max\left(0, 1 - \frac{\|\epsilon \mathbf{F}_{\text{obj}}\| - m}{m}\right) & \text{otherwise} \end{cases}$$

with c the confidence and m a magnitude threshold in UV units — around 0.04 (4% of the frame) is a reasonable starting point. The effect: a bad or extreme vector degrades to **no displacement at all** — a frozen pixel — rather than to a long smear. A frozen pixel reads as mild judder; a smear reads as broken rendering. This asymmetry in perceived severity is the whole justification for the fade.

29.9.2 Camera motion: apply fully, but bound

Pose-derived camera displacement is not an estimate. It comes from the tracker and the depth buffer, and it is the **dominant** displacement during head motion — the part the flow field cannot extrapolate at all, since flow only ever saw $t-1 \rightarrow t$ object motion.

Applying it fully is what stops the world tearing and juddering on turns. This was the missing piece in early implementations that faded all motion uniformly: the world would tear on every head turn because the one reliable, dominant term was being attenuated alongside the unreliable one.

So $\kappa = 1$, subject only to a magnitude bound that catches degenerate depth:

$$\kappa = \min\left(1, \frac{m_{\text{cam}}}{\|\mathbf{F}_{\text{cam}}\|}\right)$$

with $m_{\text{cam}} \approx 0.15$ UV. This is a clamp, not a fade — it caps the displacement without reducing it in the normal case. Its purpose is purely to contain the blow-up when a bad depth value produces an absurd reprojection.

```
float2 ExtrapolationSourceUv(
    float2 uv, float epsilon,
    float2 flowObjUv, float confidence,
    float2 flowCamUv, bool camValid, float camStrength)
{
    // Object motion: fade out with magnitude.
    float2 objOff = flowObjUv * epsilon;
    const float objMag = length(objOff);
    float objW = confidence;
    if (objMag > kMaxObjShift)
        objW *= saturate(1.0f - (objMag - kMaxObjShift) / kMaxObjShift);
    float2 src = uv - objOff * objW;

    // Camera motion: full strength, magnitude-clamped.
    if (camValid && camStrength > 0.0f) {
        const float camMag = length(flowCamUv);
        const float clampScale = (camMag > kMaxCamShift)
            ? (kMaxCamShift / camMag) : 1.0f;
        src -= flowCamUv * (camStrength * clampScale);
    }

    // Out of frame: no source data. Fall back to the unwarped pixel rather
    // than a clamped edge smear.
    if (any(src < 0.0f) || any(src > 1.0f)) src = uv;
}
```

```
    return src;
}
```

The final fallback — reverting to **u** when the source lands off-frame — is important and easy to get wrong. Clamping to the edge produces the streaked border that is the visual signature of naive extrapolation. Reverting to the unwarped pixel produces a static border instead, which is far less noticeable and which the render margin of Chapter 11 should be preventing anyway.

29.10 Gather versus scatter: when to use which

Property	Forward scatter (Ch. 9, 11)	Gather (this chapter)
Geometric exactness	Exact	Approximate at motion boundaries
Occlusion handling	Depth buffer, automatic	Weight cascade, explicit
Disocclusion	Holes, needs inpainting	Filled from the other source
Cost	Grid mesh or atomics + depth + inpaint	1 thread/pixel, 2 reads
Large displacement	Handles well	Degrades; sampling assumption breaks
Best for	Stereo warp, pose warp	Temporal interpolation

The practical answer is to use both, for different stages. The stereo warp of Chapter 9 involves a large, structured, depth-proportional displacement with genuine disocclusion — scatter with depth ordering is right. Temporal synthesis involves small displacements between two available frames — gather with the weight cascade is right, and substantially cheaper.

A pipeline using both:

1. Stereo warp (scatter): rendered eye -> synthesised eye, same instant
2. Temporal synth (gather): frames t-1, t of each eye -> intermediate/extrapolated
3. Pose warp (gather, rotation-only): final images -> latest predicted pose

Step 3 uses the gather path because rotation-only reprojection is depth-independent and exactly invertible (Chapter 26), making gather not merely cheaper but exact.

30 Robust Motion Fusion by Agreement Gating

Chapter 28 described calibrating engine motion vectors by regression against camera-derived motion. That works, but it is a startup procedure that can fail, and a mis-calibration corrupts every subsequent frame silently.

There is a better approach that is self-correcting per pixel, requires no calibration phase, and cannot corrupt the result even when the auxiliary data is completely wrong.

30.1 The principle

You have a **measured** flow field F_{meas} from optical flow — noisy, grid-quantised, but derived from the actual images and therefore correct in direction wherever it has signal.

You have **auxiliary** motion estimates that are potentially sharper or more complete but whose sign, scale, and coordinate convention you may have wrong:

- Engine motion vectors: per-pixel exact, no grid blockiness, but the encoding convention is engine-specific and possibly inverted.
- Pose-derived reprojection: analytically exact for camera motion, but depends on correctly recovered matrices, a correct depth convention, and a correct handedness assumption — any of which may be wrong.

Fuse them only where they agree in direction with the measurement. Where they agree, adopt them, weighted by how strongly they agree. Where they disagree, ignore them entirely and keep the measured flow.

The consequence is powerful: a wrong sign, a wrong scale factor, a transposed matrix, or a wrong handedness assumption causes the auxiliary source to disagree everywhere, so it is ignored everywhere, and the system degrades to measurement-only.

The failure mode is loss of refinement, never corruption.

30.2 Implementation

```
// Fuse an auxiliary motion estimate into a measured flow field, gated on
// directional agreement. Returns the (possibly refined) flow.
float2 AgreementGatedFuse(
    float2 measured,      // px
    float2 auxiliary,     // px, convention possibly wrong
    float agreeThreshold, // cosine similarity floor, e.g. 0.5
    float maxWeight,      // cap on how far to move toward aux, e.g. 0.75
    float strength)       // user/foveation modulation, 0..1
```

```

{
    const float mMag2 = dot(measured, measured);
    const float aMag2 = dot(auxiliary, auxiliary);

    // Both must carry real signal. Sub-pixel motion has no reliable direction.
    if (mMag2 < kMinMag2 || aMag2 < kMinMag2 || aMag2 > kMaxMag2)
        return measured;

    // Try both signs: this absorbs an inverted convention automatically.
    const float dotPos = dot(measured, auxiliary);
    float2 cand = (dotPos < 0.0f) ? -auxiliary : auxiliary;

    // Cosine similarity between the measurement and the candidate.
    const float agree = dot(measured, cand)
        * rsqrt(mMag2 * dot(cand, cand) + 1e-6f);

    if (agree <= agreeThreshold) return measured; // disagreement: ignore aux

    // Weight rises from 0 at the threshold to maxWeight at perfect agreement.
    const float w = saturate((agree - agreeThreshold) / (1.0f - agreeThreshold))
        * maxWeight * strength;

    return lerp(measured, cand, w);
}

```

Three details carry the robustness.

Trying both signs. This is what makes an inverted convention self-correcting rather than catastrophic. Note it corrects the *sign* only, not the *scale* — but since the fusion is a lerp toward the candidate rather than an adoption of it, moderate scale error is absorbed as a partial move.

The magnitude gate. Below a threshold, direction is meaningless: a sub-pixel vector’s direction is noise, and the cosine similarity of two noise vectors is uniform on $[-1, 1]$, so a quarter of such pixels would spuriously “agree” strongly. Require both vectors to exceed roughly half a pixel before comparing. The upper bound catches garbage values (uninitialised memory, NaN-adjacent floats) before they propagate.

The weight cap. Never move fully to the auxiliary value even at perfect agreement. Capping around 0.75 means the measurement always retains a vote, which bounds the damage from a source that agrees in direction but is badly wrong in magnitude — a real case when motion vector scale is off by a factor.

30.3 Applying it to both auxiliary sources

Chain the fusions, measurement first:

```

float2 FuseAllMotionSources(int2 p, float2 uv, float depth)
{
    // 1. Measured optical flow, in output pixels.
    float2 flow = SampleFlowBilinear(p);
    flow = SpatialSmoothFlow(p, flow, g_flowSmooth);

    // 2. Engine motion vectors: sharper, per-pixel, convention unknown.
    if (g_haveEngineMv) {
        const float2 mvPx = SampleEngineMvInOutputPixels(uv);
        flow = AgreementGatedFuse(flow, mvPx,
            /*threshold=*/0.50f,
            /*maxWeight=*/0.75f,
            g_refineStrength * g_foveationRefine);
    }

    // 3. Pose-derived camera motion: analytically exact, matrices unverified.
    if (g_haveDepth && depth > 1e-5f) {
        const float2 posePx = PoseReprojectDeltaPixels(uv, depth,
            g_currToPredicted);
        flow = AgreementGatedFuse(flow, posePx,
            /*threshold=*/0.60f,
            /*maxWeight=*/0.50f,
            g_refineStrength * g_foveationRefine);
    }
}

```

```
    return flow;
}
```

The pose source gets a **higher agreement threshold and lower weight cap** than the motion vector source. The reasoning: a wrong projection matrix produces a displacement field that is *plausibly shaped* — radially outward from a centre, scaling with depth — and can therefore achieve moderate agreement with genuine camera motion while being quantitatively wrong. This is the classic “fisheye” failure, where the world appears to bulge or pincushion during motion. A stricter gate and a smaller maximum contribution keep it in check.

30.4 Why this beats calibration

Calibration (Chapter 28) establishes a global scale and sign once. It is more precise when it succeeds — it recovers the actual magnitude, which agreement gating does not. But it has three weaknesses that agreement gating does not share:

1. **It can fail silently.** A calibration run during a scene with heavy object motion, or with a static camera, produces a wrong or unconverged result that is then applied everywhere.
2. **It is global.** One scale for the whole frame. If an engine encodes different conventions in different regions — some engines write dynamic-object motion in one convention and static reprojection in another — no single scale is right.
3. **It cannot detect per-pixel invalidity.** Pixels where the engine simply did not write motion vectors (particles, transparencies, UI) get whatever the buffer contained, and calibration has no way to reject them. Agreement gating rejects them automatically, because garbage does not agree with measured flow.

Use both. Calibrate for magnitude, then agreement-gate the calibrated result for safety. The calibration improves the refinement’s accuracy; the gate ensures a calibration failure costs you refinement rather than correctness.

30.5 Diagnostics

Instrument the gate. The fraction of pixels at which each auxiliary source is accepted is a direct measure of whether that source is working:

```
struct FusionStats {
    float mvAcceptedFraction;    // pixels where MV passed the gate
    float poseAcceptedFraction;
    float mvMeanAgreement;      // mean cosine among accepted
    float poseMeanAgreement;
    float mvSignFlipFraction;    // how often the negated candidate won
};
```

Interpretation:

- **mvAcceptedFraction near 0** — the motion vector source is not working at all. Wrong buffer, wrong format, wrong resolution mapping, or the buffer is empty.
- **mvSignFlipFraction near 1.0** — the convention is consistently inverted. The gate is handling it, but flip the sign at the source to recover full accuracy, since the flip costs you nothing but the gate’s weight cap does.
- **mvSignFlipFraction near 0.5** — the source is noise. The sign is flipping arbitrarily, which means there is no consistent relationship. Disable it.
- **poseAcceptedFraction low during head rotation** — the projection matrix or depth convention is wrong. This is the most valuable single diagnostic in the system, because pose reprojection *should* agree strongly during camera motion, and if it does not, something upstream is broken.
- **mvMeanAgreement high but visible artifacts** — direction is right, magnitude is wrong. Run the calibration.

Surface these in the debug overlay. An implementer looking at `poseAcceptedFraction = 0.04` during a head turn knows immediately to check the matrix recovery, rather than spending a week tuning the flow estimator.

30.6 Generalising the pattern

Agreement gating is a general technique for combining a trusted-but-noisy signal with an untrusted-but-precise one, and it applies elsewhere in this pipeline:

- **Depth from two sources.** If you have both a captured depth buffer and a depth estimate from stereo disparity, gate one against the other.
- **Pose sources.** If you predict pose yourself (Chapter 26) and also receive the runtime’s prediction, gate your extrapolation against the runtime’s to catch a broken predictor.
- **Flow across eyes.** When transferring a flow field from one eye to the other (Chapter 10’s cost optimisation), gate the transferred field against a cheap low-resolution flow computed directly for the target eye. Disagreement indicates the transfer assumption has broken down, typically at disocclusions.

The general form: never let an unverified source overwrite a verified one. Let it only refine, only where it corroborates, and always cap its influence. # Appendix A: Mathematical Reference

30.7 Projection matrices

Reversed-Z, infinite far plane, right-handed, row-vector convention:

$$P = \begin{bmatrix} \frac{2}{t_R - t_L} & 0 & 0 & 0 \\ 0 & \frac{2}{t_U - t_D} & 0 & 0 \\ \frac{t_R + t_L}{t_R - t_L} & \frac{t_U + t_D}{t_U - t_D} & 0 & -1 \\ 0 & 0 & z_n & 0 \end{bmatrix}$$

where $t_L = \tan(\theta_L)$ etc. are the signed tangent half-angles.

Depth reconstruction: $z_v = z_n/d$.

Finite far plane variant: $P_{22} = \frac{z_n}{z_f - z_n}$, $P_{32} = \frac{z_n z_f}{z_f - z_n}$, and $z_v = \frac{P_{32}}{d - P_{22}}$.

30.8 Stereo disparity

Horizontal NDC displacement between eyes for a point at view depth z_v :

$$\Delta x_{\text{ndc}} = \frac{b \cdot P_{00}}{z_v}$$

In pixels, for a screen of width W : $\Delta x_{\text{px}} = \frac{b P_{00} W}{2 z_v}$.

Angular disparity: $\theta \approx b/z_v$ radians.

Depth error from angular disparity error: $\Delta z \approx \frac{z_v^2}{b} \epsilon$.

30.9 AER temporal error

Disparity error from a temporal offset Δ with lateral head velocity v :

$$\frac{\epsilon}{\theta} = \frac{v \Delta}{b} \quad (\text{depth-independent})$$

30.10 Disocclusion gap width

For a depth edge between z_1 (near) and z_2 (far), in pixels:

$$w_{\text{gap}} = \frac{b P_{00} W}{2} \left| \frac{1}{z_1} - \frac{1}{z_2} \right|$$

Solving for the depth at which the gap falls below w_{max} , with $z_2 \rightarrow \infty$:

$$z_{\text{split}} = \frac{b P_{00} W}{2 w_{\text{max}}}$$

30.11 Reprojection

Full clip-space reprojection matrix from view A to view B :

$$R_{A \rightarrow B} = P_A^{-1} V_A^{-1} V_B P_B$$

With V^{-1} computed from the rigid-body structure and P^{-1} from the sparse closed form, never by general inversion.

30.12 Off-screen margin

Fraction of extra FOV needed to cover head rotation between frames:

$$m = \frac{\omega_{\text{max}} \Delta}{\text{FOV}}$$

Extra pixel cost: $(1 + m)^2 - 1$.

30.13 Speedup model

$$S = \frac{C_i + 2C_v + C_f}{\frac{1}{k} \left(C_i + \frac{2C_v}{s} + C_f \right) + C_{\text{synth}}}$$

where k is the temporal generation ratio and s is the stereo amortisation factor (2 for AER).

30.14 Tick-lock and same-tick stereo

Pair rate for synchronized sequential stereo, engine mono frame time τ :

$$f_{\text{pair}} = \frac{1}{2\tau + \delta_{\text{lock}}}$$

where δ_{lock} is tick-lock overhead (typically under 0.1 ms).

Simulation-to-submit latency: $T = 2\tau$, identical to simultaneous stereo of the same total work.

View-independent sharing saving, with $\rho = C_i / (C_i + C_v)$:

$$\text{cost}_{M1} = (2 - \rho)(C_i + C_v) + 2C_f$$

Measured $\rho_{\text{shareable}} \approx 0.06\text{--}0.12$ for deferred content with engine-cached shadow maps.

30.15 Asynchronous AER skew

Maximum tolerable skew for a disparity-error tolerance η at lateral speed v :

$$\sigma_{\text{max}} = \frac{\eta b}{v}$$

At $\eta = 0.15$, $b = 0.064$ m, $v = 1$ m/s: $\sigma_{\text{max}} = 9.6$ ms.

30.16 Alternate frame warp

Rotation-only reprojection homography (depth-independent, invertible, hole-free):

$$H = P^{-1} R(\mathbf{q}_r)^T R(\mathbf{q}_d) P$$

Full reprojection including translation:

$$R_{r \rightarrow d} = P^{-1} V(\mathbf{P}_r)^{-1} V(\mathbf{P}_d) P$$

Translational disocclusion gap, by analogy with stereo (Appendix A, stereo section) with $b \rightarrow \|\Delta \mathbf{t}\|$:

$$w_{\text{gap}}^{\text{AFW}} = \frac{\|\Delta \mathbf{t}\| P_{00} W}{2} \left| \frac{1}{z_1} - \frac{1}{z_2} \right|$$

Since $\|\Delta \mathbf{t}\| \ll b$ for realistic head motion over one frame, AFW's disocclusion is typically 10–30x smaller than the stereo warp's.

30.17 Pose prediction

Least-squares slope over a window of n samples:

$$\text{slope} = \frac{\sum x_i y_i - n \bar{x} \bar{y}}{\sum x_i^2 - n \bar{x}^2}$$

Angular velocity from consecutive orientations:

$$\omega = \frac{2 \log(\mathbf{q}_1 \mathbf{q}_0^{-1})}{t_1 - t_0}, \quad \log(\mathbf{v}, w) = \frac{\mathbf{v}}{\|\mathbf{v}\|} \arccos(w)$$

Orientation prediction:

$$\mathbf{q}(t) = \exp\left(\frac{1}{2} \omega (t - t_1)\right) \mathbf{q}_1$$

Clamp $t - t_1 \leq 35$ ms.

30.18 Geometry-shader stereo injection

Canonical clip-space separation footer:

$$x'_{\text{clip}} = x_{\text{clip}} + S(w_{\text{clip}} - C)$$

S = separation, C = convergence (depth of the zero-disparity plane). Applied pre-divide so the shift scales with view depth. Side-by-side packing with tunable pack scale P :

$$x''_{\text{clip}} = P x'_{\text{clip}} \mp \frac{1}{2} w_{\text{clip}}$$

For an engine that aspect-adjusts horizontal FOV to a 2:1 target, $P = 1.0$.

30.19 Hardware flow decoding

Grid flow stored as `short2` with 5 fractional bits:

$$\mathbf{F}_{\text{px}} = \frac{(s_x, s_y)}{32}$$

Cell index from pixel $\mathbf{p}$ with grid size g and row stride S bytes:

$$i = \left\lfloor \frac{p_y}{g} \right\rfloor \cdot \frac{S}{4} + \left\lfloor \frac{p_x}{g} \right\rfloor$$

30.20 Combined mode cost

For a configuration rendering one view per engine frame, pairing every k engine frames, and generating m frames per pair:

$$f_{\text{display}} = \frac{(1 + m)}{k \tau}, \quad \text{fraction synthesised} = \frac{m}{1 + m}$$

M1+M6 is $k = 2, m = 1$: display rate = $1/\tau$, half of frames synthesised.

30.21 Composite quality score

$$Q = w_1 q_\epsilon + w_2 q_A + w_3 q_S + w_4 q_L$$

with each term clamped to $[0, 1]$: $q_\epsilon = 1 - (\epsilon/\theta)/0.20$, $q_A = 1 - A_{\text{disocc}}/0.05$, $q_S = (Q_{\text{stereo}} - 0.90)/0.10$, $q_L = 1 - (T_{mp} - T_{mp}^{\text{ref}})/0.020$. Weights (0.35, 0.20, 0.25, 0.20).

31 Appendix B: Glossary

AER — Alternate-Eye Rendering. Rendering one eye per application frame and synthesising the other.

ATW / Asynchronous Timewarp — Runtime-side reprojection of a rendered frame to a newer head orientation, correcting rotational latency.

ASW / Spacewarp — Runtime-side generation of a synthetic frame using motion vectors, correcting both rotation and translation and increasing effective frame rate.

Census transform — A local binary pattern encoding the ordinal relationship of a pixel to its neighbours; used as an illumination-invariant matching cost.

Disocclusion — A region visible from the destination viewpoint that was hidden from the source viewpoint, and therefore has no source data.

Forward warp / scatter — Warping by computing each source pixel's destination and writing there.

Backward warp / gather — Warping by computing, for each destination pixel, the source position to read from.

Fusion — The visual system's merging of the two eyes' images into a single percept. Fails under large inter-ocular difference, producing rivalry.

IPD — Interpupillary distance, the separation between the eyes' optical centres.

Judder — Perceived discontinuity in motion caused by insufficient temporal sampling.

Motion-to-photon — Total latency from physical head motion to the corresponding photons reaching the eye.

Push-pull — A pyramid-based hole-filling algorithm: downsample with validity weighting, then upsample substituting only into holes.

Reversed-Z — Depth convention mapping the near plane to 1 and the far plane to 0, which pairs with floating-point depth buffers to give far better precision distribution.

Rivalry — Perceptual alternation between the two eyes' images when they cannot be fused.

AFW / Alternate Frame Warp — Per-eye reprojection of the alternate (stale) eye from its render pose to the predicted display pose.

Arena slot collision — Failure mode in which a hand-constructed second view shares a per-frame allocation slot with the engine's own view, producing use-after-free.

Cost map — Per-cell matching-cost output of a hardware optical flow engine, used as a confidence signal.

Eye skew (σ) — Difference in capture time between the two submitted eye images.

Frame graph — A renderer architecture that declares passes and resources, then compiles and executes them, typically with transient allocation and memory aliasing.

Held-tick frame pair — Rendering the two eyes in two consecutive engine frames with simulation frozen between them, so both represent one tick.

Matched eye poses — Warping both eyes to a single common head pose plus their respective offsets, guaranteeing internally consistent stereo at display time.

Pack scale (P) — Tunable horizontal compression applied when packing two eye images into one double-width render target.

Replay — Re-submitting a captured GPU command stream with modified constants, in order to render a second view.

Same-tick stereo — Stereo in which both eye images derive from a single simulation advancement.

Separation / convergence — In geometry-shader stereo injection, the magnitude of the clip-space offset and the view depth at which disparity is zero.

Synchronized sequential — Rendering both eyes one after the other with the simulation tick locked between them, then submitting as a pair.

Tick lock — Preventing simulation advancement between two render invocations.

Transient resource — A render target allocated from a per-frame heap whose memory is aliased with other resources later in the frame.

View-independent work — Rendering work whose result does not depend on camera placement, and which can in principle be shared between eyes.

Timewarp — See ATW.

Vergence — The inward/outward rotation of the eyes to fixate at a given distance.

View-dependent cost — Rendering work whose cost scales with the number of views rendered.

32 Appendix C: Reading List

The following are the primary sources worth reading in full. They are the foundation on which everything in this document rests.

Reprojection and latency

- Mark, W. R., McMillan, L., Bishop, G. "Post-Rendering 3D Warping." *Symposium on Interactive 3D Graphics*, 1997. The originating paper for depth-based image warping in real-time rendering; still the clearest exposition of the disocclusion problem.
- Van Waveren, J. M. P. "The Asynchronous Time Warp for Virtual Reality on Consumer Hardware." *ACM VRST*, 2016. Practical treatment of timewarp including implementation on constrained hardware.
- Smit, F. A., van Liere, R., Fröhlich, B. "An Image-Warping Architecture for VR." *IEEE VR*, 2007.

Optical flow

- Lucas, B. D., Kanade, T. "An Iterative Image Registration Technique." *IJCAI*, 1981. The foundational gradient-based method.
- Farnebäck, G. "Two-Frame Motion Estimation Based on Polynomial Expansion." *SCIA*, 2003. Dense flow with a practical GPU-friendly formulation.
- Zach, C., Pock, T., Bischof, H. "A Duality Based Approach for Realtime TV-L1 Optical Flow." *DAGM*, 2007.
- Zabih, R., Woodfill, J. "Non-parametric Local Transforms for Computing Visual Correspondence." *ECCV*, 1994. The census transform.
- Sun, D., Yang, X., Liu, M.-Y., Kautz, J. "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume." *CVPR*, 2018. For the learned approach, if you have the inference budget.
- Teed, Z., Deng, J. "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow." *ECCV*, 2020.

Frame interpolation

- Niklaus, S., Mai, L., Liu, F. "Video Frame Interpolation via Adaptive Separable Convolution." *ICCV*, 2017.
- Jiang, H. et al. "Super SloMo: High Quality Estimation of Multiple Intermediate Frames." *CVPR*, 2018. Good treatment of bidirectional warping and occlusion reasoning.
- Bao, W. et al. "Depth-Aware Video Frame Interpolation." *CVPR*, 2019. Directly relevant given that you have a depth buffer.

Perception

- Howard, I. P., Rogers, B. J. *Perceiving in Depth*. Oxford University Press, 2012. The standard reference on binocular vision; volume 2 covers stereopsis in depth.
- Blake, R., Logothetis, N. “Visual Competition.” *Nature Reviews Neuroscience*, 2002. On binocular rivalry.
- Hoffman, D. M. et al. “Vergence-Accommodation Conflicts Hinder Visual Performance and Cause Visual Fatigue.” *Journal of Vision*, 2008.
- Allison, R. S., Wilcox, L. M. “Perceptual Tolerance to Stereoscopic 3D Image Distortion.” *ACM Transactions on Applied Perception*, 2015. Directly relevant to how much disparity error you can get away with.

Graphics APIs and reference

- The OpenXR Specification, Khronos Group. Chapters on the frame loop, view configuration, and composition layers are the normative reference for everything in Chapter 4.
- The Vulkan Specification, Khronos Group. Synchronisation chapter, especially the queue family ownership transfer rules.
- Microsoft Direct3D 12 programming guide, particularly the multi-engine and resource-state sections.
- Reed, N. “Depth Precision Visualized.” 2015. The clearest explanation of why reversed-Z is the correct default.

33 Appendix D: Reference Warp Implementation

This appendix gives a complete, annotated implementation of the unified warp from Chapter 12 — the single operation that serves the stereo warp, alternate frame warp, and frame generation. It is written for D3D12/HLSL; the Vulkan/GLSL translation is mechanical.

33.1 Host-side parameter construction

```
struct UnifiedWarpConstants {
    float srcToDst[16];    // clip-space reprojection, row-major
    float invSrcVP[16];    // inverse of source view-projection
    float dstVP[16];       // destination view-projection

    float zNear;
    float srcP22, srcP32;  // for view-space depth reconstruction
    float pad0;

    float phase;           // temporal blend: 0 = pure stereo/pose warp
    float flowScale;       // units conversion for the residual flow texture
    float stretchThreshold; // Jacobian ratio above which fragments are culled
    float foveationPeriphery;

    float invSrcSize[2];
    float invDstSize[2];

    float opticalCentreUv[2];
    float aspectCorrect[2];

    uint32_t flags;        // bit 0: has residual flow
                           // bit 1: has fill source
                           // bit 2: rotation-only (use gather path)
                           // bit 3: reversed-Z

    uint32_t gridSize;
    uint32_t pad1[2];
};

static Mat4 BuildReprojection(const Mat4& srcView, const Mat4& srcProj,
                             const Mat4& dstView, const Mat4& dstProj)
{
    // Invert using structure, never a general 4x4 inverse.
    const Mat4 invSrcView = InvertRigid(srcView);    //  $R^T, -t R^T$ 
    const Mat4 invSrcProj = InvertPerspective(srcProj); // closed form
    return invSrcProj * invSrcView * dstView * dstProj;
}

static Mat4 InvertRigid(const Mat4& m) {
    Mat4 r = Mat4Identity();
    // Transpose the rotation part.
```

```

for (int i = 0; i < 3; ++i)
    for (int j = 0; j < 3; ++j)
        r[i][j] = m[j][i];
// -t * R^T
const Vec3 t{m[3][0], m[3][1], m[3][2]};
r[3][0] = -(t.x*r[0][0] + t.y*r[1][0] + t.z*r[2][0]);
r[3][1] = -(t.x*r[0][1] + t.y*r[1][1] + t.z*r[2][1]);
r[3][2] = -(t.x*r[0][2] + t.y*r[1][2] + t.z*r[2][2]);
return r;
}

static Mat4 InvertPerspective(const Mat4& p) {
    // Reversed-Z infinite form:
    // [a 0 0 0]      [1/a 0 0 0]
    // [0 b 0 0] -> [0 1/b 0 0]
    // [c d 0 -1]     [0 0 0 -1]
    // [0 0 n 0]      [c/a d/b 1/n 0]
    Mat4 r = Mat4Zero();
    r[0][0] = 1.0f / p[0][0];
    r[1][1] = 1.0f / p[1][1];
    r[2][3] = -1.0f;
    r[3][0] = p[2][0] / p[0][0];
    r[3][1] = p[2][1] / p[1][1];
    r[3][2] = 1.0f / p[3][2];
    return r;
}

```

InvertPerspective above assumes the reversed-Z infinite-far form from Chapter 4. For the finite-far variant, the last two rows change; derive them by direct inversion of the sparse matrix rather than calling a general routine, because the general routine's conditioning is poor when $z_n \ll z_f$.

33.2 The warp vertex shader

```

cbuffer WarpCB : register(b0) { UnifiedWarpConstants g; }

Texture2D<float4> gSrcColor : register(t0);
Texture2D<float> gSrcDepth : register(t1);
Texture2D<float2> gFlow : register(t2); // residual only, may be unbound
Texture2D<float> gFlowConf : register(t3);

SamplerState gLinear : register(s0);
SamplerState gPoint : register(s1);

#define FLAG_HAS_FLOW (1u << 0)
#define FLAG_HAS_FILL (1u << 1)
#define FLAG_ROTATION_ONLY (1u << 2)
#define FLAG_REVERSED_Z (1u << 3)

struct VSOut {
    float4 pos : SV_Position;
    float2 srcUv : TEXCOORD0;
    float conf : TEXCOORD1;
    float viewZ : TEXCOORD2;
};

float ViewDepthFromBuffer(float d) {
    // Reversed-Z infinite: z = near / d. Finite: z = P32 / (d - P22).
    if (g.srcP22 == 0.0f) return g.zNear / max(d, 1e-7f);
    return g.srcP32 / (d - g.srcP22);
}

VSOut WarpVS(uint vid : SV_VertexID) {
    // Grid vertex -> source UV.
    const uint gridW = (uint)(1.0f / (g.invSrcSize.x * g.gridStride)) + 1u;

```

```

const uint2 cell = uint2(vid % gridW, vid / gridW);
const float2 srcUv = float2(cell) * g.gridStride * g.invSrcSize;

const float d = gSrcDepth.SampleLevel(gPoint, srcUv, 0);

VSOut o;
o.srcUv = srcUv;
o.conf = 1.0f;
o.viewZ = ViewDepthFromBuffer(d);

// Degenerate: no geometry here (cleared depth). Push off-screen.
const bool empty = (g.flags & FLAG_REVERSED_Z) ? (d <= 0.0f) : (d >= 1.0f);
if (empty) { o.pos = float4(0, 0, 0, 0); return o; }

// 1. Geometric reprojection: source clip -> destination clip.
float4 clip = float4(srcUv.x * 2.0f - 1.0f,
                    1.0f - srcUv.y * 2.0f,
                    d, 1.0f);
float4 dstClip = mul(clip, g.srcToDst);

// 2. Residual scene motion, applied in destination NDC.
if (g.flags & FLAG_HAS_FLOW) {
    const float2 f = gFlow.SampleLevel(gLinear, srcUv, 0) * g.flowScale;
    o.conf = gFlowConf.SampleLevel(gLinear, srcUv, 0);
    // Flow is in UV units; convert to NDC and scale by temporal phase.
    dstClip.xy += float2(f.x * 2.0f, -f.y * 2.0f) * g.phase * dstClip.w;
}

o.pos = dstClip;
return o;
}

```

Three details worth highlighting.

The `dstClip.w` multiply on the flow term. The flow displacement is specified in NDC, but you are writing a clip-space position that will be divided by w . Scaling by w before output makes the post-divide displacement equal the intended NDC offset. Omitting this makes flow displacement depth-dependent, which is wrong — screen-space flow is already a screen-space quantity.

The empty-depth test. Cleared depth means no geometry. Emitting a vertex there produces a fragment at the far plane that will occlude nothing but will mark the pixel as “valid”, defeating the hole detection. Push it to a degenerate position instead.

Point sampling the depth. Bilinear depth sampling across a discontinuity produces an intermediate depth that belongs to no surface, placing the vertex in mid-air between foreground and background. Always point-sample depth for warping.

33.3 The warp pixel shader

```

float4 WarpPS(VSOut i) : SV_Target {
    // Screen-space derivatives of the source UV give the local Jacobian,
    // i.e. how much this quad has been stretched by the warp.
    const float2 dx = ddx(i.srcUv);
    const float2 dy = ddy(i.srcUv);
    const float jac = abs(dx.x * dy.y - dx.y * dy.x);

    const float nominal = (g.gridStride * g.invSrcSize.x)
                        * (g.gridStride * g.invSrcSize.y);
    const float areaRatio = jac / max(nominal, 1e-12f);

    // Foveation: relax the stretch test in the periphery, where a mild
    // smear is less objectionable than a hole.
    const float2 centred = (i.srcUv - g.opticalCentreUv) * g.aspectCorrect;
    const float radial = saturate(length(centred) * 2.0f);
    const float fovRelax = lerp(1.0f, 2.5f,
        saturate((radial - (1.0f - g.foveationPeriphery))
            / max(g.foveationPeriphery, 1e-3f)));

    const float threshold = g.stretchThreshold * fovRelax;
    const float stretch = saturate((areaRatio - threshold) / (threshold * 3.0f));
}

```

```

// Hard cull badly stretched fragments: a hole the fill stage can handle
// beats a smear across a disocclusion.
clip(0.95f - stretch);

float4 c = gSrcColor.SampleLevel(gLinear, i.srcUv, 0);

// Alpha carries combined validity: 1 = fully trusted, 0 = discard.
c.a = saturate((1.0f - stretch) * i.conf);
return c;
}

```

33.4 The rotation-only gather path

When head translation over the warp interval is negligible, the mapping is a depth-independent homography and can be inverted in closed form. This path is cheaper, exact, and produces no holes.

```

Texture2D<float4> gSrc : register(t0);
RWTexture2D<float4> gDst : register(u0);

cbuffer RotWarpCB : register(b0) {
    float4x4 gDstToSrcHomography; // destination NDC -> source NDC
    float2 gInvDstSize;
    float2 gPad;
}

[numthreads(8, 8, 1)]
void RotationOnlyWarp(uint3 tid : SV_DispatchThreadID) {
    const float2 dstUv = (float2(tid.xy) + 0.5f) * gInvDstSize;
    float4 ndc = float4(dstUv.x * 2.0f - 1.0f, 1.0f - dstUv.y * 2.0f, 1.0f, 1.0f);
    float4 s = mul(ndc, gDstToSrcHomography);

    const float2 srcUv = float2( (s.x / s.w) * 0.5f + 0.5f,
                                0.5f - (s.y / s.w) * 0.5f );

    // Outside the source frustum: this is the off-screen intake region.
    // Chapter 11's render margin is what makes this rare.
    if (any(srcUv < 0.0f) || any(srcUv > 1.0f)) {
        gDst[tid.xy] = float4(0, 0, 0, 0);
        return;
    }
    gDst[tid.xy] = gSrc.SampleLevel(gLinearClamp, srcUv, 0);
}

```

Build the homography by dropping the translation from the reprojection:

```

Mat4 BuildRotationHomography(const Quat& qSrc, const Quat& qDst, const Mat4& proj)
{
    const Mat4 rSrc = QuatToMatrix(qSrc);
    const Mat4 rDst = QuatToMatrix(qDst);
    // Destination NDC -> source NDC.
    return InvertPerspective(proj) * Transpose(rDst) * rSrc * proj;
}

```

33.5 Dispatch

```

void ExecuteWarp(CommandContext& ctx, const WarpParams& p, GpuTexture& out)
{
    const bool rotationOnly =
        Length(p.dstEyePosition - p.srcEyePosition) < kTranslationEpsilon
        && p.residualFlow == nullptr;

    if (rotationOnly) {
        UpdateRotWarpCB(ctx, p);
        ctx.SetPipeline(g_rotationOnlyPso);
        ctx.Dispatch((out.width + 7) / 8, (out.height + 7) / 8, 1);
        return;
    }
}

```

```

}

UpdateUnifiedWarpCB(ctx, p);

ctx.ClearRenderTarget(out, {0, 0, 0, 0});
ctx.ClearDepth(g_warpDepth, 0.0f);           // reversed-Z: 0 = far/empty
ctx.SetRenderTargets(&out, 1, &g_warpDepth);
ctx.SetPipeline(g_warpGridPso);               // depth GREATER_EQUAL, write on

const uint32_t gw = (p.srcWidth / p.gridStride) + 1;
const uint32_t gh = (p.srcHeight / p.gridStride) + 1;
ctx.DrawIndexed(g_gridIndexCount[p.gridStride], /*instances=*/1);

if (p.fillColor) {
    // Second source: warp the fill and composite where the primary
    // left holes. Reuses the same P50 with different bindings.
    UpdateUnifiedWarpCB_Fill(ctx, p);
    ctx.SetRenderTargets(&g_fillTarget, 1, &g_fillDepth);
    ctx.ClearRenderTarget(g_fillTarget, {0,0,0,0});
    ctx.ClearDepth(g_fillDepth, 0.0f);
    ctx.DrawIndexed(g_gridIndexCount[p.gridStride], 1);

    ctx.SetPipeline(g_compositePso);
    ctx.Dispatch((out.width + 7) / 8, (out.height + 7) / 8, 1);
}

RunPushPullInpaint(ctx, out, g_warpDepth, p.inpaintLevels);
}

```

33.6 Grid index buffer construction

Build once at initialisation, one per supported stride.

```

std::vector<uint32_t> BuildGridIndices(uint32_t gw, uint32_t gh) {
    std::vector<uint32_t> idx;
    idx.reserve((gw - 1) * (gh - 1) * 6);
    for (uint32_t y = 0; y + 1 < gh; ++y) {
        for (uint32_t x = 0; x + 1 < gw; ++x) {
            const uint32_t a = y * gw + x;
            const uint32_t b = a + 1;
            const uint32_t c = a + gw;
            const uint32_t d = c + 1;
            idx.insert(idx.end(), {a, c, b, b, c, d});
        }
    }
    return idx;
}

```

At stride 2 on a 2000x2200 source this is 1000x1100 vertices and about 6.6 M indices. Consider a triangle strip with primitive restart, or better, drop the index buffer entirely and derive the grid position from `SV_VertexID` with a `DrawInstanced` of $(gw-1)*(gh-1)*6$ vertices — avoids 26 MB of index bandwidth per warp.

34 Appendix E: Configuration Reference

The complete set of user- and community-tunable parameters, with recommended defaults and the section that explains each.

34.1 World and camera

Key	Default	Range	Chapter
<code>world.metres_per_unit</code>	1.0	0.01–100	7
<code>world.up_axis</code>	y	y, z	5
<code>world.handedness</code>	right	left, right	5
<code>camera.take_over_orientation</code>	true	bool	7
<code>camera.ipd_override_mm</code>	0 (use runtime)	50–75	6
<code>camera.near_plane_override</code>	0 (auto)	0.005–0.5	4

Key	Default	Range	Chapter
camera.render_margin_fraction	0.06	0.0–0.20	11

world.metres_per_unit is the single most important setting. Everything downstream — disparity, disocclusion width, comfort — scales with it.

34.2 Mode selection

Key	Default	Notes
mode.preferred	auto	or an explicit mode name
mode.enable_sync_sequential	true	requires verified tick lock
mode.enable_generation	true	
mode.enable_async_aer	false	opt-in; needs skew management
mode.min_dwell_frames	45	anti-oscillation
mode.ascend_frames	90	slow ascend
mode.descend_frames	3	fast descend

34.3 Tick lock

Key	Default	Notes
ticklock.method	timer_thread_scoped	OR timer_global, dt_patch, tick_skip
ticklock.epsilon_seconds	1e-6	avoids watchdog trips (F4)
ticklock.verify_on_start	true	runs the differential test
ticklock.skip_sim_dispatches	true	see Chapter 23
ticklock.share_shadow_maps	true	6–12% saving

34.4 Alternate-eye and warp

Key	Default	Notes
aer.policy	dithered	alternate, dithered, dominant, adaptive
aer.dominant_eye	left	for dominant policy
aer.full_stereo_speed_threshold	1.2	m/s, for adaptive
warp.enable_afw	true	leave on
warp.enable_stereo_warp	true	
warp.grid_stride	2	1–8 px per grid vertex
warp.stretch_threshold	2.5	Jacobian area ratio
warp.translation_epsilon_mm	2.0	below this, rotation-only path
warp.dual_source_fill	true	highest-value refinement
warp.boundary_blend_px	3	anti-halo
warp.inpaint_levels	5	
warp.z_split_metres	0 (auto)	near-field both-eyes cutoff

34.5 Frame generation

Key	Default	Notes
gen.mode	extrapolate	off, extrapolate, interpolate
gen.ratio	auto	or an integer
gen.max_phase	1.5	refuse to extrapolate further
gen.flow_source	auto	hardware, compute, camera_only
gen.flow_resolution_scale	0.5	half-res flow by default
gen.flow_transfer_between_eyes	true	halves flow cost
gen.flow_smooth	0.0	0–0.6 robust spatial low-pass
gen.flow_grid_bilinear	true	interpolate across cells; off = shimmer
gen.flow_perf_level	fast	fast, medium, slow (needs input downscale)

Key	Default	Notes
gen.flow_input_scale	1.0	downscale factor for the flow engine input
gen.warp_formulation	gather	gather, scatter for temporal synthesis
gen.refine_strength	1.0	agreement-gated fusion strength
gen.occlusion_sharpen	1.0	photometric disagreement sharpening
gen.mv_agree_threshold	0.50	cosine floor for engine MV fusion
gen.pose_agree_threshold	0.60	cosine floor for pose fusion
gen.max_object_shift_uv	0.04	object-flow fade threshold in extrapolation
gen.max_camera_shift_uv	0.15	camera reprojection clamp
gen.use_engine_motion_vectors	true	if found
gen.acceleration_model	false	usually worse; see Ch. 11

34.6 Foveation

Key	Default	Notes
fov.periphery_fraction	0.0	0 = off; 0.35–0.5 typical
fov.mode	two_zone	two_zone, graduated
fov.feather	0.08	boundary blend band, fraction of radius
fov.block_min_px	4	periphery block size just outside sweet spot
fov.block_max_px	16	periphery block size at lens edge
fov.use_eye_tracking	false	if runtime exposes gaze
fov.gaze_prediction_ms	25	
fov.outer_grid_stride	6	

34.7 UI

Key	Default	Notes
ui.mode	quad_layer	quad_layer, composited, off
ui.distance_metres	2.0	
ui.width_metres	2.2	
ui.lag_tau_ms	150	0 = rigid head lock
ui.max_deviation_degrees	15	

34.8 Diagnostics

Key	Default	Notes
diag.overlay	false	in-headset stats
diag.log_census	true	
diag.census_file	<app>.census	
diag.capture_reference_frames	false	for the validation harness
diag.pass_overrides	(none)	manual classification
diag.fusion_stats	false	agreement-gate acceptance telemetry

35 Appendix F: Symptom to Cause

A diagnostic index. Find the symptom, read across.

35.1 Geometry and stereo

Symptom	Likely cause	Check
World feels like a dollhouse or giant's playground	world.metres_per_unit wrong	Measure a known object's angular size

Symptom	Likely cause	Check
Eyes will not fuse; two different images Parallax wrong near screen edges	Projection or view matrix wrong for one eye; or SBS packing on a deferred renderer Frustum shift dropped from the projection	Disable stereo; verify mono is correct first Chapter 4; verify <code>m[2][0]</code> , <code>m[2][1]</code>
World shears in depth during strafing Depth feels flat Depth feels exaggerated / eye strain Shadows broken or absent	AER disparity error (A1) IPD too small, or world scale too large IPD too large, or world scale too small Shadow-map projection being patched with the eye transform	Chapter 6; enable AFW and stereo warp Chapter 5 classifier
Skybox has incorrect parallax Everything doubled	Translation not stripped from the skybox transform Both eyes rendering to the same target, or geometry duplication without packing	Chapter 5

35.2 Motion and temporal

Symptom	Likely cause	Check
Edges buzz or shimmer Ghost trails behind moving objects	Inter-ocular judder (A2) TAA motion vector scale not adjusted for AER	Chapter 7; enable warp Chapter 33; scale by frame span
Doubled ghost trails Ghosting in front of objects Fine shimmer at a regular period World stutters between smooth and jumpy	Same, factor of exactly 2 Motion vector sign inverted Jitter double-correction or missing correction Generated frames submitted with source pose	Chapter 28 calibration Chapter 28 static test F14
Rotor / fan appears to crawl differently in each eye Everything judders uniformly Ghosting worsens over time, never recovers	Flicker resonance (A7) Half-rate mode active, or missing display deadlines Flow cache key stale	Chapter 7; dithered schedule Check the mode overlay F11
Crawling speckle along object silhouettes Semi-transparent double image on moving edges	Nearest-cell flow sampling; no bilinear Photometric disagreement stage disabled	Chapter 27 <code>gen.occlusion_sharpen</code>
Long smears during fast head turns World tears and judders on turns, static scenes fine World bulges or pincushions during motion	Object flow not faded in extrapolation Camera reprojection faded like object flow Wrong projection matrix passing the pose gate	Chapter 38 Chapter 38: apply camera term fully Raise <code>gen.pose_agree_threshold</code>
Streaked border during turns	Extrapolation clamping to edge instead of reverting	Chapter 38

35.3 Synthesis artifacts

Symptom	Likely cause	Check
Smeared bands beside objects Halo outline around occluders Blurry patches beside near objects	Stretch culling disabled or threshold too high Dual-source boundary not blended Disocclusion too wide for inpainting	<code>warp.stretch_threshold</code> <code>warp.boundary_blend_px</code> Enable near-field both-eyes; lower <code>z_split</code> <code>camera.render_margin_fraction</code>
Band of garbage at the frame edge during turns Reprojection moves the wrong way Nearby objects move less than distant ones	Off-screen intake, no render margin Depth convention wrong Reversed-Z assumed but not in use	 F10
First-person weapon torn down one side Sharpness pulses at half display rate	Near field warped rather than rendered both eyes Rendered/warped filtering mismatch	Chapter 36 Chapter 33

35.4 Performance and stability

Symptom	Likely cause	Check
Hard performance wall around 35–40 fps	Per-frame interop import, not cached	F15; Chapter 27
Imported textures read as pure black	Texture-object path on external memory	Use the surface-read path, Chapter 27
Warped image horizontally compressed	Texel index passed where a byte offset is required	Chapter 27
Persistent half-pixel softness	Missing half-texel offset on manual bilinear	Sample at $uv * size - 0.5$
Both eyes' flow subtly degraded	One flow session shared between eyes	Allocate per-eye sessions
Slow flow preset misses the frame budget	Preset raised without input downscale	<code>gen.flow_input_scale</code>
Frame time spikes at a regular interval	Blocking on flow engine execute	Chapter 27 non-blocking path
Progressive memory growth	Import cache never cleared, or ring never recycled	F18; dwell time and hysteresis
Quality pulses at 1–2 Hz	Mode oscillation	F6; use timer freeze, not tick skip
Hang on enabling same-tick mode	Job system waiting on a skipped simulation job	F1; thread-scoped freeze
Audio stutters when stereo enabled	Global time freeze	F2
Physics objects launch on mode change	Timestep discontinuity	F4; epsilon timestep
Process exits or reports not responding	Watchdog on frozen simulation	F8
Frozen image while game is clearly running	Ring exhaustion in async AER	F9
Rare single-frame corruption	Torn slot publication	Chapter 25
Crash several frames after enabling second view	Arena slot collision	F19
“Worked yesterday”, now broken	Capability regression not detected	

35.5 UI

Symptom	Likely cause	Check
HUD smears or doubles	UI included in the warp	Chapter 13; extract to a layer
HUD swims at a random depth	UI drawn with scene depth under it	Use a quad layer
Small text illegible	UI composited into the projection layer and resampled	<code>ui.lag_tau_ms</code>
HUD whips around during turns	Rigid head lock	Chapter 13
Dark fringes around UI elements	Premultiplied alpha mismatch	Detect and bypass
Loading screens render as garbage	Screen-space stereo path active when it should not be	

36 Appendix G: Test Scene Specifications

The validation methodology in Chapter 17 depends on scenes that isolate individual artifacts. These specifications are precise enough to reproduce, either inside a target application or in a standalone harness.

36.1 T1 — Depth-staggered poles (disparity shear)

Vertical cylinders, radius 0.05 m, height 3 m, at distances 1, 2, 4, 8, 16, 32 m along the view axis, offset laterally by ± 0.4 m alternately so none occludes another. Uniform matte grey, flat lighting, no specular. Background: a featureless plane at 200 m.

Motion: pure lateral translation at 1.0 m/s for 4 s, reversing.

Measurement: per-pole horizontal disparity, measured by cross-correlating a horizontal scanline through each pole between the two eyes. Compare to the analytic $bP_{00}W/(2z)$. Report the maximum relative error across poles.

Pass: relative disparity error below 5% at all depths throughout the motion.

36.2 T2 — Oblique checkerboard (binocular rivalry)

A single plane, 50 m square, tilted 75 degrees from the view axis so it recedes to a horizon. Checker size 0.25 m, pure black and white, no filtering beyond the renderer's default.

Motion: yaw pan at 30 deg/s.

Measurement: inter-ocular SSIM computed in a band covering the region where checker period falls between 2 and 8 pixels. Compare against a full-stereo reference.

Pass: $Q_{\text{stereo}} > 0.95$.

36.3 T3 — Polished sphere (specular breakup)

A sphere of radius 0.5 m at 3 m, roughness 0.05, metallic, under a single point light at 45 degrees. Matte background.

Motion: camera orbit at 20 deg/s at constant radius.

Measurement: centroid position of the specular highlight in each eye; compare the inter-ocular offset to the analytic expectation for the surface curvature.

Pass: highlight centroid inter-ocular offset within 2 pixels of reference.

36.4 T4 — Reflective floor with edge-crossing object (screen-space effects)

A mirror-like floor plane with screen-space reflections enabled. A 1 m cube at 4 m, 1.5 m above the floor, translating laterally so its reflection crosses the frame edge.

Motion: object lateral translation at 2 m/s.

Measurement: reflection presence per eye as a binary mask; count frames where the two eyes disagree.

Pass: disagreement in fewer than 2% of frames.

36.5 T5 — Fine foliage (temporal antialiasing)

A dense cluster of alpha-tested foliage cards, roughly 200 quads within a 5 m volume at 6 m, with sub-pixel detail.

Motion: slow forward dolly at 0.5 m/s with 5 deg/s yaw.

Measurement: temporal difference metric (mean absolute difference between consecutive frames) per eye, compared to a full-rate stereo reference.

Pass: within 20% of reference. Substantially higher indicates history rejection; substantially lower indicates over-smoothing or a held frame.

36.6 T6 — Strobe and rotor (flicker resonance)

A point light strobing at exactly $f_a/2$, and a four-bladed rotor at 1.5 revolutions per second, both at 5 m.

Motion: static camera.

Measurement: per-eye luminance time series at the strobe location; cross-correlate the two eyes. Beat frequencies below 5 Hz indicate resonance.

Pass: no inter-ocular beat component above the noise floor.

36.7 T7 — Near pole against distant wall (disocclusion)

A single cylinder, radius 0.08 m, at 0.5 m, against a textured wall at 12 m. The worst case for the stereo warp.

Motion: static, then 0.3 m/s lateral.

Measurement: disocclusion area fraction — the proportion of the synthesised eye's pixels filled by inpainting rather than warped from valid source data.

Pass: below 3% with dual-source filling enabled. Report the value with and without near-field both-eyes rendering to quantify that feature's benefit.

36.8 T8 — Fast small object over flat background (flow failure)

A 0.2 m sphere crossing the frame horizontally at 12 m/s at 5 m distance, over an untextured background.

Motion: object only.

Measurement: positional error of the object's centroid in generated frames versus a rendered reference at the same timestamp.

Pass: within 4 pixels. This test specifically exercises the flow estimator's search range and low-texture behaviour; failures here point at pyramid depth.

36.9 T9 — Rapid turn (off-screen intake)

Static scene with high-frequency detail everywhere.

Motion: 180-degree yaw over 0.6 s (300 deg/s), starting and stopping abruptly.

Measurement: width in pixels of the invalid band along the leading frame edge.

Pass: zero, given a correctly sized render margin. Non-zero directly measures the margin shortfall.

36.10 T10 — Dense HUD (UI handling)

A full-screen overlay with 8 pt text, thin 1 px rules, and a minimap with fine detail, alpha-blended over the scene.

Motion: 90 deg/s yaw.

Measurement: text legibility, scored by an OCR pass over a captured eye image; and inter-ocular SSIM restricted to the HUD

region.

Pass: OCR accuracy within 2% of a static-camera baseline; HUD-region $Q_{\text{stereo}} > 0.99$ (the UI should be essentially identical between eyes).

36.11 Harness structure

```
struct TestScene {
    const char* name;
    void (*Build)(SceneBuilder&);
    Mat4 (*CameraAt)(double t);
    double durationSeconds;
    MetricResult (*Measure)(const CaptureSet& out, const CaptureSet& ref);
    float passThreshold;
};

struct RunResult {
    const char* scene;
    const char* mode;
    float value;
    bool passed;
    double meanGpuMs, p99GpuMs;
};

std::vector<RunResult> RunSuite(const std::vector<TestScene>& scenes,
                               const std::vector<StereoMode>& modes)
{
    std::vector<RunResult> results;
    for (const TestScene& s : scenes) {
        SetMode(StereoMode::Mono);
        SetFullStereoReference(true);
        const CaptureSet ref = CaptureRun(s); // ground truth
        SetFullStereoReference(false);

        for (StereoMode m : modes) {
            SetMode(m);
            WarmUp(30); // let TAA and caches settle
            const CaptureSet out = CaptureRun(s);
            const MetricResult r = s.Measure(out, ref);
            results.push_back({s.name, ModeName(m), r.value,
                              r.value <= s.passThreshold,
                              r.meanGpuMs, r.p99GpuMs});
        }
    }
    return results;
}
```

Two details that matter for reproducibility. **Warm up before capturing:** temporal accumulation, streaming, and shader compilation all take tens of frames to settle, and capturing during that window produces noisy results that will send you chasing phantom regressions. **Capture the reference with full stereo at full rate,** driven by the same scripted camera — not from a previous run, and not from a different build, because renderer changes unrelated to your work will otherwise show up as regressions.

Run the suite on every build. The metrics that matter most as regression tripwires are T1’s disparity error, T7’s disocclusion fraction, and T2’s inter-ocular SSIM; those three catch the overwhelming majority of introduced defects. # Closing Notes

Three things determine whether an implementation of this material succeeds.

First: measure before you build. The cost model in Chapter 2 tells you whether AER can help at all on your target. If the application is CPU-bound or dominated by view-independent work, you will do a great deal of engineering for very little gain. A day of profiling can save a month.

Second: the warp kernel is the whole system. Chapters 8, 11, and 12 all reduce to one parameterised operation. Build it once, build it well, test it in isolation with synthetic inputs where you know the ground truth, and everything downstream becomes tractable. Implementations that grow three separate warp paths end up with three separate sets of bugs.

Third: the artifact you cannot fix is the one that defines the experience. Disocclusion in the near field is fundamental — no amount of cleverness reconstructs data that was never rendered. Design around it by rendering the near field properly rather than by escalating the sophistication of your inpainting. The same applies to off-screen intake: render a margin rather than

hallucinating one.

The techniques here are not exotic. Depth-based reprojection is nearly thirty years old, hierarchical block matching is older, and every element has been independently implemented many times. What is hard is the integration: doing all of it, correctly, inside a frame budget, in an application whose renderer you cannot see. That is an engineering problem rather than a research problem, and it yields to careful measurement and incremental validation.